

VHDL pour l'ingénierie Hardware & FPGA

VHDL for Hardware & FPGA Engineering - Digital Logic, RTL, Architecture, Processes, FSM, CDC, Interfaces, Simulation, Synthesis & Timing - FR / EN

500 questions/réponses orientées entretien et pratique. Le document est volontairement vendor-independent: il explique le langage, le RTL et la logique numérique sans dépendre de Quartus, Vivado ou d'un FPGA particulier.

Approche: toujours relier le code VHDL au matériel décrit - portes, multiplexeurs, registres, mémoires, machines d'état, crossings et chemins de timing. Les fragments de code sont inclus lorsqu'ils clarifient la structure ou un piège d'entretien.

Note: `process(all)` et certaines constructions modernes supposent VHDL-2008. Les contraintes de pins et de timing sont volontairement abordées conceptuellement, car leur syntaxe dépend du flow FPGA.

Structure du document / Document Structure

1. Fondements HDL, FPGA et RTL / HDL, FPGA and RTL Foundations
2. Structure d'un fichier VHDL et unités de conception / VHDL File Structure and Design Units
3. Ports, directions, pins et interfaces externes / Ports, Directions, Pins, and External Interfaces
4. Architecture, concurrence et hiérarchie de base / Architecture, Concurrency, and Basic Hierarchy
5. Types, sous-types et littéraux / Types, Subtypes, and Literals
6. Opérateurs, expressions et concaténation / Operators, Expressions, and Concatenation
7. Signals, variables, constants et delta cycles / Signals, Variables, Constants, and Delta Cycles
8. Affectations concurrentes et style dataflow / Concurrent Assignments and Dataflow Style
9. Process, sensitivity list et statements séquentiels / Processes, Sensitivity Lists, and Sequential Statements
10. Logique combinatoire: mux, décodeurs et comparateurs / Combinational Logic: Muxes, Decoders, and Comparators
11. Logique séquentielle: flip-flops et registres / Sequential Logic: Flip-Flops and Registers
12. Horloges, resets, enables et détection de fronts / Clocks, Resets, Enables, and Edge Detection
13. Compteurs, timers, diviseurs et PWM / Counters, Timers, Dividers, and PWM
14. Machines d'état FSM / Finite State Machines
15. Generics, generate et matériel réutilisable / Generics, Generate, and Reusable Hardware
16. Hiérarchie avancée, packages, fonctions et procédures / Advanced Hierarchy, Packages, Functions, and Procedures
17. Arithmétique, numeric_std, conversions et overflow / Arithmetic, numeric_std, Conversions, and Overflow
18. ROM, RAM, tableaux et FIFO / ROM, RAM, Arrays, and FIFOs
19. CDC, métastabilité et synchroniseurs / CDC, Metastability, and Synchronizers
20. Entrées externes, debounce, tri-state et open-drain / External Inputs, Debounce, Tri-State, and Open-Drain
21. Blocs de protocole UART, SPI et I²C / UART, SPI, and I²C Protocol Blocks
22. Testbenches, stimuli et modèles de vérification / Testbenches, Stimulus, and Verification Models
23. Simulation, assertions, résolution et debug de waveform / Simulation, Assertions, Resolution, and Waveform Debug
24. Synthèse, timing, contraintes et portabilité / Synthesis, Timing, Constraints, and Portability
25. Troubleshooting et scénarios d'entretien / Troubleshooting and Interview Design Scenarios

1. Fondements HDL, FPGA et RTL / HDL, FPGA and RTL Foundations

1. FR - Expliquez VHDL et rôle d'un HDL. Quelle structure ou idée hardware se cache derrière ?

Réponse : VHDL est un langage de description matérielle utilisé pour modéliser, simuler et synthétiser de la logique numérique. Il décrit la structure et le comportement d'un circuit plutôt qu'un programme exécuté instruction par instruction.

EN - Explain VHDL and the role of an HDL. What hardware structure or idea is behind it?

Answer: VHDL is a hardware description language used to model, simulate, and synthesize digital logic. It describes circuit structure and behavior rather than a program executed instruction by instruction.

2. FR - Comment utiliseriez-vous VHDL et rôle d'un HDL dans un design VHDL réel ?

Réponse : Je l'utilise pour décrire un compteur, une FSM, une interface UART ou un datapath, puis je simule le RTL et je synthétise le design vers une FPGA/CPLD.

EN - How would you use VHDL and the role of an HDL in a real VHDL design?

Answer: I use it to describe a counter, FSM, UART interface, or datapath, then simulate the RTL and synthesize the design for an FPGA/CPLD.

3. FR - Comment diagnostiqueriez-vous un problème lié à VHDL et rôle d'un HDL ?

Réponse : Je vérifie d'abord si le problème vient de la logique décrite, de l'horloge/reset, des contraintes ou d'une mauvaise compréhension du parallélisme. Je compare la simulation RTL au comportement attendu avant d'accuser l'outil ou le FPGA. Concernant VHDL et rôle d'un HDL, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.

EN - How would you troubleshoot an issue involving VHDL and the role of an HDL?

Answer: I first check whether the problem comes from the described logic, clock/reset, constraints, or a misunderstanding of parallelism. I compare RTL simulation with expected behavior before blaming the tool or FPGA. For VHDL and the role of an HDL, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

4. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant VHDL et rôle d'un HDL ?

Réponse : Je raisonne toujours en matériel: registres, portes, multiplexeurs, mémoire et chemins combinatoires. Le code VHDL décrit cette structure; ce n'est pas une suite d'instructions exécutées par un CPU. Pour VHDL et rôle d'un HDL, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.

EN - What best practice or interview trap should you know regarding VHDL and the role of an HDL?

Answer: I always reason in hardware: registers, gates, multiplexers, memory, and combinational paths. VHDL describes that structure; it is not a sequence of CPU instructions. For VHDL and the role of an HDL, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

5. FR - Expliquez différence entre VHDL et C/C++. Quelle structure ou idée hardware se cache derrière ? Réponse : En C/C++, les instructions s'exécutent normalement de façon séquentielle sur un processeur. En VHDL, plusieurs statements concurrents décrivent du matériel fonctionnant en parallèle et les process synchrones décrivent souvent des registres cadencés.	EN - Explain difference between VHDL and C/C++. What hardware structure or idea is behind it? Answer: In C/C++, instructions normally execute sequentially on a processor. In VHDL, concurrent statements describe hardware operating in parallel, and synchronous processes often describe clocked registers.
6. FR - Comment utiliseriez-vous différence entre VHDL et C/C++ dans un design VHDL réel ? Réponse : Lors d'une revue, je traduis mentalement chaque bloc en matériel: une affectation concurrente peut devenir une porte ou un mux, tandis qu'un process sur <code>rising_edge(clk)</code> infère généralement des registres.	EN - How would you use difference between VHDL and C/C++ in a real VHDL design? Answer: During review, I mentally translate each block into hardware: a concurrent assignment may become a gate or mux, while a <code>rising_edge(clk)</code> process generally infers registers.
7. FR - Comment diagnostiqueriez-vous un problème lié à différence entre VHDL et C/C++ ? Réponse : Je vérifie d'abord si le problème vient de la logique décrite, de l'horloge/reset, des contraintes ou d'une mauvaise compréhension du parallélisme. Je compare la simulation RTL au comportement attendu avant d'accuser l'outil ou le FPGA. Concernant différence entre VHDL et C/C++, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving difference between VHDL and C/C++? Answer: I first check whether the problem comes from the described logic, clock/reset, constraints, or a misunderstanding of parallelism. I compare RTL simulation with expected behavior before blaming the tool or FPGA. For difference between VHDL and C/C++, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
8. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant différence entre VHDL et C/C++ ? Réponse : Je raisonne toujours en matériel: registres, portes, multiplexeurs, mémoire et chemins combinatoires. Le code VHDL décrit cette structure; ce n'est pas une suite d'instructions exécutées par un CPU. Pour différence entre VHDL et C/C++, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding difference between VHDL and C/C++? Answer: I always reason in hardware: registers, gates, multiplexers, memory, and combinational paths. VHDL describes that structure; it is not a sequence of CPU instructions. For difference between VHDL and C/C++, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
9. FR - Expliquez FPGA, CPLD et logique programmable. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FPGA contient des ressources logiques configurables, des flip-flops, mémoires, interconnexions et souvent des blocs spécialisés. Une CPLD est généralement plus petite et organisée différemment, mais les deux peuvent être décrites avec VHDL.	EN - Explain FPGA, CPLD, and programmable logic. What hardware structure or idea is behind it? Answer: An FPGA contains configurable logic, flip-flops, memories, interconnect, and often specialized blocks. A CPLD is generally smaller and organized differently, but both can be described with VHDL.
10. FR - Comment utiliseriez-vous FPGA, CPLD et logique programmable dans un design VHDL réel ? Réponse : Je garde le RTL portable autant que possible et je réserve les primitives spécifiques au constructeur aux fonctions qui exigent réellement une ressource particulière.	EN - How would you use FPGA, CPLD, and programmable logic in a real VHDL design? Answer: I keep RTL portable whenever possible and reserve vendor-specific primitives for functions that genuinely require a particular resource.
11. FR - Comment diagnostiqueriez-vous un problème lié à FPGA, CPLD et logique programmable ? Réponse : Je vérifie d'abord si le problème vient de la logique décrite, de l'horloge/reset, des contraintes ou d'une mauvaise compréhension du parallélisme. Je compare la simulation RTL au comportement attendu avant d'accuser l'outil ou le FPGA. Concernant FPGA, CPLD et logique programmable, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving FPGA, CPLD, and programmable logic? Answer: I first check whether the problem comes from the described logic, clock/reset, constraints, or a misunderstanding of parallelism. I compare RTL simulation with expected behavior before blaming the tool or FPGA. For FPGA, CPLD, and programmable logic, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
12. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FPGA, CPLD et logique programmable ? Réponse : Je raisonne toujours en matériel: registres, portes, multiplexeurs, mémoire et chemins combinatoires. Le code VHDL décrit cette structure; ce n'est pas une suite d'instructions exécutées par un CPU. Pour FPGA, CPLD et logique programmable, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding FPGA, CPLD, and programmable logic? Answer: I always reason in hardware: registers, gates, multiplexers, memory, and combinational paths. VHDL describes that structure; it is not a sequence of CPU instructions. For FPGA, CPLD, and programmable logic, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

13. FR - Expliquez RTL et niveau d'abstraction. Quelle structure ou idée hardware se cache derrière ? Réponse : RTL signifie Register Transfer Level. On décrit les registres, les transferts de données entre eux et la logique combinatoire qui calcule les prochaines valeurs à chaque cycle d'horloge.	EN - Explain RTL and abstraction level. What hardware structure or idea is behind it? Answer: RTL means Register Transfer Level. It describes registers, transfers of data between them, and combinational logic that computes next values each clock cycle.
14. FR - Comment utiliseriez-vous RTL et niveau d'abstraction dans un design VHDL réel ? Réponse : Pour une FSM, je sépare typiquement le registre d'état de la logique de prochain état et des sorties afin que la structure matérielle soit explicite et vérifiable.	EN - How would you use RTL and abstraction level in a real VHDL design? Answer: For an FSM, I typically separate the state register from next-state and output logic so the hardware structure is explicit and verifiable.
15. FR - Comment diagnostiqueriez-vous un problème lié à RTL et niveau d'abstraction ? Réponse : Je vérifie d'abord si le problème vient de la logique décrite, de l'horloge/reset, des contraintes ou d'une mauvaise compréhension du parallélisme. Je compare la simulation RTL au comportement attendu avant d'accuser l'outil ou le FPGA. Concernant RTL et niveau d'abstraction, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving RTL and abstraction level? Answer: I first check whether the problem comes from the described logic, clock/reset, constraints, or a misunderstanding of parallelism. I compare RTL simulation with expected behavior before blaming the tool or FPGA. For RTL and abstraction level, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
16. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant RTL et niveau d'abstraction ? Réponse : Je raisonne toujours en matériel: registres, portes, multiplexeurs, mémoire et chemins combinatoires. Le code VHDL décrit cette structure; ce n'est pas une suite d'instructions exécutées par un CPU. Pour RTL et niveau d'abstraction, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding RTL and abstraction level? Answer: I always reason in hardware: registers, gates, multiplexers, memory, and combinational paths. VHDL describes that structure; it is not a sequence of CPU instructions. For RTL and abstraction level, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
17. FR - Expliquez simulation, synthèse et implémentation. Quelle structure ou idée hardware se cache derrière ? Réponse : La simulation exécute le modèle VHDL dans un simulateur. La synthèse convertit le RTL synthétisable en réseau logique, puis l'implémentation place et route cette logique dans le composant cible.	EN - Explain simulation, synthesis, and implementation. What hardware structure or idea is behind it? Answer: Simulation executes the VHDL model in a simulator. Synthesis converts synthesizable RTL into a logic netlist, then implementation places and routes it in the target device.
18. FR - Comment utiliseriez-vous simulation, synthèse et implémentation dans un design VHDL réel ? Réponse : Je valide d'abord la fonction en simulation, puis les rapports de synthèse/timing, et enfin le hardware réel avec des tests reproductibles.	EN - How would you use simulation, synthesis, and implementation in a real VHDL design? Answer: I first validate function in simulation, then synthesis/timing reports, and finally real hardware with reproducible tests.
19. FR - Comment diagnostiqueriez-vous un problème lié à simulation, synthèse et implémentation ? Réponse : Je vérifie d'abord si le problème vient de la logique décrite, de l'horloge/reset, des contraintes ou d'une mauvaise compréhension du parallélisme. Je compare la simulation RTL au comportement attendu avant d'accuser l'outil ou le FPGA. Concernant simulation, synthèse et implémentation, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving simulation, synthesis, and implementation? Answer: I first check whether the problem comes from the described logic, clock/reset, constraints, or a misunderstanding of parallelism. I compare RTL simulation with expected behavior before blaming the tool or FPGA. For simulation, synthesis, and implementation, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
20. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant simulation, synthèse et implémentation ? Réponse : Je raisonne toujours en matériel: registres, portes, multiplexeurs, mémoire et chemins combinatoires. Le code VHDL décrit cette structure; ce n'est pas une suite d'instructions exécutées par un CPU. Pour simulation, synthèse et implémentation, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding simulation, synthesis, and implementation? Answer: I always reason in hardware: registers, gates, multiplexers, memory, and combinational paths. VHDL describes that structure; it is not a sequence of CPU instructions. For simulation, synthesis, and implementation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

2. Structure d'un fichier VHDL et unités de conception / VHDL File Structure and Design Units

21. FR - Expliquez clauses library et use. Quelle structure ou idée hardware se cache derrière ?

Réponse : `library` rend une bibliothèque logique connue du design, tandis que `use` rend visibles les déclarations d'un package. Pour du RTL courant, `ieee.std\_logic\_1164` et `ieee.numeric\_std` sont essentiels.

EN - Explain library and use clauses. What hardware structure or idea is behind it?

Answer: `library` makes a logical library known to the design, while `use` makes package declarations visible. For common RTL, `ieee.std\_logic\_1164` and `ieee.numeric\_std` are essential.

22. FR - Comment utiliseriez-vous clauses library et use dans un design VHDL réel ?

Réponse : Un en-tête typique importe `std\_logic\_1164` et `numeric\_std`; j'évite les anciens packages arithmétiques non standard.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

EN - How would you use library and use clauses in a real VHDL design?

Answer: A typical header imports `std\_logic\_1164` and `numeric\_std`; I avoid old non-standard arithmetic packages.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

23. FR - Comment diagnostiqueriez-vous un problème lié à clauses library et use ?

Réponse : Je contrôle l'ordre et la visibilité des unités de conception, les bibliothèques importées et les noms d'entity/architecture. Une erreur à ce niveau vient souvent d'un symbole non visible, d'un package absent ou d'une mauvaise unité analysée. Concernant clauses library et use, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.

EN - How would you troubleshoot an issue involving library and use clauses?

Answer: I check design-unit order and visibility, imported libraries, and entity/architecture names. Errors here often come from an invisible symbol, missing package, or wrong unit being analyzed. For library and use clauses, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

24. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clauses library et use ?

Réponse : Je garde les dépendances explicites, des noms cohérents et une unité de conception claire par fonction. Cela facilite compilation, réutilisation et revue de code. Pour clauses library et use, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.

EN - What best practice or interview trap should you know regarding library and use clauses?

Answer: I keep dependencies explicit, names consistent, and design units clearly separated by function. This improves compilation, reuse, and review. For library and use clauses, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

25. FR - Expliquez entity comme interface du bloc. Quelle structure ou idée hardware se cache derrière ?

Réponse : L'entity définit l'interface externe d'un bloc: son nom, ses generics et ses ports. Elle ne décrit pas, à elle seule, l'implémentation interne.

EN - Explain entity as the block interface. What hardware structure or idea is behind it?

Answer: The entity defines a block's external interface: its name, generics, and ports. By itself, it does not describe the internal implementation.

26. FR - Comment utiliseriez-vous entity comme interface du bloc dans un design VHDL réel ?

Réponse : Je place dans l'entity uniquement ce qui fait partie du contrat du bloc: clock, reset, données, handshakes et paramètres configurables.

```
entity pulse_counter is
port (
    clk : in std_logic;
    reset : in std_logic;
    pulse : in std_logic;
    count : out unsigned(15 downto 0)
);
end entity;
```

EN - How would you use entity as the block interface in a real VHDL design?

Answer: I place in the entity only what belongs to the block contract: clock, reset, data, handshakes, and configurable parameters.

```
entity pulse_counter is
port (
    clk : in std_logic;
    reset : in std_logic;
    pulse : in std_logic;
    count : out unsigned(15 downto 0)
);
end entity;
```

27. FR - Comment diagnostiqueriez-vous un problème lié à entity comme interface du bloc ?

Réponse : Je contrôle l'ordre et la visibilité des unités de conception, les bibliothèques importées et les noms d'entity/architecture. Une erreur à ce niveau vient souvent d'un symbole non visible, d'un package absent ou d'une mauvaise unité analysée. Concernant entity comme interface du bloc, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.

EN - How would you troubleshoot an issue involving entity as the block interface?

Answer: I check design-unit order and visibility, imported libraries, and entity/architecture names. Errors here often come from an invisible symbol, missing package, or wrong unit being analyzed. For entity as the block interface, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

28. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant entity comme interface du bloc ? Réponse : Je garde les dépendances explicites, des noms cohérents et une unité de conception claire par fonction. Cela facilite compilation, réutilisation et revue de code. Pour entity comme interface du bloc, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding entity as the block interface? Answer: I keep dependencies explicit, names consistent, and design units clearly separated by function. This improves compilation, reuse, and review. For entity as the block interface, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
29. FR - Expliquez architecture comme implémentation. Quelle structure ou idée hardware se cache derrière ? Réponse : L'architecture contient les déclarations internes puis les statements concurrents qui implémentent l'entity. Une même entity peut avoir plusieurs architectures pour différents modèles ou niveaux d'abstraction.	EN - Explain architecture as the implementation. What hardware structure or idea is behind it? Answer: The architecture contains internal declarations followed by concurrent statements implementing the entity. One entity can have multiple architectures for different models or abstraction levels.
30. FR - Comment utiliseriez-vous architecture comme implémentation dans un design VHDL réel ? Réponse : Je nomme souvent l'architecture `rtl` pour une implémentation synthétisable et je déclare les signaux internes avant `begin`. <pre>architecture rtl of pulse_counter is signal count_i : unsigned(15 downto 0); begin count <= count_i; end architecture;</pre>	EN - How would you use architecture as the implementation in a real VHDL design? Answer: I often name the architecture `rtl` for a synthesizable implementation and declare internal signals before `begin`. <pre>architecture rtl of pulse_counter is signal count_i : unsigned(15 downto 0); begin count <= count_i; end architecture;</pre>
31. FR - Comment diagnostiqueriez-vous un problème lié à architecture comme implémentation ? Réponse : Je contrôle l'ordre et la visibilité des unités de conception, les bibliothèques importées et les noms d'entity/architecture. Une erreur à ce niveau vient souvent d'un symbole non visible, d'un package absent ou d'une mauvaise unité analysée. Concernant architecture comme implémentation, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving architecture as the implementation? Answer: I check design-unit order and visibility, imported libraries, and entity/architecture names. Errors here often come from an invisible symbol, missing package, or wrong unit being analyzed. For architecture as the implementation, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
32. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant architecture comme implémentation ? Réponse : Je garde les dépendances explicites, des noms cohérents et une unité de conception claire par fonction. Cela facilite compilation, réutilisation et revue de code. Pour architecture comme implémentation, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding architecture as the implementation? Answer: I keep dependencies explicit, names consistent, and design units clearly separated by function. This improves compilation, reuse, and review. For architecture as the implementation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
33. FR - Expliquez package, package body et configuration. Quelle structure ou idée hardware se cache derrière ? Réponse : Les packages partagent types, constantes, composants et sous-programmes; un package body peut fournir l'implémentation des fonctions/procédures. Les configurations existent aussi, bien qu'elles soient moins fréquentes dans beaucoup de flows RTL modernes.	EN - Explain package, package body, and configuration. What hardware structure or idea is behind it? Answer: Packages share types, constants, components, and subprograms; a package body can provide function/procedure implementations. Configurations also exist, though they are less common in many modern RTL flows.
34. FR - Comment utiliseriez-vous package, package body et configuration dans un design VHDL réel ? Réponse : Je mets les types d'interface communs, constantes et fonctions pures dans un package afin que plusieurs blocs utilisent exactement les mêmes définitions.	EN - How would you use package, package body, and configuration in a real VHDL design? Answer: I place shared interface types, constants, and pure functions in a package so multiple blocks use exactly the same definitions.
35. FR - Comment diagnostiqueriez-vous un problème lié à package, package body et configuration ? Réponse : Je contrôle l'ordre et la visibilité des unités de conception, les bibliothèques importées et les noms d'entity/architecture. Une erreur à ce niveau vient souvent d'un symbole non visible, d'un package absent ou d'une mauvaise unité analysée. Concernant package, package body et configuration, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving package, package body, and configuration? Answer: I check design-unit order and visibility, imported libraries, and entity/architecture names. Errors here often come from an invisible symbol, missing package, or wrong unit being analyzed. For package, package body, and configuration, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

36. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant package, package body et configuration ? Réponse : Je garde les dépendances explicites, des noms cohérents et une unité de conception claire par fonction. Cela facilite compilation, réutilisation et revue de code. Pour package, package body et configuration, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding package, package body, and configuration? Answer: I keep dependencies explicit, names consistent, and design units clearly separated by function. This improves compilation, reuse, and review. For package, package body, and configuration, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
37. FR - Expliquez ordre de compilation et dépendances. Quelle structure ou idée hardware se cache derrière ? Réponse : Une unité VHDL doit être analysée après les packages et unités dont elle dépend. Les outils modernes calculent souvent cet ordre, mais une dépendance circulaire ou un package manquant peut encore casser le build.	EN - Explain compile order and dependencies. What hardware structure or idea is behind it? Answer: A VHDL unit must be analyzed after packages and units it depends on. Modern tools often determine this order, but circular dependencies or missing packages can still break the build.
38. FR - Comment utiliseriez-vous ordre de compilation et dépendances dans un design VHDL réel ? Réponse : Je structure le projet pour que les packages de base soient indépendants des blocs qui les utilisent et j'inspecte les messages d'analyse/élaboration en cas de symbole introuvable.	EN - How would you use compile order and dependencies in a real VHDL design? Answer: I structure the project so base packages are independent of blocks that consume them, and I inspect analysis/elaboration messages when a symbol is missing.
39. FR - Comment diagnostiqueriez-vous un problème lié à ordre de compilation et dépendances ? Réponse : Je contrôle l'ordre et la visibilité des unités de conception, les bibliothèques importées et les noms d'entity/architecture. Une erreur à ce niveau vient souvent d'un symbole non visible, d'un package absent ou d'une mauvaise unité analysée. Concernant ordre de compilation et dépendances, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving compile order and dependencies? Answer: I check design-unit order and visibility, imported libraries, and entity/architecture names. Errors here often come from an invisible symbol, missing package, or wrong unit being analyzed. For compile order and dependencies, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
40. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant ordre de compilation et dépendances ? Réponse : Je garde les dépendances explicites, des noms cohérents et une unité de conception claire par fonction. Cela facilite compilation, réutilisation et revue de code. Pour ordre de compilation et dépendances, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding compile order and dependencies? Answer: I keep dependencies explicit, names consistent, and design units clearly separated by function. This improves compilation, reuse, and review. For compile order and dependencies, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

3. Ports, directions, pins et interfaces externes / Ports, Directions, Pins, and External Interfaces

41. FR - Expliquez modes in, out, inout et buffer. Quelle structure ou idée hardware se cache derrière ? Réponse : `in` est lu par le bloc, `out` est piloté par le bloc et `inout` représente une interface bidirectionnelle. `buffer` existe mais est généralement évité dans du nouveau RTL au profit d'un signal interne puis d'un port `out`.	EN - Explain in, out, inout, and buffer modes. What hardware structure or idea is behind it? Answer: `in` is read by the block, `out` is driven by the block, and `inout` represents a bidirectional interface. `buffer` exists but is generally avoided in new RTL in favor of an internal signal feeding an `out` port.
42. FR - Comment utiliseriez-vous modes in, out, inout et buffer dans un design VHDL réel ? Réponse : Pour une broche bidirectionnelle, je sépare souvent `data_out`, `data_in` et `output_enable` en interne, puis je pilote `inout` seulement au top-level.	EN - How would you use in, out, inout, and buffer modes in a real VHDL design? Answer: For a bidirectional pin, I often separate `data_out`, `data_in`, and `output_enable` internally, then drive `inout` only at the top level.
43. FR - Comment diagnostiqueriez-vous un problème lié à modes in, out, inout et buffer ? Réponse : Je distingue le port logique VHDL du pin physique et je vérifie largeur, direction, niveau actif et domaine d'horloge. Pour un signal externe asynchrone, je cherche aussi la synchronisation avant toute logique de contrôle. Concernant modes in, out, inout et buffer, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving in, out, inout, and buffer modes? Answer: I distinguish the logical VHDL port from the physical pin and check width, direction, active level, and clock domain. For an asynchronous external signal, I also look for synchronization before control logic. For in, out, inout, and buffer modes, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

44. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant modes in, out, inout et buffer ? Réponse : Je documente clairement sens, polarité et protocole des ports. Les affectations de pins et standards électriques restent dans les contraintes du projet, pas dispersés dans le RTL. Pour modes in, out, inout et buffer, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding in, out, inout, and buffer modes? Answer: I clearly document port direction, polarity, and protocol. Pin assignments and electrical standards stay in project constraints rather than being scattered through RTL. For in, out, inout, and buffer modes, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
45. FR - Expliquez port VHDL versus pin physique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un nom comme `uart_rx` dans une entity est un signal logique. Le choix du pin physique, de la banque I/O, du standard électrique et du pull-up se fait dans le mécanisme de contraintes du flow FPGA.	EN - Explain VHDL port versus physical pin. What hardware structure or idea is behind it? Answer: A name such as `uart_rx` in an entity is a logical signal. Selection of the physical pin, I/O bank, electrical standard, and pull-up is done in the FPGA flow's constraint mechanism.
46. FR - Comment utiliseriez-vous port VHDL versus pin physique dans un design VHDL réel ? Réponse : Je peux garder le même RTL et changer de carte en modifiant surtout le top-level et les contraintes de pins, sans réécrire la logique UART.	EN - How would you use VHDL port versus physical pin in a real VHDL design? Answer: I can keep the same RTL and move to another board mainly by changing the top level and pin constraints, without rewriting the UART logic.
47. FR - Comment diagnostiqueriez-vous un problème lié à port VHDL versus pin physique ? Réponse : Je distingue le port logique VHDL du pin physique et je vérifie largeur, direction, niveau actif et domaine d'horloge. Pour un signal externe asynchrone, je cherche aussi la synchronisation avant toute logique de contrôle. Concernant port VHDL versus pin physique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving VHDL port versus physical pin? Answer: I distinguish the logical VHDL port from the physical pin and check width, direction, active level, and clock domain. For an asynchronous external signal, I also look for synchronization before control logic. For VHDL port versus physical pin, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
48. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant port VHDL versus pin physique ? Réponse : Je documente clairement sens, polarité et protocole des ports. Les affectations de pins et standards électriques restent dans les contraintes du projet, pas dispersés dans le RTL. Pour port VHDL versus pin physique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding VHDL port versus physical pin? Answer: I clearly document port direction, polarity, and protocol. Pin assignments and electrical standards stay in project constraints rather than being scattered through RTL. For VHDL port versus physical pin, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
49. FR - Expliquez largeurs de bus et ranges downto/to. Quelle structure ou idée hardware se cache derrière ? Réponse : Un vecteur peut être déclaré `7 downto 0` ou `0 to 7`. Les deux ont huit éléments, mais orientation et indices doivent rester cohérents lors des slices, concaténations et mappings.	EN - Explain bus widths and downto/to ranges. What hardware structure or idea is behind it? Answer: A vector can be declared `7 downto 0` or `0 to 7`. Both have eight elements, but orientation and indices must remain consistent during slicing, concatenation, and mapping.
50. FR - Comment utiliseriez-vous largeurs de bus et ranges downto/to dans un design VHDL réel ? Réponse : Pour les bus numériques, j'utilise généralement `N-1 downto 0` et des attributs comme `range` ou `length` pour éviter les constantes magiques. <pre> signal data : std_logic_vector(7 downto 0); signal hi : std_logic_vector(3 downto 0); hi <= data(7 downto 4); </pre>	EN - How would you use bus widths and downto/to ranges in a real VHDL design? Answer: For numeric buses, I generally use `N-1 downto 0` and attributes such as `range` or `length` to avoid magic constants. <pre> signal data : std_logic_vector(7 downto 0); signal hi : std_logic_vector(3 downto 0); hi <= data(7 downto 4); </pre>
51. FR - Comment diagnostiqueriez-vous un problème lié à largeurs de bus et ranges downto/to ? Réponse : Je distingue le port logique VHDL du pin physique et je vérifie largeur, direction, niveau actif et domaine d'horloge. Pour un signal externe asynchrone, je cherche aussi la synchronisation avant toute logique de contrôle. Concernant largeurs de bus et ranges downto/to, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving bus widths and downto/to ranges? Answer: I distinguish the logical VHDL port from the physical pin and check width, direction, active level, and clock domain. For an asynchronous external signal, I also look for synchronization before control logic. For bus widths and downto/to ranges, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

52. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant largeurs de bus et ranges downto/to ? Réponse : Je documente clairement sens, polarité et protocole des ports. Les affectations de pins et standards électriques restent dans les contraintes du projet, pas dispersés dans le RTL. Pour largeurs de bus et ranges downto/to, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding bus widths and downto/to ranges? Answer: I clearly document port direction, polarity, and protocol. Pin assignments and electrical standards stay in project constraints rather than being scattered through RTL. For bus widths and downto/to ranges, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
53. FR - Expliquez polarité active-high et active-low. Quelle structure ou idée hardware se cache derrière ? Réponse : La polarité fait partie du contrat de l'interface. Des noms comme `reset_n` ou `cs_n` indiquent généralement un signal actif à zéro; la logique doit rester cohérente avec cette convention.	EN - Explain active-high and active-low polarity. What hardware structure or idea is behind it? Answer: Polarity is part of the interface contract. Names such as `reset_n` or `cs_n` generally indicate an active-low signal; logic must remain consistent with that convention.
54. FR - Comment utiliseriez-vous polarité active-high et active-low dans un design VHDL réel ? Réponse : Je préfère convertir tôt un signal externe actif-bas en convention interne claire, ou au minimum garder le suffixe `_n` partout où la polarité compte.	EN - How would you use active-high and active-low polarity in a real VHDL design? Answer: I prefer to convert an external active-low signal early into a clear internal convention, or at least keep the `_n` suffix everywhere polarity matters.
55. FR - Comment diagnostiqueriez-vous un problème lié à polarité active-high et active-low ? Réponse : Je distingue le port logique VHDL du pin physique et je vérifie largeur, direction, niveau actif et domaine d'horloge. Pour un signal externe asynchrone, je cherche aussi la synchronisation avant toute logique de contrôle. Concernant polarité active-high et active-low, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving active-high and active-low polarity? Answer: I distinguish the logical VHDL port from the physical pin and check width, direction, active level, and clock domain. For an asynchronous external signal, I also look for synchronization before control logic. For active-high and active-low polarity, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
56. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant polarité active-high et active-low ? Réponse : Je documente clairement sens, polarité et protocole des ports. Les affectations de pins et standards électriques restent dans les contraintes du projet, pas dispersés dans le RTL. Pour polarité active-high et active-low, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding active-high and active-low polarity? Answer: I clearly document port direction, polarity, and protocol. Pin assignments and electrical standards stay in project constraints rather than being scattered through RTL. For active-high and active-low polarity, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
57. FR - Expliquez interrupt dans une FPGA versus signal d'entrée. Quelle structure ou idée hardware se cache derrière ? Réponse : VHDL n'a pas un mode de port `interrupt`. Une broche d'interruption externe est simplement un signal d'entrée; la logique FPGA peut le synchroniser, détecter son front, déclencher une FSM ou générer un IRQ vers un processeur.	EN - Explain interrupt in an FPGA versus an input signal. What hardware structure or idea is behind it? Answer: VHDL has no `interrupt` port mode. An external interrupt pin is simply an input signal; FPGA logic can synchronize it, detect an edge, trigger an FSM, or generate an IRQ toward a processor.
58. FR - Comment utiliseriez-vous interrupt dans une FPGA versus signal d'entrée dans un design VHDL réel ? Réponse : Pour un IRQ asynchrone, je fais typiquement entrée -> synchroniseur -> edge detector -> logique de contrôle; si un CPU est présent, la dernière étape peut produire une ligne IRQ propre.	EN - How would you use interrupt in an FPGA versus an input signal in a real VHDL design? Answer: For an asynchronous IRQ, I typically use input -> synchronizer -> edge detector -> control logic; if a CPU is present, the last stage can produce a clean IRQ line.
59. FR - Comment diagnostiqueriez-vous un problème lié à interrupt dans une FPGA versus signal d'entrée ? Réponse : Je distingue le port logique VHDL du pin physique et je vérifie largeur, direction, niveau actif et domaine d'horloge. Pour un signal externe asynchrone, je cherche aussi la synchronisation avant toute logique de contrôle. Concernant interrupt dans une FPGA versus signal d'entrée, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving interrupt in an FPGA versus an input signal? Answer: I distinguish the logical VHDL port from the physical pin and check width, direction, active level, and clock domain. For an asynchronous external signal, I also look for synchronization before control logic. For interrupt in an FPGA versus an input signal, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

60. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant interrupt dans une FPGA versus signal d'entrée ? Réponse : Je documente clairement sens, polarité et protocole des ports. Les affectations de pins et standards électriques restent dans les contraintes du projet, pas dispersés dans le RTL. Pour interrupt dans une FPGA versus signal d'entrée, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding interrupt in an FPGA versus an input signal? Answer: I clearly document port direction, polarity, and protocol. Pin assignments and electrical standards stay in project constraints rather than being scattered through RTL. For interrupt in an FPGA versus an input signal, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

4. Architecture, concurrence et hiérarchie de base / Architecture, Concurrency, and Basic Hierarchy

61. FR - Expliquez statements concurrents. Quelle structure ou idée hardware se cache derrière ? Réponse : Les statements placés directement dans une architecture après `begin` sont concurrents: leur ordre textuel ne fixe pas un ordre d'exécution logiciel. Ils décrivent des blocs matériels actifs en parallèle.	EN - Explain concurrent statements. What hardware structure or idea is behind it? Answer: Statements placed directly in an architecture after `begin` are concurrent: textual order does not define software execution order. They describe hardware blocks active in parallel.
62. FR - Comment utiliseriez-vous statements concurrents dans un design VHDL réel ? Réponse : Deux affectations comme `x <= a and b;` et `y <= c xor d;` représentent deux fonctions logiques parallèles. <pre>x <= a and b; y <= c xor d;</pre>	EN - How would you use concurrent statements in a real VHDL design? Answer: Two assignments such as `x <= a and b;` and `y <= c xor d;` represent two parallel logic functions. <pre>x <= a and b; y <= c xor d;</pre>
63. FR - Comment diagnostiqueriez-vous un problème lié à statements concurrents ? Réponse : Je vérifie si le designer pense séquentiellement alors que les statements sont concurrents. Je regarde les drivers de chaque signal et confirme que la hiérarchie mappe les bons ports et largeurs. Concernant statements concurrents, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving concurrent statements? Answer: I check whether the designer is thinking sequentially while statements are concurrent. I inspect every signal driver and confirm hierarchy maps the correct ports and widths. For concurrent statements, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
64. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant statements concurrents ? Réponse : Je garde la hiérarchie lisible, un propriétaire clair par signal et des interfaces simples entre blocs. Cela réduit les erreurs de mapping et facilite la simulation par sous-bloc. Pour statements concurrents, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding concurrent statements? Answer: I keep hierarchy readable, establish clear ownership per signal, and use simple block interfaces. This reduces mapping errors and simplifies sub-block simulation. For concurrent statements, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
65. FR - Expliquez signaux internes d'architecture. Quelle structure ou idée hardware se cache derrière ? Réponse : Les signaux internes relient des blocs et transportent des valeurs entre statements concurrents/process. Ils sont déclarés dans la partie déclarative de l'architecture avant `begin`.	EN - Explain internal architecture signals. What hardware structure or idea is behind it? Answer: Internal signals connect blocks and carry values between concurrent statements/processes. They are declared in the architecture declarative region before `begin`.
66. FR - Comment utiliseriez-vous signaux internes d'architecture dans un design VHDL réel ? Réponse : J'utilise un signal interne lorsque plusieurs parties du design doivent voir une valeur ou lorsqu'une sortie doit venir d'un registre interne bien défini.	EN - How would you use internal architecture signals in a real VHDL design? Answer: I use an internal signal when several parts of the design need a value or when an output should come from a clearly defined internal register.
67. FR - Comment diagnostiqueriez-vous un problème lié à signaux internes d'architecture ? Réponse : Je vérifie si le designer pense séquentiellement alors que les statements sont concurrents. Je regarde les drivers de chaque signal et confirme que la hiérarchie mappe les bons ports et largeurs. Concernant signaux internes d'architecture, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving internal architecture signals? Answer: I check whether the designer is thinking sequentially while statements are concurrent. I inspect every signal driver and confirm hierarchy maps the correct ports and widths. For internal architecture signals, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

68. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signaux internes d'architecture ? Réponse : Je garde la hiérarchie lisible, un propriétaire clair par signal et des interfaces simples entre blocs. Cela réduit les erreurs de mapping et facilite la simulation par sous-bloc. Pour signaux internes d'architecture, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding internal architecture signals? Answer: I keep hierarchy readable, establish clear ownership per signal, and use simple block interfaces. This reduces mapping errors and simplifies sub-block simulation. For internal architecture signals, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
69. FR - Expliquez hiérarchie top-level et sous-blocs. Quelle structure ou idée hardware se cache derrière ? Réponse : Un design non trivial est généralement décomposé en entités réutilisables instanciées dans un top-level. Le top-level relie horloges, resets, interfaces et pins logiques aux sous-blocs.	EN - Explain top-level hierarchy and sub-blocks. What hardware structure or idea is behind it? Answer: A non-trivial design is generally decomposed into reusable entities instantiated in a top level. The top level connects clocks, resets, interfaces, and logical pins to sub-blocks.
70. FR - Comment utiliseriez-vous hiérarchie top-level et sous-blocs dans un design VHDL réel ? Réponse : Je peux séparer `uart_rx`, `packet_parser` et `control_fsm`, puis les connecter au top par des signaux simples et testables indépendamment.	EN - How would you use top-level hierarchy and sub-blocks in a real VHDL design? Answer: I can separate `uart_rx`, `packet_parser`, and `control_fsm`, then connect them at the top with simple signals that can be tested independently.
71. FR - Comment diagnostiqueriez-vous un problème lié à hiérarchie top-level et sous-blocs ? Réponse : Je vérifie si le designer pense séquentiellement alors que les statements sont concurrents. Je regarde les drivers de chaque signal et confirme que la hiérarchie mappe les bons ports et largeurs. Concernant hiérarchie top-level et sous-blocs, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving top-level hierarchy and sub-blocks? Answer: I check whether the designer is thinking sequentially while statements are concurrent. I inspect every signal driver and confirm hierarchy maps the correct ports and widths. For top-level hierarchy and sub-blocks, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
72. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant hiérarchie top-level et sous-blocs ? Réponse : Je garde la hiérarchie lisible, un propriétaire clair par signal et des interfaces simples entre blocs. Cela réduit les erreurs de mapping et facilite la simulation par sous-bloc. Pour hiérarchie top-level et sous-blocs, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding top-level hierarchy and sub-blocks? Answer: I keep hierarchy readable, establish clear ownership per signal, and use simple block interfaces. This reduces mapping errors and simplifies sub-block simulation. For top-level hierarchy and sub-blocks, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
73. FR - Expliquez instanciation directe d'entity. Quelle structure ou idée hardware se cache derrière ? Réponse : L'instanciation directe lie explicitement une instance à une entity/architecture sans déclaration préalable de component. Elle réduit souvent le risque de divergence entre component declaration et entity réelle.	EN - Explain direct entity instantiation. What hardware structure or idea is behind it? Answer: Direct entity instantiation explicitly binds an instance to an entity/architecture without a prior component declaration. It often reduces mismatch risk between component declaration and real entity.
74. FR - Comment utiliseriez-vous instanciation directe d'entity dans un design VHDL réel ? Réponse : J'écris par exemple `u_uart : entity work. uart_rx(rtl) port map (...)` lorsqu'un style de projet autorise l'instanciation directe. <pre> u_uart : entity work. uart_rx(rtl) port map (clk => clk, rx => uart_rx, data_valid => rx_valid); </pre>	EN - How would you use direct entity instantiation in a real VHDL design? Answer: For example, I write `u_uart : entity work. uart_rx(rtl) port map (...)` when the project style allows direct instantiation. <pre> u_uart : entity work. uart_rx(rtl) port map (clk => clk, rx => uart_rx, data_valid => rx_valid); </pre>
75. FR - Comment diagnostiqueriez-vous un problème lié à instanciation directe d'entity ? Réponse : Je vérifie si le designer pense séquentiellement alors que les statements sont concurrents. Je regarde les drivers de chaque signal et confirme que la hiérarchie mappe les bons ports et largeurs. Concernant instanciation directe d'entity, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving direct entity instantiation? Answer: I check whether the designer is thinking sequentially while statements are concurrent. I inspect every signal driver and confirm hierarchy maps the correct ports and widths. For direct entity instantiation, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

76. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant instanciation directe d'entity ? Réponse : Je garde la hiérarchie lisible, un propriétaire clair par signal et des interfaces simples entre blocs. Cela réduit les erreurs de mapping et facilite la simulation par sous-bloc. Pour instanciation directe d'entity, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding direct entity instantiation? Answer: I keep hierarchy readable, establish clear ownership per signal, and use simple block interfaces. This reduces mapping errors and simplifies sub-block simulation. For direct entity instantiation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
77. FR - Expliquez nommage d'instances et traçabilité. Quelle structure ou idée hardware se cache derrière ? Réponse : Le label d'instance est important dans les messages de simulation, la hiérarchie de debug et les rapports d'implémentation. Des noms comme `u_spi`, `u_fifo_rx` ou `u_ctrl` facilitent fortement le diagnostic.	EN - Explain instance naming and traceability. What hardware structure or idea is behind it? Answer: Instance labels matter in simulation messages, debug hierarchy, and implementation reports. Names such as `u_spi`, `u_fifo_rx`, or `u_ctrl` greatly simplify diagnosis.
78. FR - Comment utiliseriez-vous nommage d'instances et traçabilité dans un design VHDL réel ? Réponse : Je nomme les instances selon leur rôle fonctionnel plutôt que `inst1/inst2` afin de retrouver immédiatement le bloc dans une waveform ou un rapport de timing.	EN - How would you use instance naming and traceability in a real VHDL design? Answer: I name instances by functional role rather than `inst1/inst2` so the block can be identified immediately in a waveform or timing report.
79. FR - Comment diagnostiqueriez-vous un problème lié à nommage d'instances et traçabilité ? Réponse : Je vérifie si le designer pense séquentiellement alors que les statements sont concurrents. Je regarde les drivers de chaque signal et confirme que la hiérarchie mappe les bons ports et largeurs. Concernant nommage d'instances et traçabilité, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving instance naming and traceability? Answer: I check whether the designer is thinking sequentially while statements are concurrent. I inspect every signal driver and confirm hierarchy maps the correct ports and widths. For instance naming and traceability, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
80. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant nommage d'instances et traçabilité ? Réponse : Je garde la hiérarchie lisible, un propriétaire clair par signal et des interfaces simples entre blocs. Cela réduit les erreurs de mapping et facilite la simulation par sous-bloc. Pour nommage d'instances et traçabilité, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding instance naming and traceability? Answer: I keep hierarchy readable, establish clear ownership per signal, and use simple block interfaces. This reduces mapping errors and simplifies sub-block simulation. For instance naming and traceability, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

5. Types, sous-types et littéraux / Types, Subtypes, and Literals

81. FR - Expliquez bit versus std_logic. Quelle structure ou idée hardware se cache derrière ? Réponse : `bit` ne possède que `0` et `1`. `std_logic` est un type résolu multi-valué qui représente aussi `U`, `X`, `Z`, etc., utile pour détecter états inconnus, haute impédance et conflits en simulation.	EN - Explain bit versus std_logic. What hardware structure or idea is behind it? Answer: `bit` has only `0` and `1`. `std_logic` is a resolved multi-valued type that also represents `U`, `X`, `Z`, etc., useful for detecting unknowns, high impedance, and contention in simulation.
82. FR - Comment utiliseriez-vous bit versus std_logic dans un design VHDL réel ? Réponse : Pour des ports et nets RTL classiques, j'utilise généralement `std_logic`; un `X` en simulation devient alors un signal de diagnostic.	EN - How would you use bit versus std_logic in a real VHDL design? Answer: For typical RTL ports and nets, I generally use `std_logic`; an `X` in simulation then becomes a diagnostic signal.
83. FR - Comment diagnostiqueriez-vous un problème lié à bit versus std_logic ? Réponse : Je recherche d'abord une incompatibilité de type, de taille ou de sens signed/unsigned. Les conversions explicites sont préférables à des casts implicites ou à des packages non standard. Concernant bit versus std_logic, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving bit versus std_logic? Answer: I first look for a type, width, or signed/unsigned mismatch. Explicit conversions are preferable to implicit casts or non-standard packages. For bit versus std_logic, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

84. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant bit versus std_logic ? Réponse : J'utilise des types qui expriment l'intention: <code>`std_logic`</code> pour un signal logique, <code>`unsigned/signed`</code> pour l'arithmétique et des types énumérés pour les états. Pour bit versus <code>std_logic</code>, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding bit versus std_logic? Answer: I use types that express intent: <code>`std_logic`</code> for logic signals, <code>`unsigned/signed`</code> for arithmetic, and enumerated types for states. For bit versus <code>std_logic</code>, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
85. FR - Expliquez std_logic_vector. Quelle structure ou idée hardware se cache derrière ? Réponse : <code>`std_logic_vector`</code> est un tableau de <code>`std_logic`</code>; il représente bien un bus mais n'exprime pas automatiquement une intention arithmétique. Pour additionner proprement, je convertis ou déclare directement <code>`unsigned`/`signed`</code>.	EN - Explain std_logic_vector. What hardware structure or idea is behind it? Answer: <code>`std_logic_vector`</code> is an array of <code>`std_logic`</code>; it represents a bus but does not automatically express arithmetic intent. For clean arithmetic, I convert or directly declare <code>`unsigned`/`signed`</code>.
86. FR - Comment utiliseriez-vous std_logic_vector dans un design VHDL réel ? Réponse : Je garde par exemple une interface externe en <code>`std_logic_vector`</code>, puis je convertis vers <code>`unsigned`</code> dans le datapath si la valeur représente un nombre non signé.	EN - How would you use std_logic_vector in a real VHDL design? Answer: I may keep an external interface as <code>`std_logic_vector`</code>, then convert to <code>`unsigned`</code> in the datapath if the value represents an unsigned number.
87. FR - Comment diagnostiqueriez-vous un problème lié à std_logic_vector ? Réponse : Je recherche d'abord une incompatibilité de type, de taille ou de sens <code>signed/unsigned</code>. Les conversions explicites sont préférables à des casts implicites ou à des packages non standard. Concernant <code>std_logic_vector</code>, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving std_logic_vector? Answer: I first look for a type, width, or signed/unsigned mismatch. Explicit conversions are preferable to implicit casts or non-standard packages. For <code>std_logic_vector</code>, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
88. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant std_logic_vector ? Réponse : J'utilise des types qui expriment l'intention: <code>`std_logic`</code> pour un signal logique, <code>`unsigned/signed`</code> pour l'arithmétique et des types énumérés pour les états. Pour <code>std_logic_vector</code>, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding std_logic_vector? Answer: I use types that express intent: <code>`std_logic`</code> for logic signals, <code>`unsigned/signed`</code> for arithmetic, and enumerated types for states. For <code>std_logic_vector</code>, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
89. FR - Expliquez unsigned et signed de numeric_std. Quelle structure ou idée hardware se cache derrière ? Réponse : <code>`unsigned`</code> et <code>`signed`</code> sont des tableaux de <code>`std_logic`</code> avec sémantique arithmétique définie par <code>`numeric_std`</code>. <code>`signed`</code> suit le complément à deux pour les valeurs négatives.	EN - Explain unsigned and signed from numeric_std. What hardware structure or idea is behind it? Answer: <code>`unsigned`</code> and <code>`signed`</code> are arrays of <code>`std_logic`</code> with arithmetic semantics defined by <code>`numeric_std`</code>. <code>`signed`</code> uses two's-complement representation for negative values.
90. FR - Comment utiliseriez-vous unsigned et signed de numeric_std dans un design VHDL réel ? Réponse : Un compteur est naturellement <code>`unsigned`</code>; une mesure centrée autour de zéro peut être <code>`signed`</code>. Je dimensionne explicitement le résultat quand une opération peut nécessiter plus de bits.	EN - How would you use unsigned and signed from numeric_std in a real VHDL design? Answer: A counter is naturally <code>`unsigned`</code>; a quantity centered around zero may be <code>`signed`</code>. I explicitly size results when an operation may require more bits.
91. FR - Comment diagnostiqueriez-vous un problème lié à unsigned et signed de numeric_std ? Réponse : Je recherche d'abord une incompatibilité de type, de taille ou de sens <code>signed/unsigned</code>. Les conversions explicites sont préférables à des casts implicites ou à des packages non standard. Concernant <code>unsigned</code> et <code>signed</code> de <code>numeric_std</code>, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving unsigned and signed from numeric_std? Answer: I first look for a type, width, or signed/unsigned mismatch. Explicit conversions are preferable to implicit casts or non-standard packages. For <code>unsigned</code> and <code>signed</code> from <code>numeric_std</code>, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
92. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant unsigned et signed de numeric_std ? Réponse : J'utilise des types qui expriment l'intention: <code>`std_logic`</code> pour un signal logique, <code>`unsigned/signed`</code> pour l'arithmétique et des types énumérés pour les états. Pour <code>unsigned</code> et <code>signed</code> de <code>numeric_std</code>, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding unsigned and signed from numeric_std? Answer: I use types that express intent: <code>`std_logic`</code> for logic signals, <code>`unsigned/signed`</code> for arithmetic, and enumerated types for states. For <code>unsigned</code> and <code>signed</code> from <code>numeric_std</code>, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

93. FR - Expliquez integer, natural et positive. Quelle structure ou idée hardware se cache derrière ? Réponse : `integer` est un entier; `natural` limite la plage à 0 et plus, et `positive` à 1 et plus. Des ranges explicites documentent la plage fonctionnelle attendue.	EN - Explain integer, natural, and positive. What hardware structure or idea is behind it? Answer: `integer` is an integer type; `natural` restricts the range to zero and above, and `positive` to one and above. Explicit ranges document expected functional limits.
94. FR - Comment utiliseriez-vous integer, natural et positive dans un design VHDL réel ? Réponse : Pour un index de tableau ou un generic de fréquence, `natural`/`positive` peut être très lisible; pour un registre bit-accurate, j'utilise souvent `unsigned`.	EN - How would you use integer, natural, and positive in a real VHDL design? Answer: For an array index or frequency generic, `natural`/`positive` can be very clear; for a bit-accurate register, I often use `unsigned`.
95. FR - Comment diagnostiqueriez-vous un problème lié à integer, natural et positive ? Réponse : Je recherche d'abord une incompatibilité de type, de taille ou de sens signed/unsigned. Les conversions explicites sont préférables à des casts implicites ou à des packages non standard. Concernant integer, natural et positive, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving integer, natural, and positive? Answer: I first look for a type, width, or signed/unsigned mismatch. Explicit conversions are preferable to implicit casts or non-standard packages. For integer, natural, and positive, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
96. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant integer, natural et positive ? Réponse : J'utilise des types qui expriment l'intention: `std_logic` pour un signal logique, `unsigned/signed` pour l'arithmétique et des types énumérés pour les états. Pour integer, natural et positive, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding integer, natural, and positive? Answer: I use types that express intent: `std_logic` for logic signals, `unsigned/signed` for arithmetic, and enumerated types for states. For integer, natural, and positive, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
97. FR - Expliquez types énumérés et subtypes. Quelle structure ou idée hardware se cache derrière ? Réponse : Un type énuméré donne des valeurs symboliques comme `IDLE`, `RUN`, `ERROR`, idéal pour les FSM. Un subtype restreint ou renomme un type existant sans créer un type complètement distinct.	EN - Explain enumerated types and subtypes. What hardware structure or idea is behind it? Answer: An enumerated type provides symbolic values such as `IDLE`, `RUN`, `ERROR`, ideal for FSMs. A subtype constrains or renames an existing type without creating a completely distinct type.
98. FR - Comment utiliseriez-vous types énumérés et subtypes dans un design VHDL réel ? Réponse : Je préfère `type state_t is (IDLE, RUN, ERROR);` à des codes numériques écrits à la main, car simulation et maintenance deviennent beaucoup plus claires. <pre>type state_t is (IDLE, RUN, ERROR); signal state : state_t := IDLE;</pre>	EN - How would you use enumerated types and subtypes in a real VHDL design? Answer: I prefer `type state_t is (IDLE, RUN, ERROR);` over hand-written numeric state codes because simulation and maintenance become much clearer. <pre>type state_t is (IDLE, RUN, ERROR); signal state : state_t := IDLE;</pre>
99. FR - Comment diagnostiqueriez-vous un problème lié à types énumérés et subtypes ? Réponse : Je recherche d'abord une incompatibilité de type, de taille ou de sens signed/unsigned. Les conversions explicites sont préférables à des casts implicites ou à des packages non standard. Concernant types énumérés et subtypes, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving enumerated types and subtypes? Answer: I first look for a type, width, or signed/unsigned mismatch. Explicit conversions are preferable to implicit casts or non-standard packages. For enumerated types and subtypes, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
100. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant types énumérés et subtypes ? Réponse : J'utilise des types qui expriment l'intention: `std_logic` pour un signal logique, `unsigned/signed` pour l'arithmétique et des types énumérés pour les états. Pour types énumérés et subtypes, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding enumerated types and subtypes? Answer: I use types that express intent: `std_logic` for logic signals, `unsigned/signed` for arithmetic, and enumerated types for states. For enumerated types and subtypes, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

6. Opérateurs, expressions et concaténation / Operators, Expressions, and Concatenation

101. FR - Expliquez opérateurs logiques. Quelle structure ou idée hardware se cache derrière ? Réponse : VHDL fournit <code>`and`, `or`, `nand`, `nor`, `xor`, `xnor` et <code>`not`</code> pour les types logiques compatibles. Sur des vecteurs, l'opération s'applique élément par élément lorsque les dimensions concordent.</code>	EN - Explain logical operators. What hardware structure or idea is behind it? Answer: VHDL provides <code>`and`, `or`, `nand`, `nor`, `xor`, `xnor`, and <code>`not`</code> for compatible logic types. On vectors, the operation applies element by element when dimensions match.</code>
102. FR - Comment utiliseriez-vous opérateurs logiques dans un design VHDL réel ? Réponse : Pour un masque, je peux écrire <code>`masked <= data and mask;`</code>; pour la logique de contrôle, je garde les expressions simples afin que la fonction booléenne soit évidente.	EN - How would you use logical operators in a real VHDL design? Answer: For masking, I can write <code>`masked <= data and mask;`</code>; for control logic, I keep expressions simple so the Boolean function is obvious.
103. FR - Comment diagnostiqueriez-vous un problème lié à opérateurs logiques ? Réponse : Je vérifie la priorité des opérateurs, les tailles de vecteurs et surtout le type des opérandes. Une expression correcte syntaxiquement peut encore tronquer ou étendre une valeur de façon inattendue. Concernant opérateurs logiques, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving logical operators? Answer: I check operator precedence, vector widths, and especially operand types. A syntactically correct expression can still truncate or extend a value unexpectedly. For logical operators, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
104. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateurs logiques ? Réponse : Je préfère des parenthèses explicites et des conversions visibles quand une expression mélange logique, comparaison et arithmétique. La lisibilité est plus importante qu'une ligne compacte. Pour opérateurs logiques, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding logical operators? Answer: I prefer explicit parentheses and visible conversions when an expression mixes logic, comparison, and arithmetic. Readability matters more than compactness. For logical operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
105. FR - Expliquez opérateurs relationnels. Quelle structure ou idée hardware se cache derrière ? Réponse : Les opérateurs <code>`=`, `/=`, `<`, `<=`, `>`, `>=`</code> produisent une valeur booléenne. Pour les comparaisons numériques, le type <code>`unsigned`</code> ou <code>`signed`</code> doit refléter l'interprétation réelle des bits.	EN - Explain relational operators. What hardware structure or idea is behind it? Answer: Operators <code>`=`, `/=`, `<`, `<=`, `>`, and <code>`>=`</code> produce a Boolean value. For numeric comparisons, <code>`unsigned`</code> or <code>`signed`</code> must reflect the actual interpretation of the bits.</code>
106. FR - Comment utiliseriez-vous opérateurs relationnels dans un design VHDL réel ? Réponse : Je compare un compteur <code>`unsigned`</code> à <code>`to_unsigned(LIMIT, counter'length)`</code> ou à une constante correctement typée plutôt que de compter sur une conversion ambiguë.	EN - How would you use relational operators in a real VHDL design? Answer: I compare an <code>`unsigned`</code> counter with <code>`to_unsigned(LIMIT, counter'length)`</code> or a correctly typed constant rather than relying on ambiguous conversion.
107. FR - Comment diagnostiqueriez-vous un problème lié à opérateurs relationnels ? Réponse : Je vérifie la priorité des opérateurs, les tailles de vecteurs et surtout le type des opérandes. Une expression correcte syntaxiquement peut encore tronquer ou étendre une valeur de façon inattendue. Concernant opérateurs relationnels, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving relational operators? Answer: I check operator precedence, vector widths, and especially operand types. A syntactically correct expression can still truncate or extend a value unexpectedly. For relational operators, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
108. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateurs relationnels ? Réponse : Je préfère des parenthèses explicites et des conversions visibles quand une expression mélange logique, comparaison et arithmétique. La lisibilité est plus importante qu'une ligne compacte. Pour opérateurs relationnels, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding relational operators? Answer: I prefer explicit parentheses and visible conversions when an expression mixes logic, comparison, and arithmetic. Readability matters more than compactness. For relational operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
109. FR - Expliquez opérateurs arithmétiques. Quelle structure ou idée hardware se cache derrière ? Réponse : Avec <code>`numeric_std`, <code>`+`, <code>`-`, <code>`*`</code> et d'autres opérateurs sont définis pour <code>`unsigned`</code> et <code>`signed`</code>. La largeur du résultat doit être comprise explicitement pour éviter overflow ou troncature.</code></code></code>	EN - Explain arithmetic operators. What hardware structure or idea is behind it? Answer: With <code>`numeric_std`, <code>`+`, <code>`-`, <code>`*`, and other operators are defined for <code>`unsigned`</code> and <code>`signed`</code>. Result width must be understood explicitly to avoid overflow or truncation.</code></code></code></code>

110. FR - Comment utiliseriez-vous opérateurs arithmétiques dans un design VHDL réel ? Réponse : Pour une accumulation, j'ajoute des bits de garde au registre accumulateur, puis je décide explicitement quand et comment réduire la largeur.	EN - How would you use arithmetic operators in a real VHDL design? Answer: For accumulation, I add guard bits to the accumulator register, then explicitly decide when and how to reduce width.
111. FR - Comment diagnostiqueriez-vous un problème lié à opérateurs arithmétiques ? Réponse : Je vérifie la priorité des opérateurs, les tailles de vecteurs et surtout le type des opérandes. Une expression correcte syntaxiquement peut encore tronquer ou étendre une valeur de façon inattendue. Concernant opérateurs arithmétiques, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving arithmetic operators? Answer: I check operator precedence, vector widths, and especially operand types. A syntactically correct expression can still truncate or extend a value unexpectedly. For arithmetic operators, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
112. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant opérateurs arithmétiques ? Réponse : Je préfère des parenthèses explicites et des conversions visibles quand une expression mélange logique, comparaison et arithmétique. La lisibilité est plus importante qu'une ligne compacte. Pour opérateurs arithmétiques, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding arithmetic operators? Answer: I prefer explicit parentheses and visible conversions when an expression mixes logic, comparison, and arithmetic. Readability matters more than compactness. For arithmetic operators, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
113. FR - Expliquez concaténation avec &. Quelle structure ou idée hardware se cache derrière ? Réponse : L'opérateur `&` concatène éléments ou vecteurs. Il est utile pour assembler un mot, ajouter un bit de signe, former un shift register ou construire un paquet.	EN - Explain concatenation with &. What hardware structure or idea is behind it? Answer: The `&` operator concatenates elements or vectors. It is useful for assembling a word, adding a sign bit, building a shift register, or constructing a packet.
114. FR - Comment utiliseriez-vous concaténation avec & dans un design VHDL réel ? Réponse : Un shift register peut être écrit <code>shift <= shift(6 downto 0) & serial_in;</code>, ce qui montre directement quels bits sont conservés et où entre le nouveau bit.	EN - How would you use concatenation with & in a real VHDL design? Answer: A shift register can be written <code>shift <= shift(6 downto 0) & serial_in;</code>, directly showing which bits are kept and where the new bit enters.
115. FR - Comment diagnostiqueriez-vous un problème lié à concaténation avec & ? Réponse : Je vérifie la priorité des opérateurs, les tailles de vecteurs et surtout le type des opérandes. Une expression correcte syntaxiquement peut encore tronquer ou étendre une valeur de façon inattendue. Concernant concaténation avec &, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving concatenation with &? Answer: I check operator precedence, vector widths, and especially operand types. A syntactically correct expression can still truncate or extend a value unexpectedly. For concatenation with &, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
116. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant concaténation avec & ? Réponse : Je préfère des parenthèses explicites et des conversions visibles quand une expression mélange logique, comparaison et arithmétique. La lisibilité est plus importante qu'une ligne compacte. Pour concaténation avec &, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding concatenation with &? Answer: I prefer explicit parentheses and visible conversions when an expression mixes logic, comparison, and arithmetic. Readability matters more than compactness. For concatenation with &, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
117. FR - Expliquez slices et attributs de tableau. Quelle structure ou idée hardware se cache derrière ? Réponse : Les slices sélectionnent une partie d'un tableau, tandis que `left`, `right`, `high`, `low`, `range` et `length` permettent d'écrire du RTL moins dépendant de tailles codées en dur.	EN - Explain slices and array attributes. What hardware structure or idea is behind it? Answer: Slices select part of an array, while `left`, `right`, `high`, `low`, `range`, and `length` allow RTL to be less dependent on hard-coded sizes.
118. FR - Comment utiliseriez-vous slices et attributs de tableau dans un design VHDL réel ? Réponse : J'utilise `data`range` dans les boucles et `counter`length` dans les conversions afin que le code reste robuste lorsque la largeur change.	EN - How would you use slices and array attributes in a real VHDL design? Answer: I use `data`range` in loops and `counter`length` in conversions so the code remains robust when widths change.

119. FR - Comment diagnostiqueriez-vous un problème lié à slices et attributs de tableau ? Réponse : Je vérifie la priorité des opérateurs, les tailles de vecteurs et surtout le type des opérandes. Une expression correcte syntaxiquement peut encore tronquer ou étendre une valeur de façon inattendue. Concernant slices et attributs de tableau, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving slices and array attributes? Answer: I check operator precedence, vector widths, and especially operand types. A syntactically correct expression can still truncate or extend a value unexpectedly. For slices and array attributes, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
120. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant slices et attributs de tableau ? Réponse : Je préfère des parenthèses explicites et des conversions visibles quand une expression mélange logique, comparaison et arithmétique. La lisibilité est plus importante qu'une ligne compacte. Pour slices et attributs de tableau, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding slices and array attributes? Answer: I prefer explicit parentheses and visible conversions when an expression mixes logic, comparison, and arithmetic. Readability matters more than compactness. For slices and array attributes, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

7. Signals, variables, constants et delta cycles / Signals, Variables, Constants, and Delta Cycles

121. FR - Expliquez signal assignment avec <=. Quelle structure ou idée hardware se cache derrière ? Réponse : Une affectation de signal avec `<=` programme une mise à jour selon la sémantique de simulation; la nouvelle valeur n'est pas immédiatement visible comme une variable dans la même séquence de statements. En logique synchrone, cela correspond bien à la mise à jour parallèle des registres.	EN - Explain signal assignment with <=. What hardware structure or idea is behind it? Answer: A signal assignment with `<=` schedules an update according to simulation semantics; the new value is not immediately visible like a variable within the same statement sequence. In synchronous logic, this matches parallel register updates.
122. FR - Comment utiliseriez-vous signal assignment avec <= dans un design VHDL réel ? Réponse : Dans un process clocké, plusieurs registres affectés avec `<=` prennent conceptuellement leur nouvelle valeur au même edge, chacun calculé depuis les anciennes valeurs.	EN - How would you use signal assignment with <= in a real VHDL design? Answer: In a clocked process, several registers assigned with `<=` conceptually take their new values on the same edge, each computed from old values.
123. FR - Comment diagnostiqueriez-vous un problème lié à signal assignment avec <= ? Réponse : Je regarde où la valeur est affectée, quand elle devient visible et si le designer confond `<=` avec `:=`. Beaucoup de bugs de simulation viennent d'une attente incorrecte sur l'update d'un signal dans le même process. Concernant signal assignment avec <=, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving signal assignment with <=? Answer: I inspect where a value is assigned, when it becomes visible, and whether the designer confuses `<=` with `:=`. Many simulation bugs come from incorrect expectations about signal updates within the same process. For signal assignment with <=, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
124. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signal assignment avec <= ? Réponse : Je choisis `signal` pour la communication entre blocs/process et `variable` pour le calcul local immédiat dans un process ou sous-programme. Je garde leur portée aussi petite que possible. Pour signal assignment avec <=, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding signal assignment with <=? Answer: I choose `signal` for communication between blocks/processes and `variable` for immediate local computation inside a process or subprogram. I keep their scope as small as possible. For signal assignment with <=, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
125. FR - Expliquez variables et opérateur :=. Quelle structure ou idée hardware se cache derrière ? Réponse : Une variable déclarée dans un process ou sous-programme est mise à jour immédiatement avec `:=` dans le flot séquentiel de ce process. Elle est utile pour les calculs intermédiaires.	EN - Explain variables and the := operator. What hardware structure or idea is behind it? Answer: A variable declared in a process or subprogram is updated immediately with `:=` in that process's sequential flow. It is useful for intermediate calculations.
126. FR - Comment utiliseriez-vous variables et opérateur := dans un design VHDL réel ? Réponse : Je peux calculer une somme combinatoire en plusieurs étapes dans une variable, puis affecter une seule fois le signal de sortie à la fin du process.	EN - How would you use variables and the := operator in a real VHDL design? Answer: I can compute a combinational sum in several steps using a variable, then assign the output signal once at the end of the process.

127. FR - Comment diagnostiqueriez-vous un problème lié à variables et opérateur := ? Réponse : Je regarde où la valeur est affectée, quand elle devient visible et si le designer confond `<=` avec `:=`. Beaucoup de bugs de simulation viennent d'une attente incorrecte sur l'update d'un signal dans le même process. Concernant variables et opérateur :=, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving variables and the := operator? Answer: I inspect where a value is assigned, when it becomes visible, and whether the designer confuses `<=` with `:=`. Many simulation bugs come from incorrect expectations about signal updates within the same process. For variables and the := operator, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
128. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant variables et opérateur := ? Réponse : Je choisis `signal` pour la communication entre blocs/process et `variable` pour le calcul local immédiat dans un process ou sous-programme. Je garde leur portée aussi petite que possible. Pour variables et opérateur :=, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding variables and the := operator? Answer: I choose `signal` for communication between blocks/processes and `variable` for immediate local computation inside a process or subprogram. I keep their scope as small as possible. For variables and the := operator, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
129. FR - Expliquez signal versus variable dans un process clocké. Quelle structure ou idée hardware se cache derrière ? Réponse : Dans un process clocké, une variable peut voir sa nouvelle valeur immédiatement pendant le même edge simulé, alors qu'un signal affecté avec `<=` conserve son ancienne valeur jusqu'à la mise à jour planifiée.	EN - Explain signal versus variable in a clocked process. What hardware structure or idea is behind it? Answer: In a clocked process, a variable can see its new value immediately during the same simulated edge, while a signal assigned with `<=` keeps its old value until the scheduled update.
130. FR - Comment utiliseriez-vous signal versus variable dans un process clocké dans un design VHDL réel ? Réponse : En entretien, je montre que `v := v + 1;` puis lecture de `v` voit le résultat, alors que `s <= s + 1;` puis lecture de `s` dans le même process voit encore l'ancienne valeur.	EN - How would you use signal versus variable in a clocked process in a real VHDL design? Answer: In an interview, I show that `v := v + 1;` followed by reading `v` sees the result, whereas `s <= s + 1;` followed by reading `s` in the same process still sees the old value.
131. FR - Comment diagnostiqueriez-vous un problème lié à signal versus variable dans un process clocké ? Réponse : Je regarde où la valeur est affectée, quand elle devient visible et si le designer confond `<=` avec `:=`. Beaucoup de bugs de simulation viennent d'une attente incorrecte sur l'update d'un signal dans le même process. Concernant signal versus variable dans un process clocké, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving signal versus variable in a clocked process? Answer: I inspect where a value is assigned, when it becomes visible, and whether the designer confuses `<=` with `:=`. Many simulation bugs come from incorrect expectations about signal updates within the same process. For signal versus variable in a clocked process, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
132. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant signal versus variable dans un process clocké ? Réponse : Je choisis `signal` pour la communication entre blocs/process et `variable` pour le calcul local immédiat dans un process ou sous-programme. Je garde leur portée aussi petite que possible. Pour signal versus variable dans un process clocké, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding signal versus variable in a clocked process? Answer: I choose `signal` for communication between blocks/processes and `variable` for immediate local computation inside a process or subprogram. I keep their scope as small as possible. For signal versus variable in a clocked process, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
133. FR - Expliquez constants. Quelle structure ou idée hardware se cache derrière ? Réponse : Une constante possède une valeur fixe après élaboration et peut être typée explicitement. Elle remplace les nombres magiques et peut être calculée à partir de generics.	EN - Explain constants. What hardware structure or idea is behind it? Answer: A constant has a fixed value after elaboration and can be explicitly typed. It replaces magic numbers and can be calculated from generics.
134. FR - Comment utiliseriez-vous constants dans un design VHDL réel ? Réponse : Je définis par exemple `CLKS_PER_BIT` à partir de `CLK_HZ` et `BAUD` pour rendre une UART paramétrable et le calcul visible au lecteur. <pre>constant CLKS_PER_BIT : positive := CLK_HZ / BAUD;</pre>	EN - How would you use constants in a real VHDL design? Answer: For example, I define `CLKS_PER_BIT` from `CLK_HZ` and `BAUD` to make a UART parameterizable and expose the calculation. <pre>constant CLKS_PER_BIT : positive := CLK_HZ / BAUD;</pre>

135. FR - Comment diagnostiqueriez-vous un problème lié à constants ? Réponse : Je regarde où la valeur est affectée, quand elle devient visible et si le designer confond `<code><=</code>` avec `<code>:=</code>`. Beaucoup de bugs de simulation viennent d'une attente incorrecte sur l'update d'un signal dans le même process. Concernant constants, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving constants? Answer: I inspect where a value is assigned, when it becomes visible, and whether the designer confuses `<code><=</code>` with `<code>:=</code>`. Many simulation bugs come from incorrect expectations about signal updates within the same process. For constants, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
136. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant constants ? Réponse : Je choisis `signal` pour la communication entre blocs/process et `variable` pour le calcul local immédiat dans un process ou sous-programme. Je garde leur portée aussi petite que possible. Pour constants, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding constants? Answer: I choose `signal` for communication between blocks/processes and `variable` for immediate local computation inside a process or subprogram. I keep their scope as small as possible. For constants, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
137. FR - Expliquez delta cycles. Quelle structure ou idée hardware se cache derrière ? Réponse : Un delta cycle est une étape de simulation sans avance de temps physique utilisée pour résoudre les événements et dépendances entre process/signaux. Plusieurs delta cycles peuvent se produire au même timestamp.	EN - Explain delta cycles. What hardware structure or idea is behind it? Answer: A delta cycle is a simulation step with no physical time advance, used to resolve events and dependencies between processes/signals. Several delta cycles can occur at the same timestamp.
138. FR - Comment utiliseriez-vous delta cycles dans un design VHDL réel ? Réponse : Quand une waveform montre des changements au même temps mais dans un ordre inattendu, je regarde les delta cycles et la chaîne d'affectations avant de conclure à un problème matériel.	EN - How would you use delta cycles in a real VHDL design? Answer: When a waveform shows changes at the same time in an unexpected order, I inspect delta cycles and the assignment chain before concluding there is a hardware problem.
139. FR - Comment diagnostiqueriez-vous un problème lié à delta cycles ? Réponse : Je regarde où la valeur est affectée, quand elle devient visible et si le designer confond `<code><=</code>` avec `<code>:=</code>`. Beaucoup de bugs de simulation viennent d'une attente incorrecte sur l'update d'un signal dans le même process. Concernant delta cycles, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving delta cycles? Answer: I inspect where a value is assigned, when it becomes visible, and whether the designer confuses `<code><=</code>` with `<code>:=</code>`. Many simulation bugs come from incorrect expectations about signal updates within the same process. For delta cycles, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
140. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant delta cycles ? Réponse : Je choisis `signal` pour la communication entre blocs/process et `variable` pour le calcul local immédiat dans un process ou sous-programme. Je garde leur portée aussi petite que possible. Pour delta cycles, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding delta cycles? Answer: I choose `signal` for communication between blocks/processes and `variable` for immediate local computation inside a process or subprogram. I keep their scope as small as possible. For delta cycles, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

8. Affectations concurrentes et style dataflow / Concurrent Assignments and Dataflow Style

141. FR - Expliquez affectation concurrente simple. Quelle structure ou idée hardware se cache derrière ? Réponse : Une affectation concurrente simple décrit une relation combinatoire continue entre entrées et sortie. En synthèse, elle devient la logique nécessaire pour réaliser l'expression.	EN - Explain simple concurrent assignment. What hardware structure or idea is behind it? Answer: A simple concurrent assignment describes a continuous combinational relationship between inputs and output. In synthesis, it becomes the logic needed to implement the expression.
142. FR - Comment utiliseriez-vous affectation concurrente simple dans un design VHDL réel ? Réponse : <code>alarm <= over_temp or over_current;</code> décrit directement une fonction OR entre deux conditions déjà calculées.	EN - How would you use simple concurrent assignment in a real VHDL design? Answer: <code>alarm <= over_temp or over_current;</code> directly describes an OR function between two already computed conditions.

143. FR - Comment diagnostiqueriez-vous un problème lié à affectation concurrente simple ? Réponse : Je vérifie que toutes les branches définissent un résultat et que les conditions ne se chevauchent pas involontairement. Pour mux et décodeurs, une table de vérité ou une simulation exhaustive révèle rapidement les erreurs. Concernant affectation concurrente simple, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving simple concurrent assignment? Answer: I verify that all branches define a result and that conditions do not unintentionally overlap. For muxes and decoders, a truth table or exhaustive simulation quickly reveals errors. For simple concurrent assignment, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
144. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant affectation concurrente simple ? Réponse : J'utilise le style concurrent lorsque la fonction est naturellement combinatoire et compacte. Si la logique devient difficile à lire, un process combinatoire structuré est souvent préférable. Pour affectation concurrente simple, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding simple concurrent assignment? Answer: I use concurrent style when the function is naturally combinational and compact. If logic becomes hard to read, a structured combinational process is often preferable. For simple concurrent assignment, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
145. FR - Expliquez conditional assignment when/else. Quelle structure ou idée hardware se cache derrière ? Réponse : La forme `value when condition else other` décrit un choix combinatoire; des conditions chaînées créent une priorité. Elle convient aux multiplexeurs simples.	EN - Explain conditional assignment with when/else. What hardware structure or idea is behind it? Answer: The `value when condition else other` form describes a combinational choice; chained conditions create priority. It suits simple multiplexers.
146. FR - Comment utiliseriez-vous conditional assignment when/else dans un design VHDL réel ? Réponse : Un mux 2:1 s'écrit `y <= a when sel='0' else b;`, ce qui synthétise un multiplexeur piloté par `sel`. <pre>y <= a when sel = '0' else b;</pre>	EN - How would you use conditional assignment with when/else in a real VHDL design? Answer: A 2:1 mux can be written `y <= a when sel='0' else b;`, synthesizing a multiplexer controlled by `sel`. <pre>y <= a when sel = '0' else b;</pre>
147. FR - Comment diagnostiqueriez-vous un problème lié à conditional assignment when/else ? Réponse : Je vérifie que toutes les branches définissent un résultat et que les conditions ne se chevauchent pas involontairement. Pour mux et décodeurs, une table de vérité ou une simulation exhaustive révèle rapidement les erreurs. Concernant conditional assignment when/else, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving conditional assignment with when/else? Answer: I verify that all branches define a result and that conditions do not unintentionally overlap. For muxes and decoders, a truth table or exhaustive simulation quickly reveals errors. For conditional assignment with when/else, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
148. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant conditional assignment when/else ? Réponse : J'utilise le style concurrent lorsque la fonction est naturellement combinatoire et compacte. Si la logique devient difficile à lire, un process combinatoire structuré est souvent préférable. Pour conditional assignment when/else, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding conditional assignment with when/else? Answer: I use concurrent style when the function is naturally combinational and compact. If logic becomes hard to read, a structured combinational process is often preferable. For conditional assignment with when/else, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
149. FR - Expliquez selected assignment with/select. Quelle structure ou idée hardware se cache derrière ? Réponse : `with ... select` exprime un choix parmi plusieurs valeurs selon une expression de sélection. Il convient bien à un mux multi-voies ou à une table de décodage combinatoire.	EN - Explain selected assignment with/select. What hardware structure or idea is behind it? Answer: `with ... select` expresses a choice among several values based on a select expression. It suits a multi-way mux or combinational decode table.
150. FR - Comment utiliseriez-vous selected assignment with/select dans un design VHDL réel ? Réponse : Je fournis `when others` lorsque toutes les valeurs possibles ne sont pas explicitement listées. <pre>with sel select y <= a when "00", b when "01", c when "10", d when others;</pre>	EN - How would you use selected assignment with/select in a real VHDL design? Answer: I provide `when others` when not all possible values are explicitly listed. <pre>with sel select y <= a when "00", b when "01", c when "10", d when others;</pre>

151. FR - Comment diagnostiqueriez-vous un problème lié à selected assignment with/select ? Réponse : Je vérifie que toutes les branches définissent un résultat et que les conditions ne se chevauchent pas involontairement. Pour mux et décodeurs, une table de vérité ou une simulation exhaustive révèle rapidement les erreurs. Concernant selected assignment with/select, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving selected assignment with/select? Answer: I verify that all branches define a result and that conditions do not unintentionally overlap. For muxes and decoders, a truth table or exhaustive simulation quickly reveals errors. For selected assignment with/select, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
152. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant selected assignment with/select ? Réponse : J'utilise le style concurrent lorsque la fonction est naturellement combinatoire et compacte. Si la logique devient difficile à lire, un process combinatoire structuré est souvent préférable. Pour selected assignment with/select, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding selected assignment with/select? Answer: I use concurrent style when the function is naturally combinational and compact. If logic becomes hard to read, a structured combinational process is often preferable. For selected assignment with/select, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
153. FR - Expliquez priorité implicite. Quelle structure ou idée hardware se cache derrière ? Réponse : Des `when/else` chaînés peuvent créer une logique de priorité: la première condition vraie sélectionne sa valeur. Cette priorité devient une vraie structure matérielle et peut allonger le chemin combinatoire.	EN - Explain implicit priority. What hardware structure or idea is behind it? Answer: Chained `when/else` statements can create priority logic: the first true condition selects its value. This priority becomes real hardware and can lengthen the combinational path.
154. FR - Comment utiliseriez-vous priorité implicite dans un design VHDL réel ? Réponse : Pour un arbitre, la priorité peut être voulue; pour un décodeur mutuellement exclusif, je préfère une structure montrant clairement que les choix ne se chevauchent pas.	EN - How would you use implicit priority in a real VHDL design? Answer: For an arbiter, priority may be intentional; for a mutually exclusive decoder, I prefer a structure clearly showing choices do not overlap.
155. FR - Comment diagnostiqueriez-vous un problème lié à priorité implicite ? Réponse : Je vérifie que toutes les branches définissent un résultat et que les conditions ne se chevauchent pas involontairement. Pour mux et décodeurs, une table de vérité ou une simulation exhaustive révèle rapidement les erreurs. Concernant priorité implicite, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving implicit priority? Answer: I verify that all branches define a result and that conditions do not unintentionally overlap. For muxes and decoders, a truth table or exhaustive simulation quickly reveals errors. For implicit priority, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
156. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant priorité implicite ? Réponse : J'utilise le style concurrent lorsque la fonction est naturellement combinatoire et compacte. Si la logique devient difficile à lire, un process combinatoire structuré est souvent préférable. Pour priorité implicite, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding implicit priority? Answer: I use concurrent style when the function is naturally combinational and compact. If logic becomes hard to read, a structured combinational process is often preferable. For implicit priority, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
157. FR - Expliquez multiple drivers concurrents. Quelle structure ou idée hardware se cache derrière ? Réponse : Deux sources qui pilotent le même `std_logic` créent plusieurs drivers. Le type résolu peut produire `X` lors d'un conflit; dans du RTL interne classique, plusieurs drivers sont souvent une erreur d'architecture.	EN - Explain multiple concurrent drivers. What hardware structure or idea is behind it? Answer: Two sources driving the same `std_logic` create multiple drivers. The resolved type can produce `X` on contention; in normal internal RTL, multiple drivers often indicate an architecture error.
158. FR - Comment utiliseriez-vous multiple drivers concurrents dans un design VHDL réel ? Réponse : Je cherche un seul propriétaire logique par registre ou signal interne et je combine les conditions dans ce propriétaire plutôt que de répartir les affectations sur plusieurs process.	EN - How would you use multiple concurrent drivers in a real VHDL design? Answer: I aim for one logical owner per internal register or signal and combine conditions in that owner rather than distributing assignments across several processes.

159. FR - Comment diagnostiqueriez-vous un problème lié à multiple drivers concurrents ? Réponse : Je vérifie que toutes les branches définissent un résultat et que les conditions ne se chevauchent pas involontairement. Pour mux et décodeurs, une table de vérité ou une simulation exhaustive révèle rapidement les erreurs. Concernant multiple drivers concurrents, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving multiple concurrent drivers? Answer: I verify that all branches define a result and that conditions do not unintentionally overlap. For muxes and decoders, a truth table or exhaustive simulation quickly reveals errors. For multiple concurrent drivers, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
160. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant multiple drivers concurrents ? Réponse : J'utilise le style concurrent lorsque la fonction est naturellement combinatoire et compacte. Si la logique devient difficile à lire, un process combinatoire structuré est souvent préférable. Pour multiple drivers concurrents, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding multiple concurrent drivers? Answer: I use concurrent style when the function is naturally combinational and compact. If logic becomes hard to read, a structured combinational process is often preferable. For multiple concurrent drivers, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

9. Process, sensitivity list et statements séquentiels / Processes, Sensitivity Lists, and Sequential Statements

161. FR - Expliquez process combinatoire. Quelle structure ou idée hardware se cache derrière ? Réponse : Un process combinatoire calcule des sorties uniquement à partir des entrées courantes et ne doit pas mémoriser implicitement une valeur. Toutes les entrées lues doivent provoquer sa réévaluation en simulation.	EN - Explain combinational process. What hardware structure or idea is behind it? Answer: A combinational process computes outputs only from current inputs and must not implicitly retain a value. All read inputs must cause reevaluation in simulation.
162. FR - Comment utiliseriez-vous process combinatoire dans un design VHDL réel ? Réponse : Je donne d'abord des valeurs par défaut aux sorties, puis je surcharge avec `if` ou `case`; cela évite les latches involontaires. <pre>process(all) begin y <= '0'; if enable = '1' then y <= a and b; end if; end process;</pre>	EN - How would you use combinational process in a real VHDL design? Answer: I first assign default values to outputs, then override them using `if` or `case`; this avoids unintended latches. <pre>process(all) begin y <= '0'; if enable = '1' then y <= a and b; end if; end process;</pre>
163. FR - Comment diagnostiqueriez-vous un problème lié à process combinatoire ? Réponse : Je vérifie si le process est combinatoire ou clocké, si la sensitivity list est correcte et si chaque sortie combinatoire reçoit une valeur sur tous les chemins. J'inspecte aussi les variables et l'ordre des statements internes. Concernant process combinatoire, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving combinational process? Answer: I verify whether the process is combinational or clocked, whether the sensitivity list is correct, and whether every combinational output receives a value on all paths. I also inspect variables and internal statement order. For combinational process, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
164. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant process combinatoire ? Réponse : Je choisis un style clair: `process(all)` pour du combinatoire VHDL-2008 et un process explicite sur clock/reset pour du séquentiel. Je ne mélange pas inutilement plusieurs intentions. Pour process combinatoire, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding combinational process? Answer: I choose a clear style: `process(all)` for VHDL-2008 combinational logic and an explicit clock/reset process for sequential logic. I avoid unnecessarily mixing intentions. For combinational process, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
165. FR - Expliquez sensitivity list explicite. Quelle structure ou idée hardware se cache derrière ? Réponse : Dans les styles VHDL classiques, un process combinatoire doit lister tous les signaux lus. Oublier un signal peut créer un écart simulation/synthèse: le hardware réagit mais la simulation ne réveille pas le process.	EN - Explain explicit sensitivity list. What hardware structure or idea is behind it? Answer: In classic VHDL styles, a combinational process must list all signals it reads. Missing one can create a simulation/synthesis mismatch: hardware reacts but simulation does not wake the process.
166. FR - Comment utiliseriez-vous sensitivity list explicite dans un design VHDL réel ? Réponse : Si le projet utilise VHDL-2008, `process(all)` réduit ce risque; sinon je révise la liste dès qu'une entrée est ajoutée au calcul.	EN - How would you use explicit sensitivity list in a real VHDL design? Answer: If the project uses VHDL-2008, `process(all)` reduces this risk; otherwise I review the list whenever an input is added to the calculation.

167. FR - Comment diagnostiqueriez-vous un problème lié à sensitivity list explicite ? Réponse : Je vérifie si le process est combinatoire ou clocké, si la sensitivity list est correcte et si chaque sortie combinatoire reçoit une valeur sur tous les chemins. J'inspecte aussi les variables et l'ordre des statements internes. Concernant sensitivity list explicite, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving explicit sensitivity list? Answer: I verify whether the process is combinational or clocked, whether the sensitivity list is correct, and whether every combinational output receives a value on all paths. I also inspect variables and internal statement order. For explicit sensitivity list, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
168. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant sensitivity list explicite ? Réponse : Je choisis un style clair: `process(all)` pour du combinatoire VHDL-2008 et un process explicite sur clock/reset pour du séquentiel. Je ne mélange pas inutilement plusieurs intentions. Pour sensitivity list explicite, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding explicit sensitivity list? Answer: I choose a clear style: `process(all)` for VHDL-2008 combinational logic and an explicit clock/reset process for sequential logic. I avoid unnecessarily mixing intentions. For explicit sensitivity list, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
169. FR - Expliquez if/elsif/else. Quelle structure ou idée hardware se cache derrière ? Réponse : `if/elsif/else` est un statement séquentiel utilisable dans un process. En combinatoire, des conditions chaînées peuvent créer une priorité; en clocké, elles décrivent souvent enable, reset ou choix de prochaine valeur.	EN - Explain if/elsif/else. What hardware structure or idea is behind it? Answer: `if/elsif/else` is a sequential statement used inside a process. In combinational logic, chained conditions can create priority; in clocked logic, they often describe enable, reset, or next-value choices.
170. FR - Comment utiliseriez-vous if/elsif/else dans un design VHDL réel ? Réponse : Je place les conditions selon la priorité matérielle réellement souhaitée et j'évite une longue chaîne si un `case` ou une FSM rend l'intention plus évidente.	EN - How would you use if/elsif/else in a real VHDL design? Answer: I order conditions according to the actual desired hardware priority and avoid a long chain when a `case` or FSM makes the intent clearer.
171. FR - Comment diagnostiqueriez-vous un problème lié à if/elsif/else ? Réponse : Je vérifie si le process est combinatoire ou clocké, si la sensitivity list est correcte et si chaque sortie combinatoire reçoit une valeur sur tous les chemins. J'inspecte aussi les variables et l'ordre des statements internes. Concernant if/elsif/else, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving if/elsif/else? Answer: I verify whether the process is combinational or clocked, whether the sensitivity list is correct, and whether every combinational output receives a value on all paths. I also inspect variables and internal statement order. For if/elsif/else, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
172. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant if/elsif/else ? Réponse : Je choisis un style clair: `process(all)` pour du combinatoire VHDL-2008 et un process explicite sur clock/reset pour du séquentiel. Je ne mélange pas inutilement plusieurs intentions. Pour if/elsif/else, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding if/elsif/else? Answer: I choose a clear style: `process(all)` for VHDL-2008 combinational logic and an explicit clock/reset process for sequential logic. I avoid unnecessarily mixing intentions. For if/elsif/else, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
173. FR - Expliquez case/when. Quelle structure ou idée hardware se cache derrière ? Réponse : `case` sélectionne une branche selon une expression discrète et exige une couverture complète, directement ou via `when others`. C'est particulièrement lisible pour les FSM et décodeurs.	EN - Explain case/when. What hardware structure or idea is behind it? Answer: `case` selects a branch based on a discrete expression and requires complete coverage, directly or through `when others`. It is especially readable for FSMs and decoders.
174. FR - Comment utiliseriez-vous case/when dans un design VHDL réel ? Réponse : Dans une FSM, j'initialise d'abord `next_state <= state`, puis `case state is` modifie le prochain état seulement lorsque la condition de transition est vraie.	EN - How would you use case/when in a real VHDL design? Answer: In an FSM, I first set `next_state <= state`, then `case state is` changes next state only when the transition condition is true.

175. FR - Comment diagnostiqueriez-vous un problème lié à case/when ? Réponse : Je vérifie si le process est combinatoire ou clocké, si la sensitivity list est correcte et si chaque sortie combinatoire reçoit une valeur sur tous les chemins. J'inspecte aussi les variables et l'ordre des statements internes. Concernant case/when, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving case/when? Answer: I verify whether the process is combinational or clocked, whether the sensitivity list is correct, and whether every combinational output receives a value on all paths. I also inspect variables and internal statement order. For case/when, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
176. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant case/when ? Réponse : Je choisis un style clair: `process(all)` pour du combinatoire VHDL-2008 et un process explicite sur clock/reset pour du séquentiel. Je ne mélange pas inutilement plusieurs intentions. Pour case/when, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding case/when? Answer: I choose a clear style: `process(all)` for VHDL-2008 combinational logic and an explicit clock/reset process for sequential logic. I avoid unnecessarily mixing intentions. For case/when, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
177. FR - Expliquez for loop dans le RTL. Quelle structure ou idée hardware se cache derrière ? Réponse : Une boucle `for` à bornes statiquement déterminables est généralement déroulée par la synthèse: elle décrit plusieurs opérations matérielles, pas une boucle exécutée dans le temps comme sur un CPU.	EN - Explain for loops in RTL. What hardware structure or idea is behind it? Answer: A `for` loop with statically determinable bounds is generally unrolled by synthesis: it describes multiple hardware operations, not a loop executed over time like on a CPU.
178. FR - Comment utiliseriez-vous for loop dans le RTL dans un design VHDL réel ? Réponse : Je peux utiliser une boucle pour réduire un vecteur ou initialiser un tableau, tout en gardant en tête le coût matériel si chaque itération crée de la logique parallèle.	EN - How would you use for loops in RTL in a real VHDL design? Answer: I can use a loop to reduce a vector or initialize an array while remembering the hardware cost if every iteration creates parallel logic.
179. FR - Comment diagnostiqueriez-vous un problème lié à for loop dans le RTL ? Réponse : Je vérifie si le process est combinatoire ou clocké, si la sensitivity list est correcte et si chaque sortie combinatoire reçoit une valeur sur tous les chemins. J'inspecte aussi les variables et l'ordre des statements internes. Concernant for loop dans le RTL, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving for loops in RTL? Answer: I verify whether the process is combinational or clocked, whether the sensitivity list is correct, and whether every combinational output receives a value on all paths. I also inspect variables and internal statement order. For for loops in RTL, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
180. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant for loop dans le RTL ? Réponse : Je choisis un style clair: `process(all)` pour du combinatoire VHDL-2008 et un process explicite sur clock/reset pour du séquentiel. Je ne mélange pas inutilement plusieurs intentions. Pour for loop dans le RTL, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding for loops in RTL? Answer: I choose a clear style: `process(all)` for VHDL-2008 combinational logic and an explicit clock/reset process for sequential logic. I avoid unnecessarily mixing intentions. For for loops in RTL, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

10. Logique combinatoire: mux, décodeurs et comparateurs / Combinational Logic: Muxes, Decoders, and Comparators

181. FR - Expliquez multiplexeur 2:1. Quelle structure ou idée hardware se cache derrière ? Réponse : Un mux 2:1 sélectionne l'une de deux entrées selon `sel`. Table fonctionnelle: `sel=0 -> y=a`, `sel=1 -> y=b`.	EN - Explain 2-to-1 multiplexer. What hardware structure or idea is behind it? Answer: A 2-to-1 mux selects one of two inputs based on `sel`. Functional table: `sel=0 -> y=a`, `sel=1 -> y=b`.
182. FR - Comment utiliseriez-vous multiplexeur 2:1 dans un design VHDL réel ? Réponse : Je peux l'écrire par affectation conditionnelle ou dans un process combinatoire; la synthèse doit produire une fonction de multiplexage équivalente. <pre>-- sel y -- 0 a -- 1 b y <= a when sel='0' else b;</pre>	EN - How would you use 2-to-1 multiplexer in a real VHDL design? Answer: I can write it as a conditional assignment or inside a combinational process; synthesis should produce equivalent multiplexing logic. <pre>-- sel y -- 0 a -- 1 b y <= a when sel='0' else b;</pre>

183. FR - Comment diagnostiqueriez-vous un problème lié à multiplexeur 2:1 ? Réponse : Je construis ou vérifie une table de vérité, puis je m'assure que toutes les combinaisons d'entrée ont une sortie définie. Un latch involontaire ou une priorité non désirée est le premier suspect. Concernant multiplexeur 2:1, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving 2-to-1 multiplexer? Answer: I build or check a truth table, then ensure every input combination has a defined output. An unintended latch or unwanted priority is the first suspect. For 2-to-1 multiplexer, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
184. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant multiplexeur 2:1 ? Réponse : Je décris la fonction le plus directement possible et je laisse la synthèse minimiser les portes. Je n'essaie pas de dessiner manuellement chaque LUT sauf raison particulière. Pour multiplexeur 2:1, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding 2-to-1 multiplexer? Answer: I describe the function as directly as possible and let synthesis minimize gates. I do not manually design every LUT unless there is a specific reason. For 2-to-1 multiplexer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
185. FR - Expliquez décodeur one-hot. Quelle structure ou idée hardware se cache derrière ? Réponse : Un décodeur transforme un code binaire en sorties sélectionnées, souvent une seule sortie active. Il faut définir le comportement pour les codes invalides ou inutilisés.	EN - Explain one-hot decoder. What hardware structure or idea is behind it? Answer: A decoder converts a binary code into selected outputs, often with exactly one active output. Behavior for invalid or unused codes must be defined.
186. FR - Comment utiliseriez-vous décodeur one-hot dans un design VHDL réel ? Réponse : Je donne une valeur par défaut `0000`, puis un `case` active le bit correspondant; le testbench peut vérifier qu'un seul bit est actif.	EN - How would you use one-hot decoder in a real VHDL design? Answer: I assign a default `0000`, then use a `case` to activate the matching bit; the testbench can verify that exactly one bit is active.
187. FR - Comment diagnostiqueriez-vous un problème lié à décodeur one-hot ? Réponse : Je construis ou vérifie une table de vérité, puis je m'assure que toutes les combinaisons d'entrée ont une sortie définie. Un latch involontaire ou une priorité non désirée est le premier suspect. Concernant décodeur one-hot, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving one-hot decoder? Answer: I build or check a truth table, then ensure every input combination has a defined output. An unintended latch or unwanted priority is the first suspect. For one-hot decoder, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
188. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant décodeur one-hot ? Réponse : Je décris la fonction le plus directement possible et je laisse la synthèse minimiser les portes. Je n'essaie pas de dessiner manuellement chaque LUT sauf raison particulière. Pour décodeur one-hot, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding one-hot decoder? Answer: I describe the function as directly as possible and let synthesis minimize gates. I do not manually design every LUT unless there is a specific reason. For one-hot decoder, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
189. FR - Expliquez encodeur et priority encoder. Quelle structure ou idée hardware se cache derrière ? Réponse : Un encodeur convertit des entrées one-hot vers un code; un priority encoder définit aussi quelle entrée gagne si plusieurs sont actives. Cette priorité crée une vraie structure matérielle.	EN - Explain encoder and priority encoder. What hardware structure or idea is behind it? Answer: An encoder converts one-hot inputs into a code; a priority encoder also defines which input wins when several are active. That priority creates real hardware.
190. FR - Comment utiliseriez-vous encodeur et priority encoder dans un design VHDL réel ? Réponse : Je documente explicitement la priorité ou je déclare l'état multi-actif invalide afin que l'intention soit claire.	EN - How would you use encoder and priority encoder in a real VHDL design? Answer: I explicitly document priority or declare the multi-active state invalid so the intent is clear.
191. FR - Comment diagnostiqueriez-vous un problème lié à encodeur et priority encoder ? Réponse : Je construis ou vérifie une table de vérité, puis je m'assure que toutes les combinaisons d'entrée ont une sortie définie. Un latch involontaire ou une priorité non désirée est le premier suspect. Concernant encodeur et priority encoder, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving encoder and priority encoder? Answer: I build or check a truth table, then ensure every input combination has a defined output. An unintended latch or unwanted priority is the first suspect. For encoder and priority encoder, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

192. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant encodeur et priority encoder ? Réponse : Je décris la fonction le plus directement possible et je laisse la synthèse minimiser les portes. Je n'essaie pas de dessiner manuellement chaque LUT sauf raison particulière. Pour encodeur et priority encoder, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding encoder and priority encoder? Answer: I describe the function as directly as possible and let synthesis minimize gates. I do not manually design every LUT unless there is a specific reason. For encoder and priority encoder, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
193. FR - Expliquez comparateur numérique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un comparateur teste égalité ou ordre de deux valeurs. Le résultat diffère si les bits sont interprétés comme `unsigned` ou `signed`, surtout lorsque le MSB vaut 1.	EN - Explain numeric comparator. What hardware structure or idea is behind it? Answer: A comparator tests equality or ordering of two values. The result differs when bits are interpreted as `unsigned` or `signed`, especially when the MSB is 1.
194. FR - Comment utiliseriez-vous comparateur numérique dans un design VHDL réel ? Réponse : Je convertis les bus vers le type numérique correct avant `<` ou `>` et je teste zéro, maximum, minimum et valeurs négatives si signé.	EN - How would you use numeric comparator in a real VHDL design? Answer: I convert buses to the correct numeric type before `<` or `>` and test zero, maximum, minimum, and negative values if signed.
195. FR - Comment diagnostiqueriez-vous un problème lié à comparateur numérique ? Réponse : Je construis ou vérifie une table de vérité, puis je m'assure que toutes les combinaisons d'entrée ont une sortie définie. Un latch involontaire ou une priorité non désirée est le premier suspect. Concernant comparateur numérique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving numeric comparator? Answer: I build or check a truth table, then ensure every input combination has a defined output. An unintended latch or unwanted priority is the first suspect. For numeric comparator, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
196. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant comparateur numérique ? Réponse : Je décris la fonction le plus directement possible et je laisse la synthèse minimiser les portes. Je n'essaie pas de dessiner manuellement chaque LUT sauf raison particulière. Pour comparateur numérique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding numeric comparator? Answer: I describe the function as directly as possible and let synthesis minimize gates. I do not manually design every LUT unless there is a specific reason. For numeric comparator, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
197. FR - Expliquez additionneur combinatoire et carry. Quelle structure ou idée hardware se cache derrière ? Réponse : Une addition de N bits peut nécessiter N+1 bits pour conserver le carry. Si le résultat reste sur N bits, l'overflow est tronqué modulo 2^N.	EN - Explain combinational adder and carry. What hardware structure or idea is behind it? Answer: An N-bit addition may require N+1 bits to preserve carry. If the result remains N bits wide, overflow is truncated modulo 2^N.
198. FR - Comment utiliseriez-vous additionneur combinatoire et carry dans un design VHDL réel ? Réponse : J'étends les opérandes avant l'addition pour obtenir un résultat N+1 bits et vérifier explicitement le carry.	EN - How would you use combinational adder and carry in a real VHDL design? Answer: I extend operands before addition to obtain an N+1-bit result and explicitly inspect carry.
199. FR - Comment diagnostiqueriez-vous un problème lié à additionneur combinatoire et carry ? Réponse : Je construis ou vérifie une table de vérité, puis je m'assure que toutes les combinaisons d'entrée ont une sortie définie. Un latch involontaire ou une priorité non désirée est le premier suspect. Concernant additionneur combinatoire et carry, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving combinational adder and carry? Answer: I build or check a truth table, then ensure every input combination has a defined output. An unintended latch or unwanted priority is the first suspect. For combinational adder and carry, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
200. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant additionneur combinatoire et carry ? Réponse : Je décris la fonction le plus directement possible et je laisse la synthèse minimiser les portes. Je n'essaie pas de dessiner manuellement chaque LUT sauf raison particulière. Pour additionneur combinatoire et carry, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding combinational adder and carry? Answer: I describe the function as directly as possible and let synthesis minimize gates. I do not manually design every LUT unless there is a specific reason. For combinational adder and carry, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

11. Logique séquentielle: flip-flops et registres / Sequential Logic: Flip-Flops and Registers

201. FR - Expliquez D flip-flop sur rising_edge. Quelle structure ou idée hardware se cache derrière ?

Réponse : Le pattern `if rising\_edge(clk) then q <= d; end if;` décrit un registre D mis à jour au front montant. Entre les fronts, le registre conserve sa valeur.

EN - Explain D flip-flop on rising_edge. What hardware structure or idea is behind it?

Answer: The pattern `if rising\_edge(clk) then q <= d; end if;` describes a D register updated on the rising edge. Between edges, the register holds its value.

202. FR - Comment utiliseriez-vous D flip-flop sur rising_edge dans un design VHDL réel ?

Réponse : C'est le bloc de base du RTL synchrone; je l'utilise pour pipeline, état, compteurs et capture de données.

```
process(clk)
begin
    if rising_edge(clk) then
        q <= d;
    end if;
end process;
```

EN - How would you use D flip-flop on rising_edge in a real VHDL design?

Answer: This is the basic building block of synchronous RTL; I use it for pipelines, state, counters, and data capture.

```
process(clk)
begin
    if rising_edge(clk) then
        q <= d;
    end if;
end process;
```

203. FR - Comment diagnostiqueriez-vous un problème lié à D flip-flop sur rising_edge ?

Réponse : Je vérifie la présence d'un edge d'horloge correct, la priorité reset/enable et les valeurs retenues lorsqu'aucune condition n'est vraie. Je m'assure aussi qu'un signal censé être registre n'a qu'un seul driver clocké. Concernant D flip-flop sur rising_edge, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.

EN - How would you troubleshoot an issue involving D flip-flop on rising_edge?

Answer: I verify the correct clock edge, reset/enable priority, and held values when no condition is true. I also ensure that a signal intended as a register has only one clocked driver. For D flip-flop on rising_edge, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

204. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant D flip-flop sur rising_edge ?

Réponse : Je garde un style de registre uniforme dans tout le projet. Les décisions de reset et enable doivent être architecturales, pas ajoutées automatiquement à chaque registre. Pour D flip-flop sur rising_edge, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.

EN - What best practice or interview trap should you know regarding D flip-flop on rising_edge?

Answer: I keep a uniform register style across the project. Reset and enable decisions should be architectural, not automatically added to every register. For D flip-flop on rising_edge, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

205. FR - Expliquez registre avec clock enable. Quelle structure ou idée hardware se cache derrière ?

Réponse : Un clock enable garde l'horloge globale inchangée et choisit si le registre prend une nouvelle valeur au front. C'est généralement préférable à fabriquer une horloge locale avec de la logique RTL.

EN - Explain register with clock enable. What hardware structure or idea is behind it?

Answer: A clock enable leaves the global clock unchanged and chooses whether the register takes a new value on the edge. It is generally preferable to creating a local clock with RTL logic.

206. FR - Comment utiliseriez-vous registre avec clock enable dans un design VHDL réel ?

Réponse : J'écris l'enable à l'intérieur du `rising\_edge`; quand `en=0`, l'absence d'affectation signifie que le registre conserve sa valeur.

EN - How would you use register with clock enable in a real VHDL design?

Answer: I write the enable inside the `rising\_edge`; when `en=0`, lack of assignment means the register holds its value.

207. FR - Comment diagnostiqueriez-vous un problème lié à registre avec clock enable ?

Réponse : Je vérifie la présence d'un edge d'horloge correct, la priorité reset/enable et les valeurs retenues lorsqu'aucune condition n'est vraie. Je m'assure aussi qu'un signal censé être registre n'a qu'un seul driver clocké. Concernant registre avec clock enable, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.

EN - How would you troubleshoot an issue involving register with clock enable?

Answer: I verify the correct clock edge, reset/enable priority, and held values when no condition is true. I also ensure that a signal intended as a register has only one clocked driver. For register with clock enable, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

208. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant registre avec clock enable ?

Réponse : Je garde un style de registre uniforme dans tout le projet. Les décisions de reset et enable doivent être architecturales, pas ajoutées automatiquement à chaque registre. Pour registre avec clock enable, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.

EN - What best practice or interview trap should you know regarding register with clock enable?

Answer: I keep a uniform register style across the project. Reset and enable decisions should be architectural, not automatically added to every register. For register with clock enable, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

209. FR - Expliquez registre parallèle et pipeline. Quelle structure ou idée hardware se cache derrière ? Réponse : Un vecteur affecté dans un process clocké infère un banc de flip-flops. Plusieurs étages successifs peuvent former un pipeline qui réduit la longueur des chemins combinatoires au prix de cycles de latence.	EN - Explain parallel register and pipeline. What hardware structure or idea is behind it? Answer: A vector assigned in a clocked process infers a bank of flip-flops. Multiple successive stages can form a pipeline that reduces combinational path length at the cost of cycle latency.
210. FR - Comment utiliseriez-vous registre parallèle et pipeline dans un design VHDL réel ? Réponse : Pour une multiplication/addition rapide, je peux enregistrer les entrées, le produit puis la somme sur plusieurs cycles pour améliorer la fréquence maximale.	EN - How would you use parallel register and pipeline in a real VHDL design? Answer: For a fast multiply/add path, I can register inputs, then product, then sum across several cycles to improve maximum clock frequency.
211. FR - Comment diagnostiqueriez-vous un problème lié à registre parallèle et pipeline ? Réponse : Je vérifie la présence d'un edge d'horloge correct, la priorité reset/enable et les valeurs retenues lorsqu'aucune condition n'est vraie. Je m'assure aussi qu'un signal censé être registre n'a qu'un seul driver clocké. Concernant registre parallèle et pipeline, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving parallel register and pipeline? Answer: I verify the correct clock edge, reset/enable priority, and held values when no condition is true. I also ensure that a signal intended as a register has only one clocked driver. For parallel register and pipeline, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
212. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant registre parallèle et pipeline ? Réponse : Je garde un style de registre uniforme dans tout le projet. Les décisions de reset et enable doivent être architecturales, pas ajoutées automatiquement à chaque registre. Pour registre parallèle et pipeline, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding parallel register and pipeline? Answer: I keep a uniform register style across the project. Reset and enable decisions should be architectural, not automatically added to every register. For parallel register and pipeline, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
213. FR - Expliquez shift register. Quelle structure ou idée hardware se cache derrière ? Réponse : Un shift register déplace les bits à chaque cycle et peut servir à sérialiser/désérialiser, retarder un signal ou détecter des motifs. La direction du shift doit être explicite.	EN - Explain shift register. What hardware structure or idea is behind it? Answer: A shift register moves bits each cycle and can serialize/deserialize, delay a signal, or detect patterns. Shift direction must be explicit.
214. FR - Comment utiliseriez-vous shift register dans un design VHDL réel ? Réponse : Pour recevoir des bits série, je décale l'ancien contenu et concatène le nouveau bit au LSB ou MSB selon l'ordre du protocole.	EN - How would you use shift register in a real VHDL design? Answer: To receive serial bits, I shift old contents and concatenate the new bit at the LSB or MSB according to protocol bit order.
215. FR - Comment diagnostiqueriez-vous un problème lié à shift register ? Réponse : Je vérifie la présence d'un edge d'horloge correct, la priorité reset/enable et les valeurs retenues lorsqu'aucune condition n'est vraie. Je m'assure aussi qu'un signal censé être registre n'a qu'un seul driver clocké. Concernant shift register, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving shift register? Answer: I verify the correct clock edge, reset/enable priority, and held values when no condition is true. I also ensure that a signal intended as a register has only one clocked driver. For shift register, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
216. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant shift register ? Réponse : Je garde un style de registre uniforme dans tout le projet. Les décisions de reset et enable doivent être architecturales, pas ajoutées automatiquement à chaque registre. Pour shift register, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding shift register? Answer: I keep a uniform register style across the project. Reset and enable decisions should be architectural, not automatically added to every register. For shift register, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
217. FR - Expliquez registre versus latch. Quelle structure ou idée hardware se cache derrière ? Réponse : Un flip-flop est déclenché par un edge d'horloge, tandis qu'un latch est sensible à un niveau d'enable et transparent pendant une partie du temps. En RTL FPGA, un latch involontaire est souvent indésirable.	EN - Explain register versus latch. What hardware structure or idea is behind it? Answer: A flip-flop is triggered by a clock edge, while a latch is level-sensitive and transparent during part of the time. In FPGA RTL, an unintended latch is often undesirable.

218. FR - Comment utiliseriez-vous registre versus latch dans un design VHDL réel ? Réponse : Si un signal combinatoire conserve une ancienne valeur parce qu'une branche ne l'affecte pas, je traite cela comme un avertissement et décide explicitement si un latch est vraiment voulu.	EN - How would you use register versus latch in a real VHDL design? Answer: If a combinational signal retains an old value because a branch does not assign it, I treat that as a warning and explicitly decide whether a latch is truly intended.
--	---

219. FR - Comment diagnostiqueriez-vous un problème lié à registre versus latch ? Réponse : Je vérifie la présence d'un edge d'horloge correct, la priorité reset/enable et les valeurs retenues lorsqu'aucune condition n'est vraie. Je m'assure aussi qu'un signal censé être registre n'a qu'un seul driver clocké. Concernant registre versus latch, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving register versus latch? Answer: I verify the correct clock edge, reset/enable priority, and held values when no condition is true. I also ensure that a signal intended as a register has only one clocked driver. For register versus latch, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
---	--

220. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant registre versus latch ? Réponse : Je garde un style de registre uniforme dans tout le projet. Les décisions de reset et enable doivent être architecturales, pas ajoutées automatiquement à chaque registre. Pour registre versus latch, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding register versus latch? Answer: I keep a uniform register style across the project. Reset and enable decisions should be architectural, not automatically added to every register. For register versus latch, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
--	---

12. Horloges, resets, enables et détection de fronts / Clocks, Resets, Enables, and Edge Detection

221. FR - Expliquez rising_edge et falling_edge. Quelle structure ou idée hardware se cache derrière ? Réponse : `rising_edge(clk)` et `falling_edge(clk)` sont les formes standard pour détecter les fronts. Mélanger de la logique sur les deux fronts complique souvent le timing et doit être justifié.	EN - Explain rising_edge and falling_edge. What hardware structure or idea is behind it? Answer: `rising_edge(clk)` and `falling_edge(clk)` are standard forms for detecting edges. Mixing logic on both edges often complicates timing and should be justified.
--	---

222. FR - Comment utiliseriez-vous rising_edge et falling_edge dans un design VHDL réel ? Réponse : Je choisis normalement un seul edge pour tout un domaine d'horloge et traite les interfaces DDR comme un cas spécialisé.	EN - How would you use rising_edge and falling_edge in a real VHDL design? Answer: I normally choose one edge for an entire clock domain and treat DDR interfaces as a specialized case.
---	---

223. FR - Comment diagnostiqueriez-vous un problème lié à rising_edge et falling_edge ? Réponse : Je commence par confirmer fréquence, source et domaine de l'horloge, puis le style de reset et sa libération. Les glitches ou clocks générées par logique sont traités comme suspects. Concernant rising_edge et falling_edge, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving rising_edge and falling_edge? Answer: I start by confirming clock frequency, source, and domain, then reset style and release behavior. Glitches or clocks generated by ordinary logic are treated as suspicious. For rising_edge and falling_edge, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
---	---

224. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant rising_edge et falling_edge ? Réponse : Je privilégie une architecture synchrone avec peu de domaines d'horloge, des clock enables et une stratégie de reset cohérente. Les crossings sont traités explicitement. Pour rising_edge et falling_edge, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding rising_edge and falling_edge? Answer: I favor a synchronous architecture with few clock domains, clock enables, and a coherent reset strategy. Crossings are handled explicitly. For rising_edge and falling_edge, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	---

225. FR - Expliquez reset synchrone. Quelle structure ou idée hardware se cache derrière ? Réponse : Un reset synchrone est testé à l'intérieur du `rising_edge`; il ne change les registres qu'au front d'horloge. Il nécessite donc une horloge active pour prendre effet.	EN - Explain synchronous reset. What hardware structure or idea is behind it? Answer: A synchronous reset is tested inside the `rising_edge`; it changes registers only on a clock edge. It therefore requires an active clock to take effect.
---	---

226. FR - Comment utiliseriez-vous reset synchrone dans un design VHDL réel ? Réponse : Je lui donne une priorité claire sur l'enable et les mises à jour normales lorsqu'il fait partie de la spécification du bloc.	EN - How would you use synchronous reset in a real VHDL design? Answer: I give it clear priority over enable and normal updates when it is part of the block specification.
--	--

227. FR - Comment diagnostiqueriez-vous un problème lié à reset synchrone ? Réponse : Je commence par confirmer fréquence, source et domaine de l'horloge, puis le style de reset et sa libération. Les glitches ou clocks générées par logique sont traités comme suspects. Concernant reset synchrone, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synchronous reset? Answer: I start by confirming clock frequency, source, and domain, then reset style and release behavior. Glitches or clocks generated by ordinary logic are treated as suspicious. For synchronous reset, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
228. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant reset synchrone ? Réponse : Je privilégie une architecture synchrone avec peu de domaines d'horloge, des clock enables et une stratégie de reset cohérente. Les crossings sont traités explicitement. Pour reset synchrone, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synchronous reset? Answer: I favor a synchronous architecture with few clock domains, clock enables, and a coherent reset strategy. Crossings are handled explicitly. For synchronous reset, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
229. FR - Expliquez reset asynchrone et libération synchronisée. Quelle structure ou idée hardware se cache derrière ? Réponse : Un reset asynchrone peut forcer un registre sans attendre l'horloge. Sa désassertion près d'un edge peut toutefois créer des problèmes de recovery/removal ou de cohérence, donc la libération est souvent synchronisée par domaine.	EN - Explain asynchronous reset and synchronized release. What hardware structure or idea is behind it? Answer: An asynchronous reset can force a register without waiting for the clock. Its deassertion near an edge can create recovery/removal or coherence problems, so release is often synchronized per domain.
230. FR - Comment utiliseriez-vous reset asynchrone et libération synchronisée dans un design VHDL réel ? Réponse : Un pattern fréquent est asynchronous assert, synchronous deassert: reset agit immédiatement mais passe par une petite chaîne de registres avant de libérer la logique.	EN - How would you use asynchronous reset and synchronized release in a real VHDL design? Answer: A common pattern is asynchronous assert, synchronous deassert: reset acts immediately but passes through a short register chain before releasing logic.
231. FR - Comment diagnostiqueriez-vous un problème lié à reset asynchrone et libération synchronisée ? Réponse : Je commence par confirmer fréquence, source et domaine de l'horloge, puis le style de reset et sa libération. Les glitches ou clocks générées par logique sont traités comme suspects. Concernant reset asynchrone et libération synchronisée, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving asynchronous reset and synchronized release? Answer: I start by confirming clock frequency, source, and domain, then reset style and release behavior. Glitches or clocks generated by ordinary logic are treated as suspicious. For asynchronous reset and synchronized release, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
232. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant reset asynchrone et libération synchronisée ? Réponse : Je privilégie une architecture synchrone avec peu de domaines d'horloge, des clock enables et une stratégie de reset cohérente. Les crossings sont traités explicitement. Pour reset asynchrone et libération synchronisée, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding asynchronous reset and synchronized release? Answer: I favor a synchronous architecture with few clock domains, clock enables, and a coherent reset strategy. Crossings are handled explicitly. For asynchronous reset and synchronized release, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
233. FR - Expliquez clock enable versus gated clock. Quelle structure ou idée hardware se cache derrière ? Réponse : Un clock enable maintient l'arbre d'horloge propre et autorise conditionnellement la mise à jour des registres. Gater une clock avec une porte logique ordinaire peut générer glitches, skew et problèmes de timing.	EN - Explain clock enable versus gated clock. What hardware structure or idea is behind it? Answer: A clock enable keeps the clock tree clean and conditionally allows register updates. Gating a clock with ordinary logic can generate glitches, skew, and timing problems.
234. FR - Comment utiliseriez-vous clock enable versus gated clock dans un design VHDL réel ? Réponse : Pour exécuter une fonction toutes les 1000 clocks, je génère typiquement une pulse d'enable d'un cycle plutôt qu'une nouvelle clock divisée dans la logique générale.	EN - How would you use clock enable versus gated clock in a real VHDL design? Answer: To execute a function every 1000 clocks, I typically generate a one-cycle enable pulse rather than a new divided clock in ordinary logic.

235. FR - Comment diagnostiqueriez-vous un problème lié à clock enable versus gated clock ? Réponse : Je commence par confirmer fréquence, source et domaine de l'horloge, puis le style de reset et sa libération. Les glitches ou clocks générées par logique sont traités comme suspects. Concernant clock enable versus gated clock, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock enable versus gated clock? Answer: I start by confirming clock frequency, source, and domain, then reset style and release behavior. Glitches or clocks generated by ordinary logic are treated as suspicious. For clock enable versus gated clock, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
236. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock enable versus gated clock ? Réponse : Je privilégie une architecture synchrone avec peu de domaines d'horloge, des clock enables et une stratégie de reset cohérente. Les crossings sont traités explicitement. Pour clock enable versus gated clock, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock enable versus gated clock? Answer: I favor a synchronous architecture with few clock domains, clock enables, and a coherent reset strategy. Crossings are handled explicitly. For clock enable versus gated clock, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
237. FR - Expliquez détection de front d'un signal synchronisé. Quelle structure ou idée hardware se cache derrière ? Réponse : Après synchronisation, un front peut être détecté en comparant la valeur courante à une copie retardée d'un cycle. Pour un front montant, `pulse <= sync and not sync_d;` produit une pulse d'un cycle.	EN - Explain edge detection of a synchronized signal. What hardware structure or idea is behind it? Answer: After synchronization, an edge can be detected by comparing the current value with a one-cycle-delayed copy. For a rising edge, `pulse <= sync and not sync_d;` produces a one-cycle pulse.
238. FR - Comment utiliseriez-vous détection de front d'un signal synchronisé dans un design VHDL réel ? Réponse : Pour une entrée asynchrone, je fais cette détection seulement après le synchroniseur afin que la logique de détection ne voie pas directement la métastabilité. <pre>process(clk) begin if rising_edge(clk) then sync_d <= sync; end if; end process; pulse <= sync and not sync_d;</pre>	EN - How would you use edge detection of a synchronized signal in a real VHDL design? Answer: For an asynchronous input, I perform this detection only after the synchronizer so the edge detector does not directly see metastability. <pre>process(clk) begin if rising_edge(clk) then sync_d <= sync; end if; end process; pulse <= sync and not sync_d;</pre>
239. FR - Comment diagnostiqueriez-vous un problème lié à détection de front d'un signal synchronisé ? Réponse : Je commence par confirmer fréquence, source et domaine de l'horloge, puis le style de reset et sa libération. Les glitches ou clocks générées par logique sont traités comme suspects. Concernant détection de front d'un signal synchronisé, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving edge detection of a synchronized signal? Answer: I start by confirming clock frequency, source, and domain, then reset style and release behavior. Glitches or clocks generated by ordinary logic are treated as suspicious. For edge detection of a synchronized signal, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
240. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant détection de front d'un signal synchronisé ? Réponse : Je privilégie une architecture synchrone avec peu de domaines d'horloge, des clock enables et une stratégie de reset cohérente. Les crossings sont traités explicitement. Pour détection de front d'un signal synchronisé, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding edge detection of a synchronized signal? Answer: I favor a synchronous architecture with few clock domains, clock enables, and a coherent reset strategy. Crossings are handled explicitly. For edge detection of a synchronized signal, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

13. Compteurs, timers, diviseurs et PWM / Counters, Timers, Dividers, and PWM

241. FR - Expliquez compteur up unsigned. Quelle structure ou idée hardware se cache derrière ? Réponse : Un compteur simple est un registre `unsigned` qui s'incrémente à chaque enable ou clock. Sa largeur fixe la période de wrap modulo 2^N si aucune limite explicite n'est appliquée.	EN - Explain unsigned up-counter. What hardware structure or idea is behind it? Answer: A simple counter is an `unsigned` register that increments on each enable or clock. Its width defines modulo-2^N wrap if no explicit limit is applied.
242. FR - Comment utiliseriez-vous compteur up unsigned dans un design VHDL réel ? Réponse : Je réinitialise à zéro et utilise `count <= count + 1;` sous enable; je teste aussi wrap et valeur maximale en simulation.	EN - How would you use unsigned up-counter in a real VHDL design? Answer: I reset it to zero and use `count <= count + 1;` under enable; I also test wraparound and maximum value in simulation.

243. FR - Comment diagnostiqueriez-vous un problème lié à compteur up unsigned ? Réponse : Je vérifie largeur du compteur, valeur terminale, off-by-one et fréquence réelle obtenue. Une erreur de comparaison d'un cycle est très fréquente dans les timers et baud generators. Concernant compteur up unsigned, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving unsigned up-counter? Answer: I check counter width, terminal value, off-by-one behavior, and actual resulting frequency. A one-cycle comparison error is very common in timers and baud generators. For unsigned up-counter, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
244. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant compteur up unsigned ? Réponse : Je calcule les rapports à partir de generics/constantes et documente le nombre exact de cycles par période ou événement. Les clock enables sont préférés aux clocks logiques dérivées. Pour compteur up unsigned, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding unsigned up-counter? Answer: I calculate ratios from generics/constants and document the exact number of cycles per period or event. Clock enables are preferred over logic-derived clocks. For unsigned up-counter, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
245. FR - Expliquez terminal count et pulse périodique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un timer à N cycles compte souvent de 0 à N-1 puis génère une pulse et revient à zéro. Comparer à N au lieu de N-1 produit une période de N+1 cycles.	EN - Explain terminal count and periodic pulse. What hardware structure or idea is behind it? Answer: An N-cycle timer often counts from 0 to N-1, generates a pulse, then returns to zero. Comparing against N instead of N-1 produces an N+1-cycle period.
246. FR - Comment utiliseriez-vous terminal count et pulse périodique dans un design VHDL réel ? Réponse : Pour 50 MHz et une pulse chaque milliseconde, je calcule 50 000 cycles et valide dans la waveform la distance exacte entre deux pulses.	EN - How would you use terminal count and periodic pulse in a real VHDL design? Answer: For 50 MHz and one pulse every millisecond, I calculate 50,000 cycles and verify the exact waveform distance between pulses.
247. FR - Comment diagnostiqueriez-vous un problème lié à terminal count et pulse périodique ? Réponse : Je vérifie largeur du compteur, valeur terminale, off-by-one et fréquence réelle obtenue. Une erreur de comparaison d'un cycle est très fréquente dans les timers et baud generators. Concernant terminal count et pulse périodique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving terminal count and periodic pulse? Answer: I check counter width, terminal value, off-by-one behavior, and actual resulting frequency. A one-cycle comparison error is very common in timers and baud generators. For terminal count and periodic pulse, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
248. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant terminal count et pulse périodique ? Réponse : Je calcule les rapports à partir de generics/constantes et documente le nombre exact de cycles par période ou événement. Les clock enables sont préférés aux clocks logiques dérivées. Pour terminal count et pulse périodique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding terminal count and periodic pulse? Answer: I calculate ratios from generics/constants and document the exact number of cycles per period or event. Clock enables are preferred over logic-derived clocks. For terminal count and periodic pulse, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
249. FR - Expliquez clock divider conceptuel. Quelle structure ou idée hardware se cache derrière ? Réponse : Le MSB d'un compteur ou une logique toggle peut produire une fréquence divisée, mais cette sortie devient une nouvelle clock avec ses propres contraintes et risques de domaine. Pour la logique interne, un enable périodique est souvent plus sûr.	EN - Explain conceptual clock divider. What hardware structure or idea is behind it? Answer: The MSB of a counter or toggle logic can produce a divided frequency, but that output becomes a new clock with its own constraints and domain risks. For internal logic, a periodic enable is often safer.
250. FR - Comment utiliseriez-vous clock divider conceptuel dans un design VHDL réel ? Réponse : Si une vraie clock divisée est nécessaire, je la traite comme une clock et utilise les ressources/contraintes adaptées à la cible.	EN - How would you use conceptual clock divider in a real VHDL design? Answer: If a real divided clock is required, I treat it as a clock and use target-appropriate clocking resources and constraints.
251. FR - Comment diagnostiqueriez-vous un problème lié à clock divider conceptuel ? Réponse : Je vérifie largeur du compteur, valeur terminale, off-by-one et fréquence réelle obtenue. Une erreur de comparaison d'un cycle est très fréquente dans les timers et baud generators. Concernant clock divider conceptuel, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving conceptual clock divider? Answer: I check counter width, terminal value, off-by-one behavior, and actual resulting frequency. A one-cycle comparison error is very common in timers and baud generators. For conceptual clock divider, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

252. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock divider conceptuel ? Réponse : Je calcule les rapports à partir de generics/constantes et documente le nombre exact de cycles par période ou événement. Les clock enables sont préférés aux clocks logiques dérivées. Pour clock divider conceptuel, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding conceptual clock divider? Answer: I calculate ratios from generics/constants and document the exact number of cycles per period or event. Clock enables are preferred over logic-derived clocks. For conceptual clock divider, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
253. FR - Expliquez générateur PWM. Quelle structure ou idée hardware se cache derrière ? Réponse : Une PWM numérique compare typiquement un compteur de phase à une valeur de duty. La résolution dépend du nombre de pas par période et la fréquence du clock/period count.	EN - Explain PWM generator. What hardware structure or idea is behind it? Answer: A digital PWM typically compares a phase counter with a duty value. Resolution depends on steps per period, and frequency on clock/period count.
254. FR - Comment utiliseriez-vous générateur PWM dans un design VHDL réel ? Réponse : Je définis clairement 0% et 100%, car `<` versus `<=` peut créer une impulsion parasite aux extrêmes si les limites sont mal gérées.	EN - How would you use PWM generator in a real VHDL design? Answer: I clearly define 0% and 100%, because `<` versus `<=` can create an unwanted pulse at the extremes if limits are mishandled.
255. FR - Comment diagnostiqueriez-vous un problème lié à générateur PWM ? Réponse : Je vérifie largeur du compteur, valeur terminale, off-by-one et fréquence réelle obtenue. Une erreur de comparaison d'un cycle est très fréquente dans les timers et baud generators. Concernant générateur PWM, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving PWM generator? Answer: I check counter width, terminal value, off-by-one behavior, and actual resulting frequency. A one-cycle comparison error is very common in timers and baud generators. For PWM generator, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
256. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant générateur PWM ? Réponse : Je calcule les rapports à partir de generics/constantes et documente le nombre exact de cycles par période ou événement. Les clock enables sont préférés aux clocks logiques dérivées. Pour générateur PWM, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding PWM generator? Answer: I calculate ratios from generics/constants and document the exact number of cycles per period or event. Clock enables are preferred over logic-derived clocks. For PWM generator, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
257. FR - Expliquez timer watchdog logique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un watchdog RTL peut compter les cycles depuis le dernier événement attendu et lever un timeout si la limite est atteinte. Il surveille efficacement un protocole interne.	EN - Explain logic watchdog timer. What hardware structure or idea is behind it? Answer: An RTL watchdog can count cycles since the last expected event and raise a timeout when the limit is reached. It effectively monitors an internal protocol.
258. FR - Comment utiliseriez-vous timer watchdog logique dans un design VHDL réel ? Réponse : Je reset le compteur lors d'un événement valide et génère un flag de timeout latched ou pulse selon le besoin, avec test explicite de la frontière N-1/N.	EN - How would you use logic watchdog timer in a real VHDL design? Answer: I reset the counter on a valid event and generate either a latched timeout flag or pulse as required, with explicit N-1/N boundary testing.
259. FR - Comment diagnostiqueriez-vous un problème lié à timer watchdog logique ? Réponse : Je vérifie largeur du compteur, valeur terminale, off-by-one et fréquence réelle obtenue. Une erreur de comparaison d'un cycle est très fréquente dans les timers et baud generators. Concernant timer watchdog logique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving logic watchdog timer? Answer: I check counter width, terminal value, off-by-one behavior, and actual resulting frequency. A one-cycle comparison error is very common in timers and baud generators. For logic watchdog timer, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
260. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant timer watchdog logique ? Réponse : Je calcule les rapports à partir de generics/constantes et documente le nombre exact de cycles par période ou événement. Les clock enables sont préférés aux clocks logiques dérivées. Pour timer watchdog logique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding logic watchdog timer? Answer: I calculate ratios from generics/constants and document the exact number of cycles per period or event. Clock enables are preferred over logic-derived clocks. For logic watchdog timer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

14. Machines d'état FSM / Finite State Machines

261. FR - Expliquez structure state/next_state. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FSM classique contient un registre `state` clocké et une logique combinatoire qui calcule `next_state`. Transition calculée pendant le cycle, état mis à jour au prochain edge.	EN - Explain state/next_state structure. What hardware structure or idea is behind it? Answer: A classic FSM contains a clocked `state` register and combinational logic computing `next_state`. The transition is computed during the cycle and state updates on the next edge.
262. FR - Comment utiliseriez-vous structure state/next_state dans un design VHDL réel ? Réponse : Je mets `next_state <= state` par défaut, puis change la valeur uniquement lorsque la condition de transition est vraie.	EN - How would you use state/next_state structure in a real VHDL design? Answer: I assign `next_state <= state` by default, then change it only when a transition condition is true.
263. FR - Comment diagnostiqueriez-vous un problème lié à structure state/next_state ? Réponse : Je vérifie l'état de reset, les conditions de transition, les valeurs par défaut et la couverture de tous les états. Je regarde aussi si les sorties dépendent seulement de l'état ou également des entrées. Concernant structure state/next_state, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving state/next_state structure? Answer: I verify reset state, transition conditions, default values, and coverage of every state. I also check whether outputs depend only on state or also on inputs. For state/next_state structure, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
264. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant structure state/next_state ? Réponse : Je nomme les états avec un type énuméré, sépare clairement registre d'état et logique combinatoire, et assure un chemin de récupération depuis les situations inattendues. Pour structure state/next_state, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding state/next_state structure? Answer: I name states with an enumerated type, clearly separate the state register from combinational logic, and ensure a recovery path from unexpected situations. For state/next_state structure, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
265. FR - Expliquez Moore versus Mealy. Quelle structure ou idée hardware se cache derrière ? Réponse : Dans une Moore, les sorties dépendent de l'état; dans une Mealy, elles peuvent dépendre de l'état et des entrées combinatoires. Une Mealy peut réagir plus vite mais demande plus d'attention aux glitches et chemins combinatoires.	EN - Explain Moore versus Mealy. What hardware structure or idea is behind it? Answer: In a Moore machine, outputs depend on state; in a Mealy machine, they may depend on state and combinational inputs. Mealy can respond sooner but needs more attention to glitches and combinational paths.
266. FR - Comment utiliseriez-vous Moore versus Mealy dans un design VHDL réel ? Réponse : Pour une sortie devant rester stable tout le cycle, je privilégie souvent une sortie enregistrée ou Moore; pour un handshake, une Mealy peut être utile si la spécification le permet.	EN - How would you use Moore versus Mealy in a real VHDL design? Answer: For an output that must remain stable for the full cycle, I often favor a registered or Moore output; for a handshake, Mealy behavior can be useful if allowed by the specification.
267. FR - Comment diagnostiqueriez-vous un problème lié à Moore versus Mealy ? Réponse : Je vérifie l'état de reset, les conditions de transition, les valeurs par défaut et la couverture de tous les états. Je regarde aussi si les sorties dépendent seulement de l'état ou également des entrées. Concernant Moore versus Mealy, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving Moore versus Mealy? Answer: I verify reset state, transition conditions, default values, and coverage of every state. I also check whether outputs depend only on state or also on inputs. For Moore versus Mealy, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
268. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant Moore versus Mealy ? Réponse : Je nomme les états avec un type énuméré, sépare clairement registre d'état et logique combinatoire, et assure un chemin de récupération depuis les situations inattendues. Pour Moore versus Mealy, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding Moore versus Mealy? Answer: I name states with an enumerated type, clearly separate the state register from combinational logic, and ensure a recovery path from unexpected situations. For Moore versus Mealy, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
269. FR - Expliquez default assignments dans next-state logic. Quelle structure ou idée hardware se cache derrière ? Réponse : Sans valeur par défaut, une sortie combinatoire non affectée dans une branche peut inférer un latch. Définir d'abord `next_state <= state` et des sorties par défaut évite ce piège.	EN - Explain default assignments in next-state logic. What hardware structure or idea is behind it? Answer: Without a default value, a combinational output not assigned in a branch can infer a latch. Assigning `next_state <= state` and default outputs first avoids this trap.

270. FR - Comment utiliseriez-vous default assignments dans next-state logic dans un design VHDL réel ? Réponse : Je relis chaque `case` comme une table: pour chaque état, chaque sortie combinatoire et le prochain état doivent avoir une valeur déterminée.	EN - How would you use default assignments in next-state logic in a real VHDL design? Answer: I review each `case` like a table: for every state, each combinational output and next state must have a determined value.
271. FR - Comment diagnostiqueriez-vous un problème lié à default assignments dans next-state logic ? Réponse : Je vérifie l'état de reset, les conditions de transition, les valeurs par défaut et la couverture de tous les états. Je regarde aussi si les sorties dépendent seulement de l'état ou également des entrées. Concernant default assignments dans next-state logic, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving default assignments in next-state logic? Answer: I verify reset state, transition conditions, default values, and coverage of every state. I also check whether outputs depend only on state or also on inputs. For default assignments in next-state logic, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
272. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant default assignments dans next-state logic ? Réponse : Je nomme les états avec un type énuméré, sépare clairement registre d'état et logique combinatoire, et assure un chemin de récupération depuis les situations inattendues. Pour default assignments dans next-state logic, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding default assignments in next-state logic? Answer: I name states with an enumerated type, clearly separate the state register from combinational logic, and ensure a recovery path from unexpected situations. For default assignments in next-state logic, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
273. FR - Expliquez états invalides et récupération. Quelle structure ou idée hardware se cache derrière ? Réponse : Avec un type énuméré, les états légaux sont explicites, mais le hardware peut encore subir reset incomplet, upset ou corruption. Une stratégie de récupération peut ramener la FSM vers IDLE/ERROR selon la criticité.	EN - Explain invalid states and recovery. What hardware structure or idea is behind it? Answer: With an enumerated type, legal states are explicit, but hardware can still experience incomplete reset, upset, or corruption. A recovery strategy can return the FSM to IDLE/ERROR depending on criticality.
274. FR - Comment utiliseriez-vous états invalides et récupération dans un design VHDL réel ? Réponse : Je prévois un chemin error/recovery cohérent et vérifie les rapports de synthèse et exigences de sûreté plutôt que de supposer qu'un état illégal est impossible.	EN - How would you use invalid states and recovery in a real VHDL design? Answer: I provide a coherent error/recovery path and verify synthesis behavior and safety requirements rather than assuming an illegal state is impossible.
275. FR - Comment diagnostiqueriez-vous un problème lié à états invalides et récupération ? Réponse : Je vérifie l'état de reset, les conditions de transition, les valeurs par défaut et la couverture de tous les états. Je regarde aussi si les sorties dépendent seulement de l'état ou également des entrées. Concernant états invalides et récupération, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving invalid states and recovery? Answer: I verify reset state, transition conditions, default values, and coverage of every state. I also check whether outputs depend only on state or also on inputs. For invalid states and recovery, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
276. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant états invalides et récupération ? Réponse : Je nomme les états avec un type énuméré, sépare clairement registre d'état et logique combinatoire, et assure un chemin de récupération depuis les situations inattendues. Pour états invalides et récupération, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding invalid states and recovery? Answer: I name states with an enumerated type, clearly separate the state register from combinational logic, and ensure a recovery path from unexpected situations. For invalid states and recovery, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
277. FR - Expliquez FSM UART TX comme exemple. Quelle structure ou idée hardware se cache derrière ? Réponse : Une UART TX se prête bien à une FSM: IDLE, START, DATA, STOP. Un compteur de baud cadence les transitions et un index de bit choisit la donnée série.	EN - Explain UART TX FSM as an example. What hardware structure or idea is behind it? Answer: A UART TX maps naturally to an FSM: IDLE, START, DATA, STOP. A baud counter times transitions and a bit index selects serial data.
278. FR - Comment utiliseriez-vous FSM UART TX comme exemple dans un design VHDL réel ? Réponse : Je sépare timing baud, registre de données et FSM; cela simplifie la simulation du start bit, huit bits LSB-first et stop bit.	EN - How would you use UART TX FSM as an example in a real VHDL design? Answer: I separate baud timing, the data register, and the FSM; this simplifies simulation of start bit, eight LSB-first data bits, and stop bit.

279. FR - Comment diagnostiqueriez-vous un problème lié à FSM UART TX comme exemple ? Réponse : Je vérifie l'état de reset, les conditions de transition, les valeurs par défaut et la couverture de tous les états. Je regarde aussi si les sorties dépendent seulement de l'état ou également des entrées. Concernant FSM UART TX comme exemple, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART TX FSM as an example? Answer: I verify reset state, transition conditions, default values, and coverage of every state. I also check whether outputs depend only on state or also on inputs. For UART TX FSM as an example, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
280. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FSM UART TX comme exemple ? Réponse : Je nomme les états avec un type énuméré, sépare clairement registre d'état et logique combinatoire, et assure un chemin de récupération depuis les situations inattendues. Pour FSM UART TX comme exemple, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART TX FSM as an example? Answer: I name states with an enumerated type, clearly separate the state register from combinational logic, and ensure a recovery path from unexpected situations. For UART TX FSM as an example, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

15. Generics, generate et matériel réutilisable / Generics, Generate, and Reusable Hardware

281. FR - Expliquez generic constant. Quelle structure ou idée hardware se cache derrière ? Réponse : Un generic est un paramètre d'élaboration fourni à une entity, par exemple largeur de bus, fréquence d'horloge ou profondeur. Il permet de réutiliser le même RTL avec plusieurs configurations matérielles.	EN - Explain generic constant. What hardware structure or idea is behind it? Answer: A generic is an elaboration-time parameter supplied to an entity, such as bus width, clock frequency, or depth. It allows the same RTL to be reused with multiple hardware configurations.
282. FR - Comment utiliseriez-vous generic constant dans un design VHDL réel ? Réponse : Je déclare <code>`WIDTH : positive := 8`</code> puis utilise <code>`WIDTH-1 downto 0`</code>, ce qui rend la largeur cohérente dans toute l'entity. <pre>generic (WIDTH : positive := 8); port (data : in std_logic_vector(WIDTH-1 downto 0));</pre>	EN - How would you use generic constant in a real VHDL design? Answer: I declare <code>`WIDTH : positive := 8`</code> and use <code>`WIDTH-1 downto 0`</code>, keeping width consistent throughout the entity. <pre>generic (WIDTH : positive := 8); port (data : in std_logic_vector(WIDTH-1 downto 0));</pre>
283. FR - Comment diagnostiqueriez-vous un problème lié à generic constant ? Réponse : Je vérifie que les generics ont des plages valides et que les expressions dérivées ne créent ni division par zéro ni largeur négative. Pour <code>`generate`</code>, je confirme aussi le nombre réel d'instances produit. Concernant generic constant, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving generic constant? Answer: I verify that generics have valid ranges and derived expressions create neither divide-by-zero nor negative widths. For <code>`generate`</code>, I also confirm the actual number of instances produced. For generic constant, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
284. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant generic constant ? Réponse : Je paramètre ce qui change réellement entre instances, mais j'évite une abstraction tellement générale qu'elle cache le hardware. Les defaults doivent être sûrs et réalistes. Pour generic constant, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding generic constant? Answer: I parameterize what genuinely varies between instances but avoid abstractions so general that they hide the hardware. Defaults should be safe and realistic. For generic constant, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
285. FR - Expliquez generic map. Quelle structure ou idée hardware se cache derrière ? Réponse : <code>`generic map`</code> fournit des valeurs spécifiques lors de l'instanciation. Avec l'association nommée, le code reste lisible même si l'ordre des generics change.	EN - Explain generic map. What hardware structure or idea is behind it? Answer: <code>`generic map`</code> supplies instance-specific values during instantiation. Named association keeps code readable even if generic order changes.
286. FR - Comment utiliseriez-vous generic map dans un design VHDL réel ? Réponse : J'écris <code>`generic map (WIDTH => 16, DEPTH => 256)`</code> plutôt qu'une liste positionnelle lorsque plusieurs paramètres sont présents.	EN - How would you use generic map in a real VHDL design? Answer: I write <code>`generic map (WIDTH => 16, DEPTH => 256)`</code> rather than a positional list when several parameters are present.

287. FR - Comment diagnostiqueriez-vous un problème lié à generic map ? Réponse : Je vérifie que les generics ont des plages valides et que les expressions dérivées ne créent ni division par zéro ni largeur négative. Pour `generate`, je confirme aussi le nombre réel d'instances produit. Concernant generic map, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving generic map? Answer: I verify that generics have valid ranges and derived expressions create neither divide-by-zero nor negative widths. For `generate`, I also confirm the actual number of instances produced. For generic map, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
288. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant generic map ? Réponse : Je paramètre ce qui change réellement entre instances, mais j'évite une abstraction tellement générale qu'elle cache le hardware. Les defaults doivent être sûrs et réalistes. Pour generic map, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding generic map? Answer: I parameterize what genuinely varies between instances but avoid abstractions so general that they hide the hardware. Defaults should be safe and realistic. For generic map, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
289. FR - Expliquez for generate. Quelle structure ou idée hardware se cache derrière ? Réponse : `for generate` répète une structure matérielle un nombre déterminé de fois à l'élaboration. Chaque itération crée une logique/instance distincte; elle ne s'exécute pas séquentiellement dans le temps.	EN - Explain for generate. What hardware structure or idea is behind it? Answer: `for generate` repeats a hardware structure a determined number of times at elaboration. Each iteration creates distinct logic/instances; it does not execute sequentially in time.
290. FR - Comment utiliseriez-vous for generate dans un design VHDL réel ? Réponse : Je peux instancier huit canaux identiques avec `for i in 0 to 7 generate`, en indexant les slices de bus pour chaque canal.	EN - How would you use for generate in a real VHDL design? Answer: I can instantiate eight identical channels with `for i in 0 to 7 generate`, indexing bus slices for each channel.
291. FR - Comment diagnostiqueriez-vous un problème lié à for generate ? Réponse : Je vérifie que les generics ont des plages valides et que les expressions dérivées ne créent ni division par zéro ni largeur négative. Pour `generate`, je confirme aussi le nombre réel d'instances produit. Concernant for generate, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving for generate? Answer: I verify that generics have valid ranges and derived expressions create neither divide-by-zero nor negative widths. For `generate`, I also confirm the actual number of instances produced. For for generate, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
292. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant for generate ? Réponse : Je paramètre ce qui change réellement entre instances, mais j'évite une abstraction tellement générale qu'elle cache le hardware. Les defaults doivent être sûrs et réalistes. Pour for generate, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding for generate? Answer: I parameterize what genuinely varies between instances but avoid abstractions so general that they hide the hardware. Defaults should be safe and realistic. For for generate, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
293. FR - Expliquez if generate. Quelle structure ou idée hardware se cache derrière ? Réponse : `if generate` inclut ou exclut du hardware selon une condition statique d'élaboration, par exemple activer une option CRC ou choisir une implémentation selon un generic.	EN - Explain if generate. What hardware structure or idea is behind it? Answer: `if generate` includes or excludes hardware based on a static elaboration condition, for example enabling a CRC option or selecting an implementation based on a generic.
294. FR - Comment utiliseriez-vous if generate dans un design VHDL réel ? Réponse : Je l'utilise pour des variantes de produit sans laisser une condition runtime permanente dans le datapath lorsque la fonction est fixe par configuration.	EN - How would you use if generate in a real VHDL design? Answer: I use it for product variants so a runtime condition is not permanently present in the datapath when the feature is fixed by configuration.
295. FR - Comment diagnostiqueriez-vous un problème lié à if generate ? Réponse : Je vérifie que les generics ont des plages valides et que les expressions dérivées ne créent ni division par zéro ni largeur négative. Pour `generate`, je confirme aussi le nombre réel d'instances produit. Concernant if generate, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving if generate? Answer: I verify that generics have valid ranges and derived expressions create neither divide-by-zero nor negative widths. For `generate`, I also confirm the actual number of instances produced. For if generate, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

296. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant if generate ? Réponse : Je paramètre ce qui change réellement entre instances, mais j'évite une abstraction tellement générale qu'elle cache le hardware. Les defaults doivent être sûrs et réalistes. Pour if generate, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding if generate? Answer: I parameterize what genuinely varies between instances but avoid abstractions so general that they hide the hardware. Defaults should be safe and realistic. For if generate, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
297. FR - Expliquez réutilisabilité et limites des generics. Quelle structure ou idée hardware se cache derrière ? Réponse : Trop de generics peuvent rendre un bloc difficile à comprendre et tester. Les paramètres doivent représenter de vraies dimensions ou choix structuraux et leurs combinaisons valides doivent être définies.	EN - Explain reusability and limits of generics. What hardware structure or idea is behind it? Answer: Too many generics can make a block difficult to understand and test. Parameters should represent real dimensions or structural choices and valid combinations must be defined.
298. FR - Comment utiliseriez-vous réutilisabilité et limites des generics dans un design VHDL réel ? Réponse : J'ajoute des assertions d'élaboration/simulation pour rejeter une profondeur nulle ou une fréquence trop basse pour produire le baud rate demandé.	EN - How would you use reusability and limits of generics in a real VHDL design? Answer: I add elaboration/simulation assertions to reject zero depth or a clock frequency too low to generate the requested baud rate.
299. FR - Comment diagnostiqueriez-vous un problème lié à réutilisabilité et limites des generics ? Réponse : Je vérifie que les generics ont des plages valides et que les expressions dérivées ne créent ni division par zéro ni largeur négative. Pour `generate`, je confirme aussi le nombre réel d'instances produit. Concernant réutilisabilité et limites des generics, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving reusability and limits of generics? Answer: I verify that generics have valid ranges and derived expressions create neither divide-by-zero nor negative widths. For `generate`, I also confirm the actual number of instances produced. For reusability and limits of generics, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
300. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant réutilisabilité et limites des generics ? Réponse : Je paramètre ce qui change réellement entre instances, mais j'évite une abstraction tellement générale qu'elle cache le hardware. Les defaults doivent être sûrs et réalistes. Pour réutilisabilité et limites des generics, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding reusability and limits of generics? Answer: I parameterize what genuinely varies between instances but avoid abstractions so general that they hide the hardware. Defaults should be safe and realistic. For reusability and limits of generics, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

16. Hiérarchie avancée, packages, fonctions et procédures / Advanced Hierarchy, Packages, Functions, and Procedures

301. FR - Expliquez package de types et constantes. Quelle structure ou idée hardware se cache derrière ? Réponse : Un package central peut définir types de bus, records, enums, constantes et prototypes de fonctions utilisés par plusieurs design units. Cela évite de dupliquer des définitions incompatibles.	EN - Explain package of types and constants. What hardware structure or idea is behind it? Answer: A central package can define bus types, records, enums, constants, and function prototypes used by several design units. This avoids duplicating incompatible definitions.
302. FR - Comment utiliseriez-vous package de types et constantes dans un design VHDL réel ? Réponse : Pour un protocole interne, je peux définir `command_t` et un record de données, puis importer le package dans le parser et la FSM de contrôle.	EN - How would you use package of types and constants in a real VHDL design? Answer: For an internal protocol, I can define `command_t` and a data record, then import the package into the parser and control FSM.
303. FR - Comment diagnostiqueriez-vous un problème lié à package de types et constantes ? Réponse : Je vérifie portée des déclarations, pureté des fonctions, types visibles et interfaces de sous-programmes. Une erreur de surcharge ou de package se lit souvent directement dans les types attendu/obtenu du compilateur. Concernant package de types et constantes, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving package of types and constants? Answer: I check declaration scope, function purity, visible types, and subprogram interfaces. Overload or package errors are often diagnosed directly from the compiler's expected/actual types. For package of types and constants, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

304. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant package de types et constantes ? Réponse : Je mets dans les packages seulement ce qui est réellement partagé et garde les fonctions petites, déterministes et testables. Les interfaces de blocs restent plus importantes que la sophistication du package. Pour package de types et constantes, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding package of types and constants? Answer: I put in packages only what is genuinely shared and keep functions small, deterministic, and testable. Block interfaces matter more than package sophistication. For package of types and constants, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
305. FR - Expliquez package body. Quelle structure ou idée hardware se cache derrière ? Réponse : Un package body contient l'implémentation des sous-programmes déclarés dans le package lorsque cette implémentation n'est pas directement dans la spécification. Il masque les détails tout en gardant l'interface partagée stable.	EN - Explain package body. What hardware structure or idea is behind it? Answer: A package body contains implementations of subprograms declared in the package when those implementations are not directly in the specification. It hides details while keeping the shared interface stable.
306. FR - Comment utiliseriez-vous package body dans un design VHDL réel ? Réponse : Je peux exposer <code>function crc8(...) return ...;</code> dans le package et placer l'algorithme dans le body, ce qui simplifie les utilisateurs du package.	EN - How would you use package body in a real VHDL design? Answer: I can expose <code>function crc8(...) return ...;</code> in the package and place the algorithm in the body, simplifying package users.
307. FR - Comment diagnostiqueriez-vous un problème lié à package body ? Réponse : Je vérifie portée des déclarations, pureté des fonctions, types visibles et interfaces de sous-programmes. Une erreur de surcharge ou de package se lit souvent directement dans les types attendu/obtenu du compilateur. Concernant package body, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving package body? Answer: I check declaration scope, function purity, visible types, and subprogram interfaces. Overload or package errors are often diagnosed directly from the compiler's expected/actual types. For package body, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
308. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant package body ? Réponse : Je mets dans les packages seulement ce qui est réellement partagé et garde les fonctions petites, déterministes et testables. Les interfaces de blocs restent plus importantes que la sophistication du package. Pour package body, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding package body? Answer: I put in packages only what is genuinely shared and keep functions small, deterministic, and testable. Block interfaces matter more than package sophistication. For package body, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
309. FR - Expliquez fonctions pures. Quelle structure ou idée hardware se cache derrière ? Réponse : Une fonction pure dépend uniquement de ses paramètres et ne produit pas d'effets de bord externes. Cela la rend idéale pour des calculs combinatoires réutilisables et faciles à tester.	EN - Explain pure functions. What hardware structure or idea is behind it? Answer: A pure function depends only on its parameters and produces no external side effects. This makes it ideal for reusable combinational calculations that are easy to test.
310. FR - Comment utiliseriez-vous fonctions pures dans un design VHDL réel ? Réponse : J'utilise une fonction pour parité, conversion, réduction ou CRC combinatoire lorsque cela améliore la lisibilité sans cacher une quantité excessive de logique.	EN - How would you use pure functions in a real VHDL design? Answer: I use a function for parity, conversion, reduction, or combinational CRC when it improves readability without hiding excessive logic.
311. FR - Comment diagnostiqueriez-vous un problème lié à fonctions pures ? Réponse : Je vérifie portée des déclarations, pureté des fonctions, types visibles et interfaces de sous-programmes. Une erreur de surcharge ou de package se lit souvent directement dans les types attendu/obtenu du compilateur. Concernant fonctions pures, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving pure functions? Answer: I check declaration scope, function purity, visible types, and subprogram interfaces. Overload or package errors are often diagnosed directly from the compiler's expected/actual types. For pure functions, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

312. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant fonctions pures ? Réponse : Je mets dans les packages seulement ce qui est réellement partagé et garde les fonctions petites, déterministes et testables. Les interfaces de blocs restent plus importantes que la sophistication du package. Pour fonctions pures, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding pure functions? Answer: I put in packages only what is genuinely shared and keep functions small, deterministic, and testable. Block interfaces matter more than package sophistication. For pure functions, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
313. FR - Expliquez procedures. Quelle structure ou idée hardware se cache derrière ? Réponse : Une procédure peut avoir plusieurs paramètres <code>`in`</code>, <code>`out`</code> ou <code>`inout`</code> et regrouper une séquence de statements. Elle est utile en RTL ou surtout en testbench, mais il faut comprendre sa sémantique de synthèse.	EN - Explain procedures. What hardware structure or idea is behind it? Answer: A procedure can have multiple <code>`in`</code>, <code>`out`</code>, or <code>`inout`</code> parameters and group a sequence of statements. It can be useful in RTL or especially testbenches, but its synthesis semantics must be understood.
314. FR - Comment utiliseriez-vous procedures dans un design VHDL réel ? Réponse : Dans un testbench, une procédure <code>`uart_send_byte`</code> peut générer proprement start/data/stop avec des <code>`wait`</code>, ce qui évite de répéter le stimulus.	EN - How would you use procedures in a real VHDL design? Answer: In a testbench, a <code>`uart_send_byte`</code> procedure can cleanly generate start/data/stop timing using <code>`wait`</code>, avoiding repeated stimulus.
315. FR - Comment diagnostiqueriez-vous un problème lié à procedures ? Réponse : Je vérifie portée des déclarations, pureté des fonctions, types visibles et interfaces de sous-programmes. Une erreur de surcharge ou de package se lit souvent directement dans les types attendu/obtenu du compilateur. Concernant procedures, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving procedures? Answer: I check declaration scope, function purity, visible types, and subprogram interfaces. Overload or package errors are often diagnosed directly from the compiler's expected/actual types. For procedures, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
316. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant procedures ? Réponse : Je mets dans les packages seulement ce qui est réellement partagé et garde les fonctions petites, déterministes et testables. Les interfaces de blocs restent plus importantes que la sophistication du package. Pour procedures, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding procedures? Answer: I put in packages only what is genuinely shared and keep functions small, deterministic, and testable. Block interfaces matter more than package sophistication. For procedures, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
317. FR - Expliquez records pour interfaces structurées. Quelle structure ou idée hardware se cache derrière ? Réponse : Un record regroupe des champs de types différents sous un seul objet VHDL. Il peut rendre une interface interne très lisible, mais le support direct au top-level dépend des outils/flows.	EN - Explain records for structured interfaces. What hardware structure or idea is behind it? Answer: A record groups fields of different types into one VHDL object. It can make internal interfaces very readable, although direct top-level support depends on tools/flows.
318. FR - Comment utiliseriez-vous records pour interfaces structurées dans un design VHDL réel ? Réponse : Je peux regrouper <code>`data`</code>, <code>`valid`</code>, <code>`last`</code> dans un record interne tout en gardant des ports simples au top si la portabilité externe est prioritaire.	EN - How would you use records for structured interfaces in a real VHDL design? Answer: I can group <code>`data`</code>, <code>`valid`</code>, and <code>`last`</code> in an internal record while keeping simple top-level ports if external portability is the priority.
319. FR - Comment diagnostiqueriez-vous un problème lié à records pour interfaces structurées ? Réponse : Je vérifie portée des déclarations, pureté des fonctions, types visibles et interfaces de sous-programmes. Une erreur de surcharge ou de package se lit souvent directement dans les types attendu/obtenu du compilateur. Concernant records pour interfaces structurées, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving records for structured interfaces? Answer: I check declaration scope, function purity, visible types, and subprogram interfaces. Overload or package errors are often diagnosed directly from the compiler's expected/actual types. For records for structured interfaces, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

320. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant records pour interfaces structurées ? Réponse : Je mets dans les packages seulement ce qui est réellement partagé et garde les fonctions petites, déterministes et testables. Les interfaces de blocs restent plus importantes que la sophistication du package. Pour records pour interfaces structurées, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding records for structured interfaces? Answer: I put in packages only what is genuinely shared and keep functions small, deterministic, and testable. Block interfaces matter more than package sophistication. For records for structured interfaces, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	--

17. Arithmétique, numeric_std, conversions et overflow / Arithmetic, numeric_std, Conversions, and Overflow

321. FR - Expliquez numeric_std versus packages non standard. Quelle structure ou idée hardware se cache derrière ? Réponse : `ieee.numeric_std` définit l'arithmétique standard pour `unsigned` et `signed`. Les anciens packages comme `std_logic_unsigned` ont existé dans certains outils mais créent ambiguïtés et problèmes de portabilité.	EN - Explain numeric_std versus non-standard arithmetic packages. What hardware structure or idea is behind it? Answer: `ieee.numeric_std` defines standard arithmetic for `unsigned` and `signed`. Older packages such as `std_logic_unsigned` existed in some tools but create ambiguity and portability issues.
322. FR - Comment utiliseriez-vous numeric_std versus packages non standard dans un design VHDL réel ? Réponse : Dans un nouveau projet, je garde `std_logic_vector` pour les bits/interfaces et convertis explicitement vers `unsigned` ou `signed` pour l'arithmétique.	EN - How would you use numeric_std versus non-standard arithmetic packages in a real VHDL design? Answer: In a new project, I keep `std_logic_vector` for bits/interfaces and explicitly convert to `unsigned` or `signed` for arithmetic.
323. FR - Comment diagnostiqueriez-vous un problème lié à numeric_std versus packages non standard ? Réponse : Je vérifie type numérique, largeur avant/après chaque opération et conversions. Les bugs de signe et de troncature sont prioritaires, surtout aux valeurs négatives ou proches du maximum. Concernant numeric_std versus packages non standard, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving numeric_std versus non-standard arithmetic packages? Answer: I check numeric type, width before/after each operation, and conversions. Sign and truncation bugs are high priority, especially for negative values or values near maximum. For numeric_std versus non-standard arithmetic packages, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
324. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant numeric_std versus packages non standard ? Réponse : Je standardise sur `numeric_std`, dimensionne explicitement les résultats et teste les valeurs limites. Un design numérique doit être bit-accurate, pas seulement correct avec quelques valeurs nominales. Pour numeric_std versus packages non standard, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding numeric_std versus non-standard arithmetic packages? Answer: I standardize on `numeric_std`, explicitly size results, and test boundary values. A numeric design must be bit-accurate, not merely correct for a few nominal values. For numeric_std versus non-standard arithmetic packages, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
325. FR - Expliquez conversions std_logic_vector/unsigned/signed. Quelle structure ou idée hardware se cache derrière ? Réponse : Les conversions `unsigned(v)`, `signed(v)` et `std_logic_vector(n)` changent l'interprétation de type sans forcément changer les bits. Le designer doit savoir si les bits représentent réellement une valeur signée ou non signée.	EN - Explain std_logic_vector/unsigned/signed conversions. What hardware structure or idea is behind it? Answer: Conversions `unsigned(v)`, `signed(v)`, and `std_logic_vector(n)` change type interpretation without necessarily changing the bits. The designer must know whether the bits really represent a signed or unsigned value.
326. FR - Comment utiliseriez-vous conversions std_logic_vector/unsigned/signed dans un design VHDL réel ? Réponse : Pour un bus ADC non signé, j'écris `adc_value <= unsigned(adc_bus);`; je n'utilise `signed` que si le protocole définit le MSB comme bit de signe en complément à deux.	EN - How would you use std_logic_vector/unsigned/signed conversions in a real VHDL design? Answer: For an unsigned ADC bus, I write `adc_value <= unsigned(adc_bus);`; I use `signed` only if the protocol defines the MSB as a two's-complement sign bit.
327. FR - Comment diagnostiqueriez-vous un problème lié à conversions std_logic_vector/unsigned/signed ? Réponse : Je vérifie type numérique, largeur avant/après chaque opération et conversions. Les bugs de signe et de troncature sont prioritaires, surtout aux valeurs négatives ou proches du maximum. Concernant conversions std_logic_vector/unsigned/signed, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving std_logic_vector/unsigned/signed conversions? Answer: I check numeric type, width before/after each operation, and conversions. Sign and truncation bugs are high priority, especially for negative values or values near maximum. For std_logic_vector/unsigned/signed conversions, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

328. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant conversions std_logic_vector/unsigned/signed ? Réponse : Je standardise sur `numeric_std`, dimensionne explicitement les résultats et teste les valeurs limites. Un design numérique doit être bit-accurate, pas seulement correct avec quelques valeurs nominales. Pour conversions std_logic_vector/unsigned/signed, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding std_logic_vector/unsigned/signed conversions? Answer: I standardize on `numeric_std`, explicitly size results, and test boundary values. A numeric design must be bit-accurate, not merely correct for a few nominal values. For std_logic_vector/unsigned/signed conversions, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
329. FR - Expliquez to_unsigned, to_signed et to_integer. Quelle structure ou idée hardware se cache derrière ? Réponse : `to_unsigned(value,size)` et `to_signed(value,size)` créent des vecteurs numériques d'une largeur donnée; `to_integer` convertit un `unsigned` ou `signed` vers integer lorsque la plage le permet.	EN - Explain to_unsigned, to_signed, and to_integer. What hardware structure or idea is behind it? Answer: `to_unsigned(value,size)` and `to_signed(value,size)` create numeric vectors of a given width; `to_integer` converts an `unsigned` or `signed` to integer when range permits.
330. FR - Comment utiliseriez-vous to_unsigned, to_signed et to_integer dans un design VHDL réel ? Réponse : Je fournis la largeur via `length` quand elle dépend du signal, par exemple `to_unsigned(LIMIT, counter'length)`.	EN - How would you use to_unsigned, to_signed, and to_integer in a real VHDL design? Answer: I provide width using `length` when it depends on the signal, for example `to_unsigned(LIMIT, counter'length)`.
331. FR - Comment diagnostiqueriez-vous un problème lié à to_unsigned, to_signed et to_integer ? Réponse : Je vérifie type numérique, largeur avant/après chaque opération et conversions. Les bugs de signe et de troncature sont prioritaires, surtout aux valeurs négatives ou proches du maximum. Concernant to_unsigned, to_signed et to_integer, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving to_unsigned, to_signed, and to_integer? Answer: I check numeric type, width before/after each operation, and conversions. Sign and truncation bugs are high priority, especially for negative values or values near maximum. For to_unsigned, to_signed, and to_integer, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
332. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant to_unsigned, to_signed et to_integer ? Réponse : Je standardise sur `numeric_std`, dimensionne explicitement les résultats et teste les valeurs limites. Un design numérique doit être bit-accurate, pas seulement correct avec quelques valeurs nominales. Pour to_unsigned, to_signed et to_integer, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding to_unsigned, to_signed, and to_integer? Answer: I standardize on `numeric_std`, explicitly size results, and test boundary values. A numeric design must be bit-accurate, not merely correct for a few nominal values. For to_unsigned, to_signed, and to_integer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
333. FR - Expliquez resize et extension de signe. Quelle structure ou idée hardware se cache derrière ? Réponse : `resize` adapte la largeur d'un `signed` ou `unsigned`; pour `signed`, l'extension conserve le signe. Réduire la largeur peut tronquer les MSB et changer la valeur.	EN - Explain resize and sign extension. What hardware structure or idea is behind it? Answer: `resize` adapts the width of a `signed` or `unsigned`; for `signed`, extension preserves sign. Reducing width can truncate MSBs and change the value.
334. FR - Comment utiliseriez-vous resize et extension de signe dans un design VHDL réel ? Réponse : Avant d'accumuler une valeur signée 12 bits dans 20 bits, je fais une extension signée explicite vers la largeur de l'accumulateur.	EN - How would you use resize and sign extension in a real VHDL design? Answer: Before accumulating a signed 12-bit value into 20 bits, I explicitly sign-extend it to the accumulator width.
335. FR - Comment diagnostiqueriez-vous un problème lié à resize et extension de signe ? Réponse : Je vérifie type numérique, largeur avant/après chaque opération et conversions. Les bugs de signe et de troncature sont prioritaires, surtout aux valeurs négatives ou proches du maximum. Concernant resize et extension de signe, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving resize and sign extension? Answer: I check numeric type, width before/after each operation, and conversions. Sign and truncation bugs are high priority, especially for negative values or values near maximum. For resize and sign extension, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
336. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant resize et extension de signe ? Réponse : Je standardise sur `numeric_std`, dimensionne explicitement les résultats et teste les valeurs limites. Un design numérique doit être bit-accurate, pas seulement correct avec quelques valeurs nominales. Pour resize et extension de signe, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding resize and sign extension? Answer: I standardize on `numeric_std`, explicitly size results, and test boundary values. A numeric design must be bit-accurate, not merely correct for a few nominal values. For resize and sign extension, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

337. FR - Expliquez overflow, saturation et wraparound. Quelle structure ou idée hardware se cache derrière ? Réponse : L'arithmétique vectorielle fixe produit naturellement du wraparound si le résultat dépasse la largeur et qu'aucune logique de saturation n'est ajoutée. La saturation est une décision d'architecture.	EN - Explain overflow, saturation, and wraparound. What hardware structure or idea is behind it? Answer: Fixed-width vector arithmetic naturally wraps around when the result exceeds width unless saturation logic is added. Saturation is an architectural decision.
338. FR - Comment utiliseriez-vous overflow, saturation et wraparound dans un design VHDL réel ? Réponse : Pour un contrôle ou traitement de signal, je décide explicitement entre wrap, saturation et flag overflow, puis teste maximum+1, minimum-1 et combinaisons de signe.	EN - How would you use overflow, saturation, and wraparound in a real VHDL design? Answer: For control or signal processing, I explicitly choose among wrap, saturation, and overflow flag behavior, then test max+1, min-1, and sign combinations.
339. FR - Comment diagnostiqueriez-vous un problème lié à overflow, saturation et wraparound ? Réponse : Je vérifie type numérique, largeur avant/après chaque opération et conversions. Les bugs de signe et de troncature sont prioritaires, surtout aux valeurs négatives ou proches du maximum. Concernant overflow, saturation et wraparound, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving overflow, saturation, and wraparound? Answer: I check numeric type, width before/after each operation, and conversions. Sign and truncation bugs are high priority, especially for negative values or values near maximum. For overflow, saturation, and wraparound, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
340. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant overflow, saturation et wraparound ? Réponse : Je standardise sur <code>`numeric_std`</code> , dimensionne explicitement les résultats et teste les valeurs limites. Un design numérique doit être bit-accurate, pas seulement correct avec quelques valeurs nominales. Pour overflow, saturation et wraparound, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding overflow, saturation, and wraparound? Answer: I standardize on <code>`numeric_std`</code> , explicitly size results, and test boundary values. A numeric design must be bit-accurate, not merely correct for a few nominal values. For overflow, saturation, and wraparound, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

18. ROM, RAM, tableaux et FIFO / ROM, RAM, Arrays, and FIFOs

341. FR - Expliquez array type pour mémoire. Quelle structure ou idée hardware se cache derrière ? Réponse : Une mémoire RTL est souvent décrite comme un array de mots, par exemple <code>`array(0 to DEPTH-1) of std_logic_vector(WIDTH-1 downto 0)`</code> . L'adresse sélectionne un élément du tableau.	EN - Explain array type for memory. What hardware structure or idea is behind it? Answer: An RTL memory is often described as an array of words, for example <code>`array(0 to DEPTH-1) of std_logic_vector(WIDTH-1 downto 0)`</code> . The address selects one array element.
342. FR - Comment utiliseriez-vous array type pour mémoire dans un design VHDL réel ? Réponse : Je convertis une adresse <code>`unsigned`</code> avec <code>`to_integer(addr)`</code> seulement après avoir garanti qu'elle reste dans la plage de profondeur.	EN - How would you use array type for memory in a real VHDL design? Answer: I convert an <code>`unsigned`</code> address using <code>`to_integer(addr)`</code> only after ensuring it remains within memory depth.
343. FR - Comment diagnostiqueriez-vous un problème lié à array type pour mémoire ? Réponse : Je vérifie profondeur, largeur, indices et surtout comportement de lecture/écriture attendu. Pour une mémoire inférée, le style RTL doit correspondre à ce que l'outil peut mapper sur les ressources de la cible. Concernant array type pour mémoire, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving array type for memory? Answer: I verify depth, width, indices, and especially expected read/write behavior. For inferred memory, RTL style must match what the tool can map onto target resources. For array type for memory, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
344. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant array type pour mémoire ? Réponse : Je garde la description mémoire simple et portable, puis je confirme dans le rapport de synthèse qu'elle a été inférée comme RAM/ROM plutôt qu'en registres dispersés lorsque cela compte. Pour array type pour mémoire, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding array type for memory? Answer: I keep memory descriptions simple and portable, then confirm in synthesis reports that the structure was inferred as RAM/ROM rather than distributed registers when that matters. For array type for memory, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

345. FR - Expliquez ROM constante. Quelle structure ou idée hardware se cache derrière ? Réponse : Une ROM peut être décrite par une constante array initialisée avec des mots fixes. La synthèse peut mapper cette structure vers LUTs ou block memory selon taille et cible.	EN - Explain constant ROM. What hardware structure or idea is behind it? Answer: A ROM can be described by a constant array initialized with fixed words. Synthesis may map this structure to LUTs or block memory depending on size and target.
346. FR - Comment utiliseriez-vous ROM constante dans un design VHDL réel ? Réponse : Pour une table de calibration fixe, je définis le type et une constante `ROM`, puis lis `ROM(address)` avec une adresse contrôlée.	EN - How would you use constant ROM in a real VHDL design? Answer: For a fixed calibration table, I define the type and a constant `ROM`, then read `ROM(address)` using a controlled address.
347. FR - Comment diagnostiqueriez-vous un problème lié à ROM constante ? Réponse : Je vérifie profondeur, largeur, indices et surtout comportement de lecture/écriture attendu. Pour une mémoire inférée, le style RTL doit correspondre à ce que l'outil peut mapper sur les ressources de la cible. Concernant ROM constante, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving constant ROM? Answer: I verify depth, width, indices, and especially expected read/write behavior. For inferred memory, RTL style must match what the tool can map onto target resources. For constant ROM, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
348. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant ROM constante ? Réponse : Je garde la description mémoire simple et portable, puis je confirme dans le rapport de synthèse qu'elle a été inférée comme RAM/ROM plutôt qu'en registres dispersés lorsque cela compte. Pour ROM constante, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding constant ROM? Answer: I keep memory descriptions simple and portable, then confirm in synthesis reports that the structure was inferred as RAM/ROM rather than distributed registers when that matters. For constant ROM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
349. FR - Expliquez RAM synchrone simple-port. Quelle structure ou idée hardware se cache derrière ? Réponse : Une RAM synchrone met généralement l'écriture et souvent la lecture sous contrôle d'horloge. Le comportement read-during-write peut être write-first, read-first ou no-change selon style et ressource.	EN - Explain single-port synchronous RAM. What hardware structure or idea is behind it? Answer: A synchronous RAM generally places writes and often reads under clock control. Read-during-write behavior may be write-first, read-first, or no-change depending on style and resource.
350. FR - Comment utiliseriez-vous RAM synchrone simple-port dans un design VHDL réel ? Réponse : Je définis explicitement ce que doit voir la sortie si lecture et écriture ciblent la même adresse et vérifie que l'inférence de la cible respecte cette sémantique.	EN - How would you use single-port synchronous RAM in a real VHDL design? Answer: I explicitly define what the output should see when read and write target the same address and verify that target inference preserves that semantic.
351. FR - Comment diagnostiqueriez-vous un problème lié à RAM synchrone simple-port ? Réponse : Je vérifie profondeur, largeur, indices et surtout comportement de lecture/écriture attendu. Pour une mémoire inférée, le style RTL doit correspondre à ce que l'outil peut mapper sur les ressources de la cible. Concernant RAM synchrone simple-port, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving single-port synchronous RAM? Answer: I verify depth, width, indices, and especially expected read/write behavior. For inferred memory, RTL style must match what the tool can map onto target resources. For single-port synchronous RAM, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
352. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant RAM synchrone simple-port ? Réponse : Je garde la description mémoire simple et portable, puis je confirme dans le rapport de synthèse qu'elle a été inférée comme RAM/ROM plutôt qu'en registres dispersés lorsque cela compte. Pour RAM synchrone simple-port, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding single-port synchronous RAM? Answer: I keep memory descriptions simple and portable, then confirm in synthesis reports that the structure was inferred as RAM/ROM rather than distributed registers when that matters. For single-port synchronous RAM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
353. FR - Expliquez dual-port RAM conceptuel. Quelle structure ou idée hardware se cache derrière ? Réponse : Une dual-port RAM permet deux accès avec ports/adresses distincts, parfois sur des clocks différentes selon la primitive cible. Les collisions sur la même adresse nécessitent une sémantique connue.	EN - Explain conceptual dual-port RAM. What hardware structure or idea is behind it? Answer: A dual-port RAM allows two accesses with separate ports/adresses, sometimes on different clocks depending on the target primitive. Collisions on the same address require known semantics.

354. FR - Comment utiliseriez-vous dual-port RAM conceptuel dans un design VHDL réel ? Réponse : Je l'utilise pour partager un buffer entre deux clients; pour un CDC asynchrone complet, je préfère une architecture FIFO conçue pour cela.	EN - How would you use conceptual dual-port RAM in a real VHDL design? Answer: I use it to share a buffer between two clients; for a full asynchronous CDC, I prefer a FIFO architecture designed for that purpose.
355. FR - Comment diagnostiqueriez-vous un problème lié à dual-port RAM conceptuel ? Réponse : Je vérifie profondeur, largeur, indices et surtout comportement de lecture/écriture attendu. Pour une mémoire inférée, le style RTL doit correspondre à ce que l'outil peut mapper sur les ressources de la cible. Concernant dual-port RAM conceptuel, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving conceptual dual-port RAM? Answer: I verify depth, width, indices, and especially expected read/write behavior. For inferred memory, RTL style must match what the tool can map onto target resources. For conceptual dual-port RAM, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
356. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant dual-port RAM conceptuel ? Réponse : Je garde la description mémoire simple et portable, puis je confirme dans le rapport de synthèse qu'elle a été inférée comme RAM/ROM plutôt qu'en registres dispersés lorsque cela compte. Pour dual-port RAM conceptuel, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding conceptual dual-port RAM? Answer: I keep memory descriptions simple and portable, then confirm in synthesis reports that the structure was inferred as RAM/ROM rather than distributed registers when that matters. For conceptual dual-port RAM, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
357. FR - Expliquez FIFO: pointers, full/empty et count. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FIFO conserve l'ordre des données avec pointeurs de lecture/écriture et flags `full/empty`. Une FIFO synchrone utilise une seule clock; une FIFO asynchrone traite aussi le crossing de domaines.	EN - Explain FIFO: pointers, full/empty, and count. What hardware structure or idea is behind it? Answer: A FIFO preserves data order using read/write pointers and `full/empty` flags. A synchronous FIFO uses one clock; an asynchronous FIFO also handles clock-domain crossing.
358. FR - Comment utiliseriez-vous FIFO: pointers, full/empty et count dans un design VHDL réel ? Réponse : Je teste reset, premier write/read, wrap, full, empty, write-when-full et read-when-empty. Pour l'asynchrone, j'utilise une architecture éprouvée.	EN - How would you use FIFO: pointers, full/empty, and count in a real VHDL design? Answer: I test reset, first write/read, wraparound, full, empty, write-when-full, and read-when-empty. For asynchronous FIFO, I use a proven architecture.
359. FR - Comment diagnostiqueriez-vous un problème lié à FIFO: pointers, full/empty et count ? Réponse : Je vérifie profondeur, largeur, indices et surtout comportement de lecture/écriture attendu. Pour une mémoire inférée, le style RTL doit correspondre à ce que l'outil peut mapper sur les ressources de la cible. Concernant FIFO: pointers, full/empty et count, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving FIFO: pointers, full/empty, and count? Answer: I verify depth, width, indices, and especially expected read/write behavior. For inferred memory, RTL style must match what the tool can map onto target resources. For FIFO: pointers, full/empty, and count, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
360. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant FIFO: pointers, full/empty et count ? Réponse : Je garde la description mémoire simple et portable, puis je confirme dans le rapport de synthèse qu'elle a été inférée comme RAM/ROM plutôt qu'en registres dispersés lorsque cela compte. Pour FIFO: pointers, full/empty et count, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding FIFO: pointers, full/empty, and count? Answer: I keep memory descriptions simple and portable, then confirm in synthesis reports that the structure was inferred as RAM/ROM rather than distributed registers when that matters. For FIFO: pointers, full/empty, and count, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

19. CDC, métastabilité et synchroniseurs / CDC, Metastability, and Synchronizers

361. FR - Expliquez métastabilité. Quelle structure ou idée hardware se cache derrière ? Réponse : Lorsqu'un flip-flop échantillonne un signal changeant près de son edge, setup/hold peut être violé et la sortie devenir temporairement métastable. On ne peut pas éliminer la probabilité, mais on peut la réduire.	EN - Explain metastability. What hardware structure or idea is behind it? Answer: When a flip-flop samples a signal changing near its edge, setup/hold can be violated and the output can become temporarily metastable. The probability cannot be eliminated, but it can be reduced.
--	---

362. FR - Comment utiliseriez-vous métastabilité dans un design VHDL réel ? Réponse : Je ne distribue jamais la sortie du premier flop synchroniseur à la logique fonctionnelle; j'utilise l'étage suivant après un temps de résolution supplémentaire.	EN - How would you use metastability in a real VHDL design? Answer: I never distribute the first synchronizer flop output to functional logic; I use the following stage after additional resolution time.
363. FR - Comment diagnostiqueriez-vous un problème lié à métastabilité ? Réponse : Je vérifie immédiatement si le signal traverse un domaine d'horloge. Je distingue bit de contrôle, pulse, bus multi-bit et flux de données, car chacun exige une technique CDC différente. Concernant métastabilité, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving metastability? Answer: I immediately check whether the signal crosses a clock domain. I distinguish a control bit, pulse, multi-bit bus, and data stream because each needs a different CDC technique. For metastability, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
364. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant métastabilité ? Réponse : Je traite chaque crossing explicitement et documente clocks source/destination. Un simple deux-flop est pour un bit de contrôle stable, pas une solution universelle pour n'importe quel bus. Pour métastabilité, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding metastability? Answer: I handle every crossing explicitly and document source/destination clocks. A simple two-flop synchronizer is for a stable control bit, not a universal solution for any bus. For metastability, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
365. FR - Expliquez two-flop synchronizer. Quelle structure ou idée hardware se cache derrière ? Réponse : Deux flip-flops en série dans le domaine destination sont une technique classique pour un signal asynchrone mono-bit stable assez longtemps. Le premier peut métastabiliser; le second réduit fortement le risque propagé.	EN - Explain two-flop synchronizer. What hardware structure or idea is behind it? Answer: Two flip-flops in series in the destination domain are a classic technique for a single-bit asynchronous signal stable long enough. The first may metastabilize; the second greatly reduces propagated risk.
366. FR - Comment utiliseriez-vous two-flop synchronizer dans un design VHDL réel ? Réponse : Je garde les deux étages sans logique combinatoire entre eux et structure la chaîne pour que l'outil d'implémentation la reconnaisse correctement.	EN - How would you use two-flop synchronizer in a real VHDL design? Answer: I keep the two stages with no combinational logic between them and structure the chain so implementation can recognize it correctly.
367. FR - Comment diagnostiqueriez-vous un problème lié à two-flop synchronizer ? Réponse : Je vérifie immédiatement si le signal traverse un domaine d'horloge. Je distingue bit de contrôle, pulse, bus multi-bit et flux de données, car chacun exige une technique CDC différente. Concernant two-flop synchronizer, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving two-flop synchronizer? Answer: I immediately check whether the signal crosses a clock domain. I distinguish a control bit, pulse, multi-bit bus, and data stream because each needs a different CDC technique. For two-flop synchronizer, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
368. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant two-flop synchronizer ? Réponse : Je traite chaque crossing explicitement et documente clocks source/destination. Un simple deux-flop est pour un bit de contrôle stable, pas une solution universelle pour n'importe quel bus. Pour two-flop synchronizer, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding two-flop synchronizer? Answer: I handle every crossing explicitly and document source/destination clocks. A simple two-flop synchronizer is for a stable control bit, not a universal solution for any bus. For two-flop synchronizer, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
369. FR - Expliquez crossing d'une pulse. Quelle structure ou idée hardware se cache derrière ? Réponse : Une pulse d'un cycle source peut être plus courte qu'un cycle destination et être manquée par un simple synchroniseur. Toggle, pulse-stretch ou handshake convertissent l'événement en information capturable.	EN - Explain crossing a pulse. What hardware structure or idea is behind it? Answer: A one-source-cycle pulse may be shorter than one destination cycle and be missed by a simple synchronizer. Toggle, pulse-stretch, or handshake techniques convert the event into capturable information.
370. FR - Comment utiliseriez-vous crossing d'une pulse dans un design VHDL réel ? Réponse : Pour des événements rares, je peux toggler un bit à chaque événement, synchroniser ce bit puis détecter son changement dans le domaine destination.	EN - How would you use crossing a pulse in a real VHDL design? Answer: For rare events, I can toggle a bit on each event, synchronize that bit, then detect its change in the destination domain.

371. FR - Comment diagnostiqueriez-vous un problème lié à crossing d'une pulse ? Réponse : Je vérifie immédiatement si le signal traverse un domaine d'horloge. Je distingue bit de contrôle, pulse, bus multi-bit et flux de données, car chacun exige une technique CDC différente. Concernant crossing d'une pulse, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving crossing a pulse? Answer: I immediately check whether the signal crosses a clock domain. I distinguish a control bit, pulse, multi-bit bus, and data stream because each needs a different CDC technique. For crossing a pulse, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
372. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant crossing d'une pulse ? Réponse : Je traite chaque crossing explicitement et documente clocks source/destination. Un simple deux-flop est pour un bit de contrôle stable, pas une solution universelle pour n'importe quel bus. Pour crossing d'une pulse, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding crossing a pulse? Answer: I handle every crossing explicitly and document source/destination clocks. A simple two-flop synchronizer is for a stable control bit, not a universal solution for any bus. For crossing a pulse, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
373. FR - Expliquez crossing d'un bus multi-bit. Quelle structure ou idée hardware se cache derrière ? Réponse : Synchroniser chaque bit d'un bus séparément ne garantit pas une valeur cohérente, car les bits peuvent être capturés sur des cycles différents. Il faut valid/handshake, Gray code ou FIFO selon le cas.	EN - Explain crossing a multi-bit bus. What hardware structure or idea is behind it? Answer: Synchronizing each bit of a bus independently does not guarantee a coherent value because bits can be captured on different cycles. Valid/handshake, Gray code, or FIFO is needed depending on the case.
374. FR - Comment utiliseriez-vous crossing d'un bus multi-bit dans un design VHDL réel ? Réponse : Pour une configuration lente, je fige le bus source, synchronise un strobe/handshake et capture le bus seulement quand sa stabilité est garantie.	EN - How would you use crossing a multi-bit bus in a real VHDL design? Answer: For slow configuration data, I hold the source bus stable, synchronize a strobe/handshake, and capture the bus only when stability is guaranteed.
375. FR - Comment diagnostiqueriez-vous un problème lié à crossing d'un bus multi-bit ? Réponse : Je vérifie immédiatement si le signal traverse un domaine d'horloge. Je distingue bit de contrôle, pulse, bus multi-bit et flux de données, car chacun exige une technique CDC différente. Concernant crossing d'un bus multi-bit, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving crossing a multi-bit bus? Answer: I immediately check whether the signal crosses a clock domain. I distinguish a control bit, pulse, multi-bit bus, and data stream because each needs a different CDC technique. For crossing a multi-bit bus, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
376. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant crossing d'un bus multi-bit ? Réponse : Je traite chaque crossing explicitement et documente clocks source/destination. Un simple deux-flop est pour un bit de contrôle stable, pas une solution universelle pour n'importe quel bus. Pour crossing d'un bus multi-bit, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding crossing a multi-bit bus? Answer: I handle every crossing explicitly and document source/destination clocks. A simple two-flop synchronizer is for a stable control bit, not a universal solution for any bus. For crossing a multi-bit bus, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
377. FR - Expliquez asynchronous FIFO et Gray pointers. Quelle structure ou idée hardware se cache derrière ? Réponse : Une FIFO asynchrone utilise typiquement une mémoire dual-port et des pointeurs lecture/écriture convertis en Gray code afin qu'un seul bit change entre valeurs adjacentes traversant les domaines.	EN - Explain asynchronous FIFO and Gray pointers. What hardware structure or idea is behind it? Answer: An asynchronous FIFO typically uses dual-port memory and read/write pointers converted to Gray code so only one bit changes between adjacent values crossing domains.
378. FR - Comment utiliseriez-vous asynchronous FIFO et Gray pointers dans un design VHDL réel ? Réponse : Je préfère une FIFO vendor-proven ou une architecture CDC largement vérifiée, puis analyse contraintes et flags full/empty plutôt que de réinventer le schéma pour un produit critique.	EN - How would you use asynchronous FIFO and Gray pointers in a real VHDL design? Answer: I prefer a vendor-proven FIFO or widely verified CDC architecture, then analyze constraints and full/empty flags rather than reinventing the scheme for a critical product.
379. FR - Comment diagnostiqueriez-vous un problème lié à asynchronous FIFO et Gray pointers ? Réponse : Je vérifie immédiatement si le signal traverse un domaine d'horloge. Je distingue bit de contrôle, pulse, bus multi-bit et flux de données, car chacun exige une technique CDC différente. Concernant asynchronous FIFO et Gray pointers, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving asynchronous FIFO and Gray pointers? Answer: I immediately check whether the signal crosses a clock domain. I distinguish a control bit, pulse, multi-bit bus, and data stream because each needs a different CDC technique. For asynchronous FIFO and Gray pointers, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

380. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant asynchronous FIFO et Gray pointers ? Réponse : Je traite chaque crossing explicitement et documente clocks source/destination. Un simple deux-flop est pour un bit de contrôle stable, pas une solution universelle pour n'importe quel bus. Pour asynchronous FIFO et Gray pointers, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding asynchronous FIFO and Gray pointers? Answer: I handle every crossing explicitly and document source/destination clocks. A simple two-flop synchronizer is for a stable control bit, not a universal solution for any bus. For asynchronous FIFO and Gray pointers, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
---	---

20. Entrées externes, debounce, tri-state et open-drain / External Inputs, Debounce, Tri-State, and Open-Drain

381. FR - Expliquez debounce d'un bouton. Quelle structure ou idée hardware se cache derrière ? Réponse : Un bouton mécanique peut produire plusieurs transitions rapides lors d'un seul appui. Un debouncer exige que l'entrée synchronisée reste stable pendant un certain nombre de cycles avant d'accepter le nouvel état.	EN - Explain button debounce. What hardware structure or idea is behind it? Answer: A mechanical button can produce multiple rapid transitions during one press. A debouncer requires the synchronized input to remain stable for a number of cycles before accepting the new state.
382. FR - Comment utiliseriez-vous debounce d'un bouton dans un design VHDL réel ? Réponse : Je synchronise d'abord le bouton, puis utilise un compteur de stabilité; après N cycles identiques, je mets à jour l'état debounced et éventuellement génère une pulse de front.	EN - How would you use button debounce in a real VHDL design? Answer: I synchronize the button first, then use a stability counter; after N identical cycles, I update the debounced state and optionally generate an edge pulse.
383. FR - Comment diagnostiqueriez-vous un problème lié à debounce d'un bouton ? Réponse : Je vérifie d'abord la nature électrique et temporelle du signal: asynchrone, bruité, mécanique, bidirectionnel ou open-drain. Le RTL ne peut pas corriger une hypothèse électrique fausse sur la carte. Concernant debounce d'un bouton, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving button debounce? Answer: I first verify the signal's electrical and timing nature: asynchronous, noisy, mechanical, bidirectional, or open-drain. RTL cannot fix an incorrect electrical assumption on the board. For button debounce, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
384. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant debounce d'un bouton ? Réponse : Je sépare adaptation électrique, synchronisation et fonction logique. Les tri-states sont confinés aux I/O lorsque la technologie ne supporte pas de tri-state interne réel. Pour debounce d'un bouton, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding button debounce? Answer: I separate electrical interfacing, synchronization, and logical function. Tri-states are confined to I/O when the technology does not support real internal tri-states. For button debounce, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
385. FR - Expliquez glitch filter numérique. Quelle structure ou idée hardware se cache derrière ? Réponse : Un filtre de glitch rejette les impulsions plus courtes qu'une durée définie. Il ressemble au debounce mais peut utiliser une fenêtre de plusieurs échantillons ou un compteur up/down selon le bruit.	EN - Explain digital glitch filter. What hardware structure or idea is behind it? Answer: A glitch filter rejects pulses shorter than a defined duration. It resembles debounce but may use a multi-sample window or up/down counter depending on the noise.
386. FR - Comment utiliseriez-vous glitch filter numérique dans un design VHDL réel ? Réponse : Je choisis la durée selon le signal réel et la latence admissible, puis teste les impulsions juste en dessous et juste au-dessus du seuil.	EN - How would you use digital glitch filter in a real VHDL design? Answer: I choose duration based on the real signal and acceptable latency, then test pulses just below and just above the threshold.
387. FR - Comment diagnostiqueriez-vous un problème lié à glitch filter numérique ? Réponse : Je vérifie d'abord la nature électrique et temporelle du signal: asynchrone, bruité, mécanique, bidirectionnel ou open-drain. Le RTL ne peut pas corriger une hypothèse électrique fausse sur la carte. Concernant glitch filter numérique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving digital glitch filter? Answer: I first verify the signal's electrical and timing nature: asynchronous, noisy, mechanical, bidirectional, or open-drain. RTL cannot fix an incorrect electrical assumption on the board. For digital glitch filter, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.

388. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant glitch filter numérique ? Réponse : Je sépare adaptation électrique, synchronisation et fonction logique. Les tri-states sont confinés aux I/O lorsque la technologie ne supporte pas de tri-state interne réel. Pour glitch filter numérique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding digital glitch filter? Answer: I separate electrical interfacing, synchronization, and logical function. Tri-states are confined to I/O when the technology does not support real internal tri-states. For digital glitch filter, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
389. FR - Expliquez tri-state avec Z. Quelle structure ou idée hardware se cache derrière ? Réponse : La valeur `Z` représente la haute impédance en simulation. Pour une broche bidirectionnelle, on pilote la donnée quand output-enable est actif et `Z` sinon; dans beaucoup de FPGA, les tri-states réels existent seulement aux I/O.	EN - Explain tri-state using Z. What hardware structure or idea is behind it? Answer: The value `Z` represents high impedance in simulation. For a bidirectional pin, data is driven when output-enable is active and `Z` otherwise; in many FPGAs, real tri-states exist only at I/O.
390. FR - Comment utiliseriez-vous tri-state avec Z dans un design VHDL réel ? Réponse : Au top-level, une forme typique est <code>io_pin <= tx_data when oe='1' else 'Z';</code> puis <code>rx_data <= io_pin;</code> <code>io_pin <= tx_data when oe = '1' else 'Z';</code> <code>rx_data <= io_pin;</code>	EN - How would you use tri-state using Z in a real VHDL design? Answer: At the top level, a typical form is <code>io_pin <= tx_data when oe='1' else 'Z';</code> followed by <code>rx_data <= io_pin;</code> <code>io_pin <= tx_data when oe = '1' else 'Z';</code> <code>rx_data <= io_pin;</code>
391. FR - Comment diagnostiqueriez-vous un problème lié à tri-state avec Z ? Réponse : Je vérifie d'abord la nature électrique et temporelle du signal: asynchrone, bruité, mécanique, bidirectionnel ou open-drain. Le RTL ne peut pas corriger une hypothèse électrique fausse sur la carte. Concernant tri-state avec Z, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving tri-state using Z? Answer: I first verify the signal's electrical and timing nature: asynchronous, noisy, mechanical, bidirectional, or open-drain. RTL cannot fix an incorrect electrical assumption on the board. For tri-state using Z, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
392. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant tri-state avec Z ? Réponse : Je sépare adaptation électrique, synchronisation et fonction logique. Les tri-states sont confinés aux I/O lorsque la technologie ne supporte pas de tri-state interne réel. Pour tri-state avec Z, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding tri-state using Z? Answer: I separate electrical interfacing, synchronization, and logical function. Tri-states are confined to I/O when the technology does not support real internal tri-states. For tri-state using Z, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
393. FR - Expliquez open-drain/open-collector logique. Quelle structure ou idée hardware se cache derrière ? Réponse : Une sortie open-drain ne force normalement que le niveau bas; le niveau haut vient d'un pull-up externe. En VHDL top-level, on pilote `0` ou `Z`, pas `1` activement dans ce modèle.	EN - Explain open-drain/open-collector logic. What hardware structure or idea is behind it? Answer: An open-drain output normally drives only low; the high level comes from an external pull-up. At VHDL top level, it drives `0` or `Z`, not actively `1` in this model.
394. FR - Comment utiliseriez-vous open-drain/open-collector logique dans un design VHDL réel ? Réponse : Pour SDA I2C, <code>sda <= '0' when drive_low='1' else 'Z';</code> représente correctement l'intention logique, sous réserve des contraintes électriques de la carte.	EN - How would you use open-drain/open-collector logic in a real VHDL design? Answer: For I2C SDA, <code>sda <= '0' when drive_low='1' else 'Z';</code> correctly represents the logical intent, subject to board electrical constraints.
395. FR - Comment diagnostiqueriez-vous un problème lié à open-drain/open-collector logique ? Réponse : Je vérifie d'abord la nature électrique et temporelle du signal: asynchrone, bruité, mécanique, bidirectionnel ou open-drain. Le RTL ne peut pas corriger une hypothèse électrique fausse sur la carte. Concernant open-drain/open-collector logique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving open-drain/open-collector logic? Answer: I first verify the signal's electrical and timing nature: asynchronous, noisy, mechanical, bidirectional, or open-drain. RTL cannot fix an incorrect electrical assumption on the board. For open-drain/open-collector logic, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
396. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant open-drain/open-collector logique ? Réponse : Je sépare adaptation électrique, synchronisation et fonction logique. Les tri-states sont confinés aux I/O lorsque la technologie ne supporte pas de tri-state interne réel. Pour open-drain/open-collector logique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding open-drain/open-collector logic? Answer: I separate electrical interfacing, synchronization, and logical function. Tri-states are confined to I/O when the technology does not support real internal tri-states. For open-drain/open-collector logic, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

397. FR - Expliquez entrée asynchrone vers événement interne. Quelle structure ou idée hardware se cache derrière ? Réponse : Une entrée externe ne doit pas directement commander une FSM synchrone si elle est asynchrone au clock. La chaîne habituelle est synchronisation, qualification éventuelle, puis edge/event detection.	EN - Explain asynchronous input to internal event. What hardware structure or idea is behind it? Answer: An external input should not directly control a synchronous FSM if it is asynchronous to the clock. The usual chain is synchronization, optional qualification, then edge/event detection.
398. FR - Comment utiliseriez-vous entrée asynchrone vers événement interne dans un design VHDL réel ? Réponse : Pour une alarme externe, je peux faire synchronizer -> glitch filter -> rising-edge pulse -> status register/IRQ, ce qui sépare clairement les responsabilités.	EN - How would you use asynchronous input to internal event in a real VHDL design? Answer: For an external alarm, I can use synchronizer -> glitch filter -> rising-edge pulse -> status register/IRQ, clearly separating responsibilities.
399. FR - Comment diagnostiqueriez-vous un problème lié à entrée asynchrone vers événement interne ? Réponse : Je vérifie d'abord la nature électrique et temporelle du signal: asynchrone, bruité, mécanique, bidirectionnel ou open-drain. Le RTL ne peut pas corriger une hypothèse électrique fausse sur la carte. Concernant entrée asynchrone vers événement interne, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving asynchronous input to internal event? Answer: I first verify the signal's electrical and timing nature: asynchronous, noisy, mechanical, bidirectional, or open-drain. RTL cannot fix an incorrect electrical assumption on the board. For asynchronous input to internal event, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
400. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant entrée asynchrone vers événement interne ? Réponse : Je sépare adaptation électrique, synchronisation et fonction logique. Les tri-states sont confinés aux I/O lorsque la technologie ne supporte pas de tri-state interne réel. Pour entrée asynchrone vers événement interne, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding asynchronous input to internal event? Answer: I separate electrical interfacing, synchronization, and logical function. Tri-states are confined to I/O when the technology does not support real internal tri-states. For asynchronous input to internal event, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

21. Blocs de protocole UART, SPI et I²C / UART, SPI, and I²C Protocol Blocks

401. FR - Expliquez UART TX. Quelle structure ou idée hardware se cache derrière ? Réponse : UART TX génère une ligne idle haute, un start bit bas, les bits de données généralement LSB-first, puis un ou plusieurs stop bits hauts. Un baud tick ou compteur définit la durée de chaque bit.	EN - Explain UART TX. What hardware structure or idea is behind it? Answer: UART TX generates an idle-high line, a low start bit, data bits usually LSB-first, then one or more high stop bits. A baud tick or counter defines each bit duration.
402. FR - Comment utiliseriez-vous UART TX dans un design VHDL réel ? Réponse : Je capture l'octet à l'acceptation de `tx_start`, le garde dans un shift register, puis la FSM produit START/DATA/STOP et un `tx_busy` ou `tx_done`.	EN - How would you use UART TX in a real VHDL design? Answer: I capture the byte when `tx_start` is accepted, hold it in a shift register, then the FSM produces START/DATA/STOP along with `tx_busy` or `tx_done`.
403. FR - Comment diagnostiqueriez-vous un problème lié à UART TX ? Réponse : Je vérifie timing, polarité, ordre des bits, sampling point et handshakes avec la logique interne. Une capture logic analyzer et une simulation du protocole doivent raconter la même histoire. Concernant UART TX, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART TX? Answer: I verify timing, polarity, bit order, sampling point, and handshakes with internal logic. A logic-analyzer capture and protocol simulation should tell the same story. For UART TX, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
404. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UART TX ? Réponse : Je sépare le moteur série du protocole de haut niveau et j'utilise des interfaces `valid/ready` ou pulses clairement définies vers le reste du design. Pour UART TX, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART TX? Answer: I separate the serial engine from the higher-level protocol and use clearly defined `valid/ready` interfaces or pulses toward the rest of the design. For UART TX, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

405. FR - Expliquez UART RX et sampling. Quelle structure ou idée hardware se cache derrière ? Réponse : UART RX détecte le start bit puis échantillonne chaque bit autour de son centre. Comme RX est asynchrone au clock local, l'entrée doit être synchronisée; un oversampling peut améliorer la robustesse.	EN - Explain UART RX and sampling. What hardware structure or idea is behind it? Answer: UART RX detects the start bit and then samples each bit near its center. Because RX is asynchronous to the local clock, the input must be synchronized; oversampling can improve robustness.
406. FR - Comment utiliseriez-vous UART RX et sampling dans un design VHDL réel ? Réponse : Je valide start, huit données et stop, puis lève `data_valid`; je peux aussi détecter framing error lorsque le stop bit n'est pas haut au moment attendu.	EN - How would you use UART RX and sampling in a real VHDL design? Answer: I validate start, eight data bits, and stop, then assert `data_valid`; I can also detect framing error when the stop bit is not high at the expected time.
407. FR - Comment diagnostiqueriez-vous un problème lié à UART RX et sampling ? Réponse : Je vérifie timing, polarité, ordre des bits, sampling point et handshakes avec la logique interne. Une capture logic analyzer et une simulation du protocole doivent raconter la même histoire. Concernant UART RX et sampling, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving UART RX and sampling? Answer: I verify timing, polarity, bit order, sampling point, and handshakes with internal logic. A logic-analyzer capture and protocol simulation should tell the same story. For UART RX and sampling, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
408. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant UART RX et sampling ? Réponse : Je sépare le moteur série du protocole de haut niveau et j'utilise des interfaces `valid/ready` ou pulses clairement définies vers le reste du design. Pour UART RX et sampling, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding UART RX and sampling? Answer: I separate the serial engine from the higher-level protocol and use clearly defined `valid/ready` interfaces or pulses toward the rest of the design. For UART RX and sampling, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
409. FR - Expliquez SPI master. Quelle structure ou idée hardware se cache derrière ? Réponse : Un SPI master contrôle SCLK et chip-select et échange des bits sur MOSI/MISO. CPOL et CPHA définissent niveau idle et edge d'échantillonnage/changement; un mauvais mode donne souvent des bits décalés.	EN - Explain SPI master. What hardware structure or idea is behind it? Answer: An SPI master controls SCLK and chip-select and exchanges bits on MOSI/MISO. CPOL and CPHA define idle level and sample/change edges; the wrong mode often produces shifted bits.
410. FR - Comment utiliseriez-vous SPI master dans un design VHDL réel ? Réponse : Je définis le mode SPI en paramètres, garde CS stable pendant la transaction et vérifie première/dernière edge avec waveform ou logic analyzer.	EN - How would you use SPI master in a real VHDL design? Answer: I define SPI mode as parameters, keep CS stable during the transaction, and verify first/last edge with a waveform or logic analyzer.
411. FR - Comment diagnostiqueriez-vous un problème lié à SPI master ? Réponse : Je vérifie timing, polarité, ordre des bits, sampling point et handshakes avec la logique interne. Une capture logic analyzer et une simulation du protocole doivent raconter la même histoire. Concernant SPI master, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SPI master? Answer: I verify timing, polarity, bit order, sampling point, and handshakes with internal logic. A logic-analyzer capture and protocol simulation should tell the same story. For SPI master, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
412. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant SPI master ? Réponse : Je sépare le moteur série du protocole de haut niveau et j'utilise des interfaces `valid/ready` ou pulses clairement définies vers le reste du design. Pour SPI master, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SPI master? Answer: I separate the serial engine from the higher-level protocol and use clearly defined `valid/ready` interfaces or pulses toward the rest of the design. For SPI master, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
413. FR - Expliquez SPI slave. Quelle structure ou idée hardware se cache derrière ? Réponse : Un SPI slave reçoit une clock externe et CS, ce qui crée potentiellement un domaine d'horloge différent du système interne. L'architecture doit décider si la logique fonctionne sur SCLK ou utilise un CDC vers la clock système.	EN - Explain SPI slave. What hardware structure or idea is behind it? Answer: An SPI slave receives an external clock and CS, potentially creating a clock domain different from the internal system. The architecture must decide whether logic runs on SCLK or uses CDC into the system clock domain.

414. FR - Comment utiliseriez-vous SPI slave dans un design VHDL réel ? Réponse : Je garde le shift engine près du domaine SPI puis transfère les octets complets vers le domaine système via handshake/FIFO si les clocks sont indépendantes.	EN - How would you use SPI slave in a real VHDL design? Answer: I keep the shift engine close to the SPI domain, then transfer complete bytes into the system domain via handshake/FIFO when clocks are independent.
415. FR - Comment diagnostiqueriez-vous un problème lié à SPI slave ? Réponse : Je vérifie timing, polarité, ordre des bits, sampling point et handshakes avec la logique interne. Une capture logic analyzer et une simulation du protocole doivent raconter la même histoire. Concernant SPI slave, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving SPI slave? Answer: I verify timing, polarity, bit order, sampling point, and handshakes with internal logic. A logic-analyzer capture and protocol simulation should tell the same story. For SPI slave, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
416. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant SPI slave ? Réponse : Je sépare le moteur série du protocole de haut niveau et j'utilise des interfaces `valid/ready` ou pulses clairement définies vers le reste du design. Pour SPI slave, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding SPI slave? Answer: I separate the serial engine from the higher-level protocol and use clearly defined `valid/ready` interfaces or pulses toward the rest of the design. For SPI slave, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
417. FR - Expliquez I²C open-drain, START/STOP et ACK. Quelle structure ou idée hardware se cache derrière ? Réponse : I²C utilise SDA/SCL open-drain. START est SDA haut->bas pendant SCL haut, STOP bas->haut pendant SCL haut; après huit bits, le récepteur fournit ACK/NACK au neuvième clock.	EN - Explain I²C open-drain, START/STOP, and ACK. What hardware structure or idea is behind it? Answer: I²C uses open-drain SDA/SCL. START is SDA high-to-low while SCL is high, STOP low-to-high while SCL is high; after eight bits, the receiver provides ACK/NACK on the ninth clock.
418. FR - Comment utiliseriez-vous I²C open-drain, START/STOP et ACK dans un design VHDL réel ? Réponse : Je sépare `drive_low` de la lecture réelle des pins, car le master doit aussi observer les lignes pour ACK, arbitration ou clock stretching selon les fonctions supportées.	EN - How would you use I²C open-drain, START/STOP, and ACK in a real VHDL design? Answer: I separate `drive_low` control from actual pin reading because the master must also observe the lines for ACK, arbitration, or clock stretching depending on supported features.
419. FR - Comment diagnostiqueriez-vous un problème lié à I²C open-drain, START/STOP et ACK ? Réponse : Je vérifie timing, polarité, ordre des bits, sampling point et handshakes avec la logique interne. Une capture logic analyzer et une simulation du protocole doivent raconter la même histoire. Concernant I²C open-drain, START/STOP et ACK, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving I²C open-drain, START/STOP, and ACK? Answer: I verify timing, polarity, bit order, sampling point, and handshakes with internal logic. A logic-analyzer capture and protocol simulation should tell the same story. For I²C open-drain, START/STOP, and ACK, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
420. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant I²C open-drain, START/STOP et ACK ? Réponse : Je sépare le moteur série du protocole de haut niveau et j'utilise des interfaces `valid/ready` ou pulses clairement définies vers le reste du design. Pour I²C open-drain, START/STOP et ACK, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding I²C open-drain, START/STOP, and ACK? Answer: I separate the serial engine from the higher-level protocol and use clearly defined `valid/ready` interfaces or pulses toward the rest of the design. For I²C open-drain, START/STOP, and ACK, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

22. Testbenches, stimuli et modèles de vérification / Testbenches, Stimulus, and Verification Models

421. FR - Expliquez entity de testbench sans ports. Quelle structure ou idée hardware se cache derrière ? Réponse : Un testbench top-level n'a généralement pas de ports, car il crée lui-même clocks, reset, stimuli et observation autour du DUT. Il instancie ensuite le design under test comme sous-bloc.	EN - Explain portless testbench entity. What hardware structure or idea is behind it? Answer: A top-level testbench generally has no ports because it creates clocks, reset, stimuli, and observation around the DUT. It then instantiates the design under test as a sub-block.
422. FR - Comment utiliseriez-vous entity de testbench sans ports dans un design VHDL réel ? Réponse : Je déclare des signaux locaux équivalents aux ports du DUT puis mappe ces signaux vers l'instance.	EN - How would you use portless testbench entity in a real VHDL design? Answer: I declare local signals corresponding to DUT ports and then map those signals into the instance.

423. FR - Comment diagnostiqueriez-vous un problème lié à entity de testbench sans ports ? Réponse : Je vérifie que le testbench possède un arrêt clair, un reset réaliste et des checks automatiques. Une simulation qui montre seulement une waveform sans assertion est moins robuste qu'un test auto-vérifiant. Concernant entity de testbench sans ports, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving portless testbench entity? Answer: I verify that the testbench has a clear stop condition, realistic reset, and automatic checks. A simulation that only displays waveforms without assertions is less robust than a self-checking test. For portless testbench entity, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
424. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant entity de testbench sans ports ? Réponse : Je construis des tests reproductibles avec stimulus, assertions et cas limites. Le testbench n'a pas besoin d'être synthétisable et doit exploiter cette liberté pour vérifier efficacement le RTL. Pour entity de testbench sans ports, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding portless testbench entity? Answer: I build reproducible tests with stimulus, assertions, and corner cases. The testbench does not need to be synthesizable and should use that freedom to verify RTL effectively. For portless testbench entity, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
425. FR - Expliquez génération de clock en simulation. Quelle structure ou idée hardware se cache derrière ? Réponse : Un testbench peut utiliser `wait for` ou une affectation retardée pour créer une clock idéale. Ce code est destiné à la simulation et non à la synthèse du DUT.	EN - Explain clock generation in simulation. What hardware structure or idea is behind it? Answer: A testbench can use `wait for` or a delayed assignment to create an ideal clock. This code is intended for simulation, not DUT synthesis.
426. FR - Comment utiliseriez-vous génération de clock en simulation dans un design VHDL réel ? Réponse : Pour 100 MHz, je définis une période de 10 ns et bascule la clock toutes les 5 ns; je réutilise `CLK_PERIOD` dans tout le test. <pre>constant CLK_PERIOD : time := 10 ns; clk <= not clk after CLK_PERIOD/2;</pre>	EN - How would you use clock generation in simulation in a real VHDL design? Answer: For 100 MHz, I define a 10 ns period and toggle the clock every 5 ns; I reuse `CLK_PERIOD` throughout the test. <pre>constant CLK_PERIOD : time := 10 ns; clk <= not clk after CLK_PERIOD/2;</pre>
427. FR - Comment diagnostiqueriez-vous un problème lié à génération de clock en simulation ? Réponse : Je vérifie que le testbench possède un arrêt clair, un reset réaliste et des checks automatiques. Une simulation qui montre seulement une waveform sans assertion est moins robuste qu'un test auto-vérifiant. Concernant génération de clock en simulation, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock generation in simulation? Answer: I verify that the testbench has a clear stop condition, realistic reset, and automatic checks. A simulation that only displays waveforms without assertions is less robust than a self-checking test. For clock generation in simulation, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
428. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant génération de clock en simulation ? Réponse : Je construis des tests reproductibles avec stimulus, assertions et cas limites. Le testbench n'a pas besoin d'être synthétisable et doit exploiter cette liberté pour vérifier efficacement le RTL. Pour génération de clock en simulation, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock generation in simulation? Answer: I build reproducible tests with stimulus, assertions, and corner cases. The testbench does not need to be synthesizable and should use that freedom to verify RTL effectively. For clock generation in simulation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
429. FR - Expliquez stimulus process et wait. Quelle structure ou idée hardware se cache derrière ? Réponse : Un stimulus process peut appliquer reset, valeurs d'entrée et transactions avec `wait for` ou `wait until`. Comme il n'est pas synthétisé, il peut exprimer directement le temps de test.	EN - Explain stimulus process and wait. What hardware structure or idea is behind it? Answer: A stimulus process can apply reset, input values, and transactions using `wait for` or `wait until`. Because it is not synthesized, it can directly express test timing.
430. FR - Comment utiliseriez-vous stimulus process et wait dans un design VHDL réel ? Réponse : Je synchronise souvent le stimulus fonctionnel avec `wait until rising_edge(clk);` afin d'éviter des changements arbitraires juste autour d'un edge du DUT.	EN - How would you use stimulus process and wait in a real VHDL design? Answer: I often synchronize functional stimulus with `wait until rising_edge(clk);` to avoid arbitrary changes right around a DUT edge.

431. FR - Comment diagnostiqueriez-vous un problème lié à stimulus process et wait ? Réponse : Je vérifie que le testbench possède un arrêt clair, un reset réaliste et des checks automatiques. Une simulation qui montre seulement une waveform sans assertion est moins robuste qu'un test auto-vérifiant. Concernant stimulus process et wait, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving stimulus process and wait? Answer: I verify that the testbench has a clear stop condition, realistic reset, and automatic checks. A simulation that only displays waveforms without assertions is less robust than a self-checking test. For stimulus process and wait, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
432. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant stimulus process et wait ? Réponse : Je construis des tests reproductibles avec stimulus, assertions et cas limites. Le testbench n'a pas besoin d'être synthétisable et doit exploiter cette liberté pour vérifier efficacement le RTL. Pour stimulus process et wait, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding stimulus process and wait? Answer: I build reproducible tests with stimulus, assertions, and corner cases. The testbench does not need to be synthesizable and should use that freedom to verify RTL effectively. For stimulus process and wait, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
433. FR - Expliquez assertions self-checking. Quelle structure ou idée hardware se cache derrière ? Réponse : `assert` vérifie une condition et peut produire un message/severity lorsqu'elle est fausse. Un testbench self-checking compare automatiquement la sortie DUT à la valeur attendue.	EN - Explain self-checking assertions. What hardware structure or idea is behind it? Answer: `assert` checks a condition and can produce a message/severity when false. A self-checking testbench automatically compares DUT output with the expected value.
434. FR - Comment utiliseriez-vous assertions self-checking dans un design VHDL réel ? Réponse : Après une transaction UART, je peux écrire une assertion sur l'octet reçu et terminer la simulation en succès seulement si tous les checks passent.	EN - How would you use self-checking assertions in a real VHDL design? Answer: After a UART transaction, I can assert the received byte and end simulation successfully only if all checks pass.
435. FR - Comment diagnostiqueriez-vous un problème lié à assertions self-checking ? Réponse : Je vérifie que le testbench possède un arrêt clair, un reset réaliste et des checks automatiques. Une simulation qui montre seulement une waveform sans assertion est moins robuste qu'un test auto-vérifiant. Concernant assertions self-checking, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving self-checking assertions? Answer: I verify that the testbench has a clear stop condition, realistic reset, and automatic checks. A simulation that only displays waveforms without assertions is less robust than a self-checking test. For self-checking assertions, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
436. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant assertions self-checking ? Réponse : Je construis des tests reproductibles avec stimulus, assertions et cas limites. Le testbench n'a pas besoin d'être synthétisable et doit exploiter cette liberté pour vérifier efficacement le RTL. Pour assertions self-checking, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding self-checking assertions? Answer: I build reproducible tests with stimulus, assertions, and corner cases. The testbench does not need to be synthesizable and should use that freedom to verify RTL effectively. For self-checking assertions, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
437. FR - Expliquez corner cases et tests de frontières. Quelle structure ou idée hardware se cache derrière ? Réponse : Les tests utiles couvrent reset pendant activité, valeurs min/max, overflow, transitions simultanées, full/empty et conditions juste avant/après les limites. Le nominal seul ne suffit pas.	EN - Explain corner cases and boundary tests. What hardware structure or idea is behind it? Answer: Useful tests cover reset during activity, min/max values, overflow, simultaneous transitions, full/empty, and conditions just before/after boundaries. Nominal testing alone is insufficient.
438. FR - Comment utiliseriez-vous corner cases et tests de frontières dans un design VHDL réel ? Réponse : Pour un compteur terminal N, je teste N-1, N, wrap, enable à zéro, reset au milieu et plusieurs périodes consécutives afin de révéler les off-by-one.	EN - How would you use corner cases and boundary tests in a real VHDL design? Answer: For a terminal counter N, I test N-1, N, wraparound, enable low, mid-operation reset, and several consecutive periods to expose off-by-one errors.

439. FR - Comment diagnostiqueriez-vous un problème lié à corner cases et tests de frontières ? Réponse : Je vérifie que le testbench possède un arrêt clair, un reset réaliste et des checks automatiques. Une simulation qui montre seulement une waveform sans assertion est moins robuste qu'un test auto-vérifiant. Concernant corner cases et tests de frontières, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving corner cases and boundary tests? Answer: I verify that the testbench has a clear stop condition, realistic reset, and automatic checks. A simulation that only displays waveforms without assertions is less robust than a self-checking test. For corner cases and boundary tests, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
440. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant corner cases et tests de frontières ? Réponse : Je construis des tests reproductibles avec stimulus, assertions et cas limites. Le testbench n'a pas besoin d'être synthétisable et doit exploiter cette liberté pour vérifier efficacement le RTL. Pour corner cases et tests de frontières, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding corner cases and boundary tests? Answer: I build reproducible tests with stimulus, assertions, and corner cases. The testbench does not need to be synthesizable and should use that freedom to verify RTL effectively. For corner cases and boundary tests, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

23. Simulation, assertions, résolution et debug de waveform / Simulation, Assertions, Resolution, and Waveform Debug

441. FR - Expliquez valeurs U, X, Z et résolution std_logic. Quelle structure ou idée hardware se cache derrière ? Réponse : 'U' indique souvent un signal non initialisé, 'X' une valeur inconnue/conflit et 'Z' la haute impédance. La résolution de 'std_logic' combine les valeurs de plusieurs drivers selon une table définie.	EN - Explain U, X, Z values and std_logic resolution. What hardware structure or idea is behind it? Answer: 'U' often indicates an uninitialized signal, 'X' an unknown/contention condition, and 'Z' high impedance. 'std_logic' resolution combines multiple driver values using a defined table.
442. FR - Comment utiliseriez-vous valeurs U, X, Z et résolution std_logic dans un design VHDL réel ? Réponse : Quand un bus devient 'X', je cherche les drivers concurrents ou une source inconnue au premier instant où 'X' apparaît plutôt que de forcer arbitrairement une valeur.	EN - How would you use U, X, Z values and std_logic resolution in a real VHDL design? Answer: When a bus becomes 'X', I look for competing drivers or an unknown source at the first instant 'X' appears rather than arbitrarily forcing a value.
443. FR - Comment diagnostiqueriez-vous un problème lié à valeurs U, X, Z et résolution std_logic ? Réponse : Je pars de la première divergence temporelle, pas du dernier symptôme. Je remonte les drivers et vérifie 'U/X/Z', delta cycles, reset et événements d'horloge autour de cette divergence. Concernant valeurs U, X, Z et résolution std_logic, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving U, X, Z values and std_logic resolution? Answer: I start from the first time-domain divergence, not the final symptom. I trace drivers and inspect 'U/X/Z', delta cycles, reset, and clock events around that divergence. For U, X, Z values and std_logic resolution, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
444. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant valeurs U, X, Z et résolution std_logic ? Réponse : Je garde des waveforms lisibles avec groupes fonctionnels et noms explicites, mais complète l'inspection visuelle par des assertions lorsque le comportement peut être vérifié automatiquement. Pour valeurs U, X, Z et résolution std_logic, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding U, X, Z values and std_logic resolution? Answer: I keep waveforms readable with functional groups and explicit names, but complement visual inspection with assertions when behavior can be checked automatically. For U, X, Z values and std_logic resolution, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
445. FR - Expliquez initialisation en simulation. Quelle structure ou idée hardware se cache derrière ? Réponse : Sans initialisation ni reset, de nombreux signaux 'std_logic' commencent à 'U' en simulation. Une valeur initiale VHDL peut être supportée différemment en synthèse selon la cible.	EN - Explain simulation initialization. What hardware structure or idea is behind it? Answer: Without initialization or reset, many 'std_logic' signals begin as 'U' in simulation. A VHDL initial value may have different synthesis support depending on target.
446. FR - Comment utiliseriez-vous initialisation en simulation dans un design VHDL réel ? Réponse : Je teste le vrai comportement de power-up requis par le produit et ne masque pas un reset manquant simplement en initialisant tous les signaux dans le testbench.	EN - How would you use simulation initialization in a real VHDL design? Answer: I test the real product power-up behavior and do not hide a missing reset simply by initializing every signal in the testbench.

447. FR - Comment diagnostiqueriez-vous un problème lié à initialisation en simulation ? Réponse : Je pars de la première divergence temporelle, pas du dernier symptôme. Je remonte les drivers et vérifie `U/X/Z`, delta cycles, reset et événements d'horloge autour de cette divergence. Concernant initialisation en simulation, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving simulation initialization? Answer: I start from the first time-domain divergence, not the final symptom. I trace drivers and inspect `U/X/Z`, delta cycles, reset, and clock events around that divergence. For simulation initialization, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
448. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant initialisation en simulation ? Réponse : Je garde des waveforms lisibles avec groupes fonctionnels et noms explicites, mais complète l'inspection visuelle par des assertions lorsque le comportement peut être vérifié automatiquement. Pour initialisation en simulation, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding simulation initialization? Answer: I keep waveforms readable with functional groups and explicit names, but complement visual inspection with assertions when behavior can be checked automatically. For simulation initialization, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
449. FR - Expliquez assert severity levels. Quelle structure ou idée hardware se cache derrière ? Réponse : Une assertion peut associer une severity comme note, warning, error ou failure. La politique de simulation du projet décide quelles severities interrompent le run.	EN - Explain assert severity levels. What hardware structure or idea is behind it? Answer: An assertion can associate a severity such as note, warning, error, or failure. Project simulation policy decides which severities stop the run.
450. FR - Comment utiliseriez-vous assert severity levels dans un design VHDL réel ? Réponse : J'utilise `failure` pour une violation qui invalide le test et `warning` pour un événement toléré mais digne d'attention, avec messages décrivant valeur et contexte.	EN - How would you use assert severity levels in a real VHDL design? Answer: I use `failure` for a violation that invalidates the test and `warning` for a tolerated but noteworthy event, with messages describing value and context.
451. FR - Comment diagnostiqueriez-vous un problème lié à assert severity levels ? Réponse : Je pars de la première divergence temporelle, pas du dernier symptôme. Je remonte les drivers et vérifie `U/X/Z`, delta cycles, reset et événements d'horloge autour de cette divergence. Concernant assert severity levels, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving assert severity levels? Answer: I start from the first time-domain divergence, not the final symptom. I trace drivers and inspect `U/X/Z`, delta cycles, reset, and clock events around that divergence. For assert severity levels, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
452. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant assert severity levels ? Réponse : Je garde des waveforms lisibles avec groupes fonctionnels et noms explicites, mais complète l'inspection visuelle par des assertions lorsque le comportement peut être vérifié automatiquement. Pour assert severity levels, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding assert severity levels? Answer: I keep waveforms readable with functional groups and explicit names, but complement visual inspection with assertions when behavior can be checked automatically. For assert severity levels, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
453. FR - Expliquez debug par première divergence. Quelle structure ou idée hardware se cache derrière ? Réponse : Le dernier signal faux est souvent seulement une conséquence. La méthode efficace est de trouver le premier cycle où état, compteur, handshake ou donnée diverge de l'attendu.	EN - Explain debug by first divergence. What hardware structure or idea is behind it? Answer: The final wrong signal is often only a consequence. An effective method is to find the first cycle where state, counter, handshake, or data diverges from expected behavior.
454. FR - Comment utiliseriez-vous debug par première divergence dans un design VHDL réel ? Réponse : Je place state, next_state, counters, enables et valid/ready dans la waveform, puis remonte causalement vers l'entrée qui a créé la première divergence.	EN - How would you use debug by first divergence in a real VHDL design? Answer: I place state, next_state, counters, enables, and valid/ready in the waveform, then trace causally back to the input that created the first divergence.

455. FR - Comment diagnostiqueriez-vous un problème lié à debug par première divergence ? Réponse : Je pars de la première divergence temporelle, pas du dernier symptôme. Je remonte les drivers et vérifie `U/X/Z`, delta cycles, reset et événements d'horloge autour de cette divergence. Concernant debug par première divergence, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving debug by first divergence? Answer: I start from the first time-domain divergence, not the final symptom. I trace drivers and inspect `U/X/Z`, delta cycles, reset, and clock events around that divergence. For debug by first divergence, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
456. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant debug par première divergence ? Réponse : Je garde des waveforms lisibles avec groupes fonctionnels et noms explicites, mais complète l'inspection visuelle par des assertions lorsque le comportement peut être vérifié automatiquement. Pour debug par première divergence, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding debug by first divergence? Answer: I keep waveforms readable with functional groups and explicit names, but complement visual inspection with assertions when behavior can be checked automatically. For debug by first divergence, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
457. FR - Expliquez simulation RTL versus post-synthesis/post-route. Quelle structure ou idée hardware se cache derrière ? Réponse : La simulation RTL vérifie surtout la fonction abstraite. Une simulation post-synthèse/post-route peut inclure netlist et parfois délais, utile pour certains besoins mais plus lourde.	EN - Explain RTL versus post-synthesis/post-route simulation. What hardware structure or idea is behind it? Answer: RTL simulation primarily checks abstract function. Post-synthesis/post-route simulation can include the netlist and sometimes delays, useful for certain needs but heavier.
458. FR - Comment utiliseriez-vous simulation RTL versus post-synthesis/post-route dans un design VHDL réel ? Réponse : Je corrige les erreurs fonctionnelles au RTL, utilise static timing analysis pour le timing normal et réserve les simulations plus basses couches aux besoins spécifiques du flow.	EN - How would you use RTL versus post-synthesis/post-route simulation in a real VHDL design? Answer: I fix functional errors at RTL, use static timing analysis for normal timing verification, and reserve lower-level simulations for specific flow needs.
459. FR - Comment diagnostiqueriez-vous un problème lié à simulation RTL versus post-synthesis/post-route ? Réponse : Je pars de la première divergence temporelle, pas du dernier symptôme. Je remonte les drivers et vérifie `U/X/Z`, delta cycles, reset et événements d'horloge autour de cette divergence. Concernant simulation RTL versus post-synthesis/post-route, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving RTL versus post-synthesis/post-route simulation? Answer: I start from the first time-domain divergence, not the final symptom. I trace drivers and inspect `U/X/Z`, delta cycles, reset, and clock events around that divergence. For RTL versus post-synthesis/post-route simulation, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
460. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant simulation RTL versus post-synthesis/post-route ? Réponse : Je garde des waveforms lisibles avec groupes fonctionnels et noms explicites, mais complète l'inspection visuelle par des assertions lorsque le comportement peut être vérifié automatiquement. Pour simulation RTL versus post-synthesis/post-route, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding RTL versus post-synthesis/post-route simulation? Answer: I keep waveforms readable with functional groups and explicit names, but complement visual inspection with assertions when behavior can be checked automatically. For RTL versus post-synthesis/post-route simulation, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

24. Synthèse, timing, contraintes et portabilité / Synthesis, Timing, Constraints, and Portability

461. FR - Expliquez synthesizable versus simulation-only VHDL. Quelle structure ou idée hardware se cache derrière ? Réponse : Tout VHDL légal n'est pas forcément synthétisable. Des constructs comme délais `after`, beaucoup de `wait for`, allocation dynamique ou modèles très abstraits sont typiquement destinés à la simulation plutôt qu'au RTL matériel.	EN - Explain synthesizable versus simulation-only VHDL. What hardware structure or idea is behind it? Answer: Not all legal VHDL is necessarily synthesizable. Constructs such as `after` delays, many `wait for` statements, dynamic allocation, or highly abstract models are typically intended for simulation rather than hardware RTL.
462. FR - Comment utiliseriez-vous synthesizable versus simulation-only VHDL dans un design VHDL réel ? Réponse : Je garde le DUT dans un sous-ensemble RTL clairement synthétisable et place le code temporel de stimulus dans le testbench.	EN - How would you use synthesizable versus simulation-only VHDL in a real VHDL design? Answer: I keep the DUT within a clearly synthesizable RTL subset and place time-based stimulus code in the testbench.

463. FR - Comment diagnostiqueriez-vous un problème lié à synthesizable versus simulation-only VHDL ? Réponse : Je sépare trois questions: le RTL synthétise-t-il, la structure inférée est-elle celle voulue, et le timing est-il fermé avec les bonnes contraintes? Un design qui compile n'est pas automatiquement correct ou rapide. Concernant synthesizable versus simulation-only VHDL, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving synthesizable versus simulation-only VHDL? Answer: I separate three questions: does RTL synthesize, is the inferred structure what was intended, and is timing closed under correct constraints? A design that compiles is not automatically correct or fast. For synthesizable versus simulation-only VHDL, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
464. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant synthesizable versus simulation-only VHDL ? Réponse : Je lis les warnings de synthèse, vérifie resources/clock reports et traite les contraintes comme aussi importantes que le RTL. La portabilité vient d'un style standard et de l'isolement des primitives vendor-specific. Pour synthesizable versus simulation-only VHDL, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding synthesizable versus simulation-only VHDL? Answer: I read synthesis warnings, verify resource/clock reports, and treat constraints as important as RTL. Portability comes from standard coding style and isolation of vendor-specific primitives. For synthesizable versus simulation-only VHDL, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
465. FR - Expliquez inference versus primitives vendor-specific. Quelle structure ou idée hardware se cache derrière ? Réponse : L'inference décrit un comportement standard que le synthétiseur mappe vers les ressources disponibles; l'instanciation de primitive demande explicitement une ressource vendor-specific.	EN - Explain inference versus vendor-specific primitives. What hardware structure or idea is behind it? Answer: Inference describes standard behavior that synthesis maps to available resources; primitive instantiation explicitly requests a vendor-specific resource.
466. FR - Comment utiliseriez-vous inference versus primitives vendor-specific dans un design VHDL réel ? Réponse : J'infère registres, RAM et arithmétique lorsque le style est bien supporté, mais isole PLL, transceivers ou I/O spécialisés dans des wrappers vendor-specific si nécessaire.	EN - How would you use inference versus vendor-specific primitives in a real VHDL design? Answer: I infer registers, RAM, and arithmetic when well supported, but isolate PLLs, transceivers, or specialized I/O in vendor-specific wrappers when necessary.
467. FR - Comment diagnostiqueriez-vous un problème lié à inference versus primitives vendor-specific ? Réponse : Je sépare trois questions: le RTL synthétise-t-il, la structure inférée est-elle celle voulue, et le timing est-il fermé avec les bonnes contraintes? Un design qui compile n'est pas automatiquement correct ou rapide. Concernant inference versus primitives vendor-specific, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving inference versus vendor-specific primitives? Answer: I separate three questions: does RTL synthesize, is the inferred structure what was intended, and is timing closed under correct constraints? A design that compiles is not automatically correct or fast. For inference versus vendor-specific primitives, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
468. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant inference versus primitives vendor-specific ? Réponse : Je lis les warnings de synthèse, vérifie resources/clock reports et traite les contraintes comme aussi importantes que le RTL. La portabilité vient d'un style standard et de l'isolement des primitives vendor-specific. Pour inference versus primitives vendor-specific, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding inference versus vendor-specific primitives? Answer: I read synthesis warnings, verify resource/clock reports, and treat constraints as important as RTL. Portability comes from standard coding style and isolation of vendor-specific primitives. For inference versus vendor-specific primitives, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
469. FR - Expliquez setup, hold et chemin critique. Quelle structure ou idée hardware se cache derrière ? Réponse : Setup exige que la donnée arrive assez avant l'edge de capture; hold exige qu'elle reste stable assez après. Le chemin critique est le chemin limitant fréquence ou slack dans l'analyse de timing.	EN - Explain setup, hold, and critical path. What hardware structure or idea is behind it? Answer: Setup requires data to arrive sufficiently before the capture edge; hold requires it to remain stable sufficiently after. The critical path is the path limiting frequency or slack in timing analysis.
470. FR - Comment utiliseriez-vous setup, hold et chemin critique dans un design VHDL réel ? Réponse : Si setup échoue, je cherche profondeur combinatoire, fanout, routing ou mauvaise contrainte et j'envisage pipeline/retiming architectural plutôt qu'une optimisation syntaxique superficielle.	EN - How would you use setup, hold, and critical path in a real VHDL design? Answer: If setup fails, I look for combinational depth, fanout, routing, or incorrect constraints and consider architectural pipelining/retiming rather than superficial syntax optimization.

471. FR - Comment diagnostiqueriez-vous un problème lié à setup, hold et chemin critique ? Réponse : Je sépare trois questions: le RTL synthétise-t-il, la structure inférée est-elle celle voulue, et le timing est-il fermé avec les bonnes contraintes? Un design qui compile n'est pas automatiquement correct ou rapide. Concernant setup, hold et chemin critique, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving setup, hold, and critical path? Answer: I separate three questions: does RTL synthesize, is the inferred structure what was intended, and is timing closed under correct constraints? A design that compiles is not automatically correct or fast. For setup, hold, and critical path, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
472. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant setup, hold et chemin critique ? Réponse : Je lis les warnings de synthèse, vérifie resources/clock reports et traite les contraintes comme aussi importantes que le RTL. La portabilité vient d'un style standard et de l'isolement des primitives vendor-specific. Pour setup, hold et chemin critique, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding setup, hold, and critical path? Answer: I read synthesis warnings, verify resource/clock reports, and treat constraints as important as RTL. Portability comes from standard coding style and isolation of vendor-specific primitives. For setup, hold, and critical path, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
473. FR - Expliquez clock constraints et fréquence déclarée. Quelle structure ou idée hardware se cache derrière ? Réponse : Le timing analyzer doit connaître les clocks et leurs relations. Sans contrainte correcte, un rapport 'timing met' peut être sans valeur car l'outil ne vérifie pas la fréquence réellement requise.	EN - Explain clock constraints and declared frequency. What hardware structure or idea is behind it? Answer: The timing analyzer must know clocks and their relationships. Without correct constraints, a 'timing met' report can be meaningless because the tool may not check the actually required frequency.
474. FR - Comment utiliseriez-vous clock constraints et fréquence déclarée dans un design VHDL réel ? Réponse : Je vérifie que chaque clock primaire/générée est déclarée avec la bonne période et que les crossings asynchrones sont identifiés par les contraintes adaptées.	EN - How would you use clock constraints and declared frequency in a real VHDL design? Answer: I verify that each primary/generated clock is declared with the correct period and asynchronous crossings are identified by appropriate constraints.
475. FR - Comment diagnostiqueriez-vous un problème lié à clock constraints et fréquence déclarée ? Réponse : Je sépare trois questions: le RTL synthétise-t-il, la structure inférée est-elle celle voulue, et le timing est-il fermé avec les bonnes contraintes? Un design qui compile n'est pas automatiquement correct ou rapide. Concernant clock constraints et fréquence déclarée, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving clock constraints and declared frequency? Answer: I separate three questions: does RTL synthesize, is the inferred structure what was intended, and is timing closed under correct constraints? A design that compiles is not automatically correct or fast. For clock constraints and declared frequency, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
476. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant clock constraints et fréquence déclarée ? Réponse : Je lis les warnings de synthèse, vérifie resources/clock reports et traite les contraintes comme aussi importantes que le RTL. La portabilité vient d'un style standard et de l'isolement des primitives vendor-specific. Pour clock constraints et fréquence déclarée, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding clock constraints and declared frequency? Answer: I read synthesis warnings, verify resource/clock reports, and treat constraints as important as RTL. Portability comes from standard coding style and isolation of vendor-specific primitives. For clock constraints and declared frequency, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
477. FR - Expliquez false path, multicycle et prudence. Quelle structure ou idée hardware se cache derrière ? Réponse : Une false path exclut un chemin de l'analyse parce qu'il n'a pas à respecter une relation de timing normale; une multicycle path autorise plusieurs cycles. Une mauvaise exception peut masquer un vrai bug.	EN - Explain false paths, multicycle paths, and caution. What hardware structure or idea is behind it? Answer: A false path excludes a path from analysis because it need not meet a normal timing relationship; a multicycle path allows several cycles. A bad exception can hide a real bug.
478. FR - Comment utiliseriez-vous false path, multicycle et prudence dans un design VHDL réel ? Réponse : Je n'ajoute jamais une exception seulement pour faire disparaître une violation; je documente la raison fonctionnelle, les endpoints et la preuve que le chemin est réellement false ou multicycle.	EN - How would you use false paths, multicycle paths, and caution in a real VHDL design? Answer: I never add an exception merely to remove a violation; I document the functional reason, endpoints, and evidence that the path is truly false or multicycle.

479. FR - Comment diagnostiqueriez-vous un problème lié à false path, multicycle et prudence ? Réponse : Je sépare trois questions: le RTL synthétise-t-il, la structure inférée est-elle celle voulue, et le timing est-il fermé avec les bonnes contraintes? Un design qui compile n'est pas automatiquement correct ou rapide. Concernant false path, multicycle et prudence, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving false paths, multicycle paths, and caution? Answer: I separate three questions: does RTL synthesize, is the inferred structure what was intended, and is timing closed under correct constraints? A design that compiles is not automatically correct or fast. For false paths, multicycle paths, and caution, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
480. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant false path, multicycle et prudence ? Réponse : Je lis les warnings de synthèse, vérifie resources/clock reports et traite les contraintes comme aussi importantes que le RTL. La portabilité vient d'un style standard et de l'isolement des primitives vendor-specific. Pour false path, multicycle et prudence, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding false paths, multicycle paths, and caution? Answer: I read synthesis warnings, verify resource/clock reports, and treat constraints as important as RTL. Portability comes from standard coding style and isolation of vendor-specific primitives. For false paths, multicycle paths, and caution, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

25. Troubleshooting et scénarios d'entretien / Troubleshooting and Interview Design Scenarios

481. FR - Expliquez design compile mais ne fonctionne pas sur la carte. Quelle structure ou idée hardware se cache derrière ? Réponse : Un build réussi prouve surtout que syntaxe/synthèse/implémentation ont terminé, pas que la fonction réelle est correcte. Les causes peuvent être pin mapping, reset, clock, contraintes, niveau électrique, protocole ou hypothèse de test.	EN - Explain design compiles but does not work on the board. What hardware structure or idea is behind it? Answer: A successful build mainly proves syntax/synthesis/implementation completed, not that real function is correct. Causes may include pin mapping, reset, clock, constraints, electrical level, protocol, or test assumptions.
482. FR - Comment utiliseriez-vous design compile mais ne fonctionne pas sur la carte dans un design VHDL réel ? Réponse : Je vérifie alimentation/clock/reset, pin constraints, puis expose un heartbeat ou des signaux internes pour voir si la FSM tourne et si les entrées arrivent réellement.	EN - How would you use design compiles but does not work on the board in a real VHDL design? Answer: I verify power/clock/reset and pin constraints, then expose a heartbeat or internal signals to see whether the FSM is running and inputs actually arrive.
483. FR - Comment diagnostiqueriez-vous un problème lié à design compile mais ne fonctionne pas sur la carte ? Réponse : Je décompose le problème: spécification, stimulus, logique RTL, reset/clock, CDC, synthèse, contraintes, puis hardware. Je cherche une preuve mesurable à chaque étape et la première divergence observable. Concernant design compile mais ne fonctionne pas sur la carte, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving design compiles but does not work on the board? Answer: I decompose the problem into specification, stimulus, RTL logic, reset/clock, CDC, synthesis, constraints, then hardware. I look for measurable evidence at each step and the first observable divergence. For design compiles but does not work on the board, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
484. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant design compile mais ne fonctionne pas sur la carte ? Réponse : En entretien, j'explique ma méthode avant de proposer un patch. Je montre que je peux relier une ligne VHDL à une structure matérielle, une waveform, un timing path et une mesure sur la carte. Pour design compile mais ne fonctionne pas sur la carte, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding design compiles but does not work on the board? Answer: In an interview, I explain my method before proposing a patch. I show that I can connect a line of VHDL to a hardware structure, waveform, timing path, and board-level measurement. For design compiles but does not work on the board, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
485. FR - Expliquez simulation correcte mais hardware intermittent. Quelle structure ou idée hardware se cache derrière ? Réponse : Un comportement intermittent absent de simulation idéale fait penser à CDC, metastability, reset release, timing marginal, bruit électrique ou hypothèses non modélisées.	EN - Explain simulation correct but hardware intermittent. What hardware structure or idea is behind it? Answer: Intermittent behavior absent from ideal simulation suggests CDC, metastability, reset release, marginal timing, electrical noise, or unmodeled assumptions.
486. FR - Comment utiliseriez-vous simulation correcte mais hardware intermittent dans un design VHDL réel ? Réponse : Je collecte timing reports, CDC review et captures internes/externes, puis stresse fréquence, température ou cadence d'événements pour rendre le problème reproductible sans masquer sa cause.	EN - How would you use simulation correct but hardware intermittent in a real VHDL design? Answer: I collect timing reports, CDC review, and internal/external captures, then stress frequency, temperature, or event rate to make the problem reproducible without hiding the cause.

487. FR - Comment diagnostiqueriez-vous un problème lié à simulation correcte mais hardware intermittent ? Réponse : Je décompose le problème: spécification, stimulus, logique RTL, reset/clock, CDC, synthèse, contraintes, puis hardware. Je cherche une preuve mesurable à chaque étape et la première divergence observable. Concernant simulation correcte mais hardware intermittent, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving simulation correct but hardware intermittent? Answer: I decompose the problem into specification, stimulus, RTL logic, reset/clock, CDC, synthesis, constraints, then hardware. I look for measurable evidence at each step and the first observable divergence. For simulation correct but hardware intermittent, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
488. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant simulation correcte mais hardware intermittent ? Réponse : En entretien, j'explique ma méthode avant de proposer un patch. Je montre que je peux relier une ligne VHDL à une structure matérielle, une waveform, un timing path et une mesure sur la carte. Pour simulation correcte mais hardware intermittent, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding simulation correct but hardware intermittent? Answer: In an interview, I explain my method before proposing a patch. I show that I can connect a line of VHDL to a hardware structure, waveform, timing path, and board-level measurement. For simulation correct but hardware intermittent, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
489. FR - Expliquez latch involontaire détecté en synthèse. Quelle structure ou idée hardware se cache derrière ? Réponse : Un latch report dans un bloc censé combinatoire indique souvent qu'une sortie ou variable n'est pas affectée sur tous les chemins. Le hardware doit alors mémoriser l'ancienne valeur.	EN - Explain unintended latch reported by synthesis. What hardware structure or idea is behind it? Answer: A latch report in logic intended to be combinational usually means an output or variable is not assigned on every path. Hardware must then remember the old value.
490. FR - Comment utiliseriez-vous latch involontaire détecté en synthèse dans un design VHDL réel ? Réponse : Je donne une valeur par défaut ou couvre toutes les branches, puis re-simule les combinaisons qui auparavant laissaient la valeur inchangée.	EN - How would you use unintended latch reported by synthesis in a real VHDL design? Answer: I provide a default assignment or cover every branch, then re-simulate the combinations that previously left the value unchanged.
491. FR - Comment diagnostiqueriez-vous un problème lié à latch involontaire détecté en synthèse ? Réponse : Je décompose le problème: spécification, stimulus, logique RTL, reset/clock, CDC, synthèse, contraintes, puis hardware. Je cherche une preuve mesurable à chaque étape et la première divergence observable. Concernant latch involontaire détecté en synthèse, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving unintended latch reported by synthesis? Answer: I decompose the problem into specification, stimulus, RTL logic, reset/clock, CDC, synthesis, constraints, then hardware. I look for measurable evidence at each step and the first observable divergence. For unintended latch reported by synthesis, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
492. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant latch involontaire détecté en synthèse ? Réponse : En entretien, j'explique ma méthode avant de proposer un patch. Je montre que je peux relier une ligne VHDL à une structure matérielle, une waveform, un timing path et une mesure sur la carte. Pour latch involontaire détecté en synthèse, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding unintended latch reported by synthesis? Answer: In an interview, I explain my method before proposing a patch. I show that I can connect a line of VHDL to a hardware structure, waveform, timing path, and board-level measurement. For unintended latch reported by synthesis, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
493. FR - Expliquez timing failure après ajout d'une fonction. Quelle structure ou idée hardware se cache derrière ? Réponse : Une nouvelle comparaison, arithmetic chain, large mux ou fanout peut rallonger le chemin critique. Le bon correctif est souvent architectural: pipeline, partition, register duplication ou calcul multi-cycle.	EN - Explain timing failure after adding a feature. What hardware structure or idea is behind it? Answer: A new comparison, arithmetic chain, large mux, or fanout can lengthen the critical path. The right fix is often architectural: pipelining, partitioning, register duplication, or multi-cycle computation.
494. FR - Comment utiliseriez-vous timing failure après ajout d'une fonction dans un design VHDL réel ? Réponse : Je lis le chemin exact startpoint->endpoint et les contributions logic/routing, puis modifie la structure au lieu de simplement réordonner le code.	EN - How would you use timing failure after adding a feature in a real VHDL design? Answer: I read the exact startpoint-to-endpoint path and logic/routing contributions, then change the structure rather than merely reordering code.

495. FR - Comment diagnostiqueriez-vous un problème lié à timing failure après ajout d'une fonction ? Réponse : Je décompose le problème: spécification, stimulus, logique RTL, reset/clock, CDC, synthèse, contraintes, puis hardware. Je cherche une preuve mesurable à chaque étape et la première divergence observable. Concernant timing failure après ajout d'une fonction, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving timing failure after adding a feature? Answer: I decompose the problem into specification, stimulus, RTL logic, reset/clock, CDC, synthesis, constraints, then hardware. I look for measurable evidence at each step and the first observable divergence. For timing failure after adding a feature, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
496. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant timing failure après ajout d'une fonction ? Réponse : En entretien, j'explique ma méthode avant de proposer un patch. Je montre que je peux relier une ligne VHDL à une structure matérielle, une waveform, un timing path et une mesure sur la carte. Pour timing failure après ajout d'une fonction, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding timing failure after adding a feature? Answer: In an interview, I explain my method before proposing a patch. I show that I can connect a line of VHDL to a hardware structure, waveform, timing path, and board-level measurement. For timing failure after adding a feature, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.
497. FR - Expliquez exercice d'entretien: besoin vers RTL vérifiable. Quelle structure ou idée hardware se cache derrière ? Réponse : Pour 'compter des pulses externes et déclencher une sortie après N événements', je décompose en synchronizer, edge detector, counter, comparator et output register/FSM. Chaque bloc a reset et cas limites.	EN - Explain interview exercise: requirement to verifiable RTL. What hardware structure or idea is behind it? Answer: For 'count external pulses and trigger an output after N events,' I decompose into synchronizer, edge detector, counter, comparator, and output register/FSM. Each block has reset and corner cases.
498. FR - Comment utiliseriez-vous exercice d'entretien: besoin vers RTL vérifiable dans un design VHDL réel ? Réponse : En entretien, je clarifie fréquence d'entrée, relation au clock, largeur N, comportement reset/retrigger; ensuite je dessine datapath/control, écris RTL et propose un testbench avec pulses proches, reset et overflow.	EN - How would you use interview exercise: requirement to verifiable RTL in a real VHDL design? Answer: In an interview, I clarify input frequency, relation to clock, N width, reset/retrigger behavior; then sketch datapath/control, write RTL, and propose a testbench with close pulses, reset, and overflow.
499. FR - Comment diagnostiqueriez-vous un problème lié à exercice d'entretien: besoin vers RTL vérifiable ? Réponse : Je décompose le problème: spécification, stimulus, logique RTL, reset/clock, CDC, synthèse, contraintes, puis hardware. Je cherche une preuve mesurable à chaque étape et la première divergence observable. Concernant exercice d'entretien: besoin vers RTL vérifiable, je reviens ensuite au signal ou à la construction précise et je vérifie types, largeur, drivers, waveform et hardware inféré.	EN - How would you troubleshoot an issue involving interview exercise: requirement to verifiable RTL? Answer: I decompose the problem into specification, stimulus, RTL logic, reset/clock, CDC, synthesis, constraints, then hardware. I look for measurable evidence at each step and the first observable divergence. For interview exercise: requirement to verifiable RTL, I then return to the exact signal or construct and verify types, width, drivers, waveform, and inferred hardware.
500. FR - Quelle bonne pratique ou quel piège d'entretien faut-il connaître concernant exercice d'entretien: besoin vers RTL vérifiable ? Réponse : En entretien, j'explique ma méthode avant de proposer un patch. Je montre que je peux relier une ligne VHDL à une structure matérielle, une waveform, un timing path et une mesure sur la carte. Pour exercice d'entretien: besoin vers RTL vérifiable, j'explique aussi les hypothèses, les cas limites et le résultat matériel attendu afin que la revue puisse vérifier l'intention.	EN - What best practice or interview trap should you know regarding interview exercise: requirement to verifiable RTL? Answer: In an interview, I explain my method before proposing a patch. I show that I can connect a line of VHDL to a hardware structure, waveform, timing path, and board-level measurement. For interview exercise: requirement to verifiable RTL, I also explain assumptions, corner cases, and the expected hardware result so a reviewer can verify intent.

Rappels essentiels / Essential Reminders

FR: VHDL décrit du matériel concurrent; pensez en registres, logique combinatoire, mémoire, horloges et interfaces. EN: VHDL describes concurrent hardware; think in registers, combinational logic, memory, clocks, and interfaces.

FR: Un port logique n'est pas un pin physique. Le pin mapping et les standards I/O appartiennent aux contraintes de projet. EN: A logical port is not a physical pin. Pin mapping and I/O standards belong to project constraints.

FR: Pour l'arithmétique, privilégiez IEEE `numeric\_std` avec `unsigned/signed` et des largeurs explicites. EN: For arithmetic, prefer IEEE `numeric\_std` with `unsigned/signed` and explicit widths.

FR: Un signal asynchrone doit être traité comme un CDC; un bus multi-bit ne se synchronise pas bit par bit sans protocole. EN: An asynchronous signal must be treated as CDC; a multi-bit bus is not synchronized bit by bit without a protocol.

FR: Simulation fonctionnelle, synthèse et static timing analysis répondent à des questions différentes et sont toutes nécessaires. EN: Functional simulation, synthesis, and static timing analysis answer different questions and are all necessary.